



National Institute of Electronics & Information Technology
Gorakhpur Center

Ministry of Electronics & Information Technology (MeitY), Government of India

Data Science using Python Programming

Unit-01: < Introduction to Python >

Contents to be covered

1. Introduction to Python
2. Varieties of Python
3. Installation of Python
4. Setting Environment Variable
5. Basic Data Types
6. First Python Program

Introduction to Python

- It was created by Guido van Rossum in Netherland, and released in 1991.
- The name "Python" was adopted from the Rossum's favourite comedy series "Monty Python's Flying Circus".
- It is a high level general purpose programming language.
- It is compiled to byte code and executed in Python Virtual Machine.
- It is suitable for use as a scripting language, Web application implementation language, etc.

Introduction to Python

- It has a strong structuring constructs (nested code blocks, functions, classes, modules, and packages) and the use of objects and object oriented programming, enables us to write clear, logical applications for small and large tasks.
- Python is interpreted language.
- Python is available as Free and Open Source.
- Python run on different platform like Windows , Linux , Unix etc

Varieties(Flavors) of Python:

Flavors of python refer to the different types of python compilers.

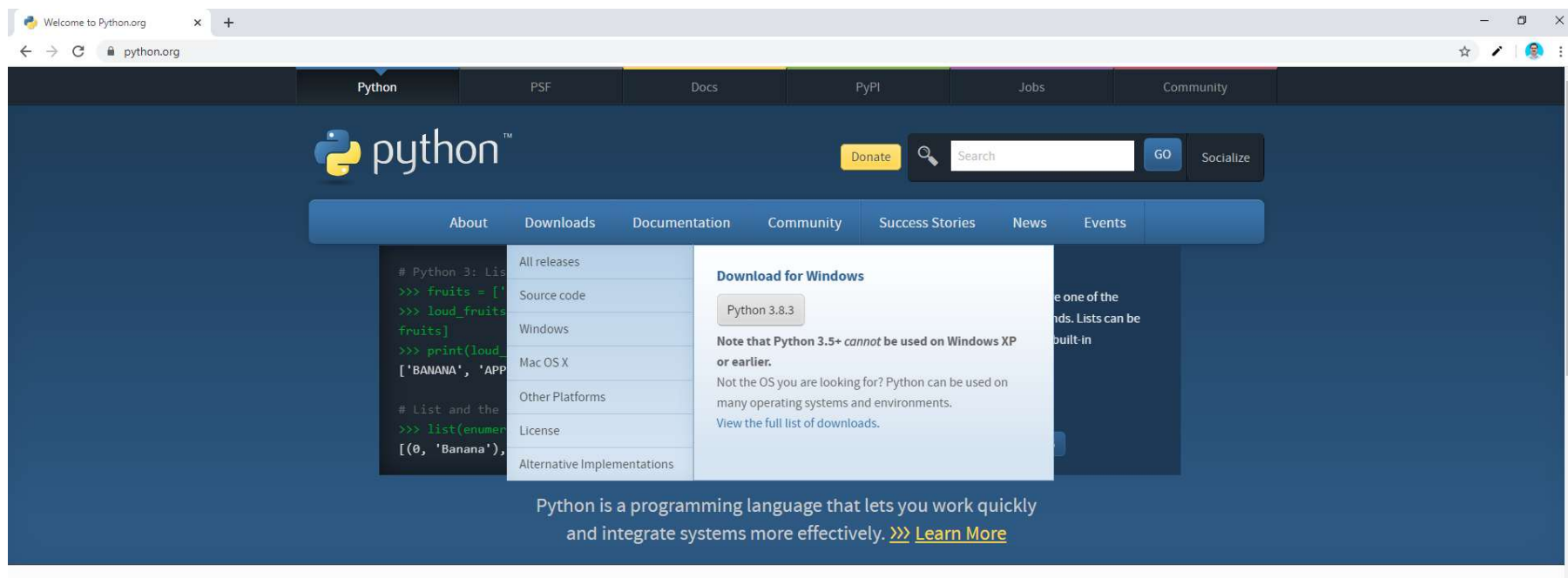
These are useful to integrate various programming languages into python.

. CPython Standard Python 2.x implemented in C.(python programs run in c)

- Jython Python programs run in java environment <http://www.jython.org/>
- PyPy Python with a JIT compiler and stackless mode <http://pypy.org/>(written in python)
- Stackless Python with enhanced thread support and microthreads etc. <http://www.stackless.com/>(tasklets->threads->process)
- IronPython Python for .NET and the CLR <http://ironpython.net/>
- Python 3 – The new, new Python. This is intended as a replacement for Python 2.x. <http://www.python.org/doc/>

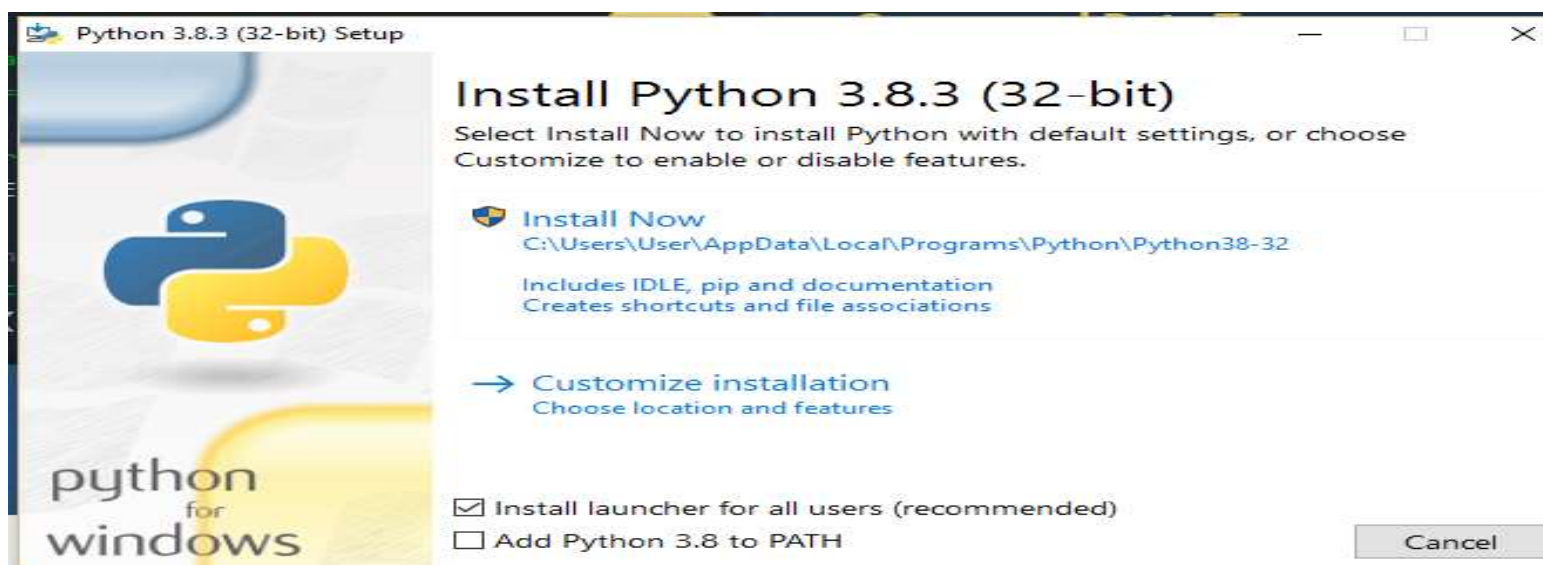
Installation of Python on Windows OS

- Download the up-to-date source code, binaries, documentation available on the official website of Python <https://www.python.org/>.



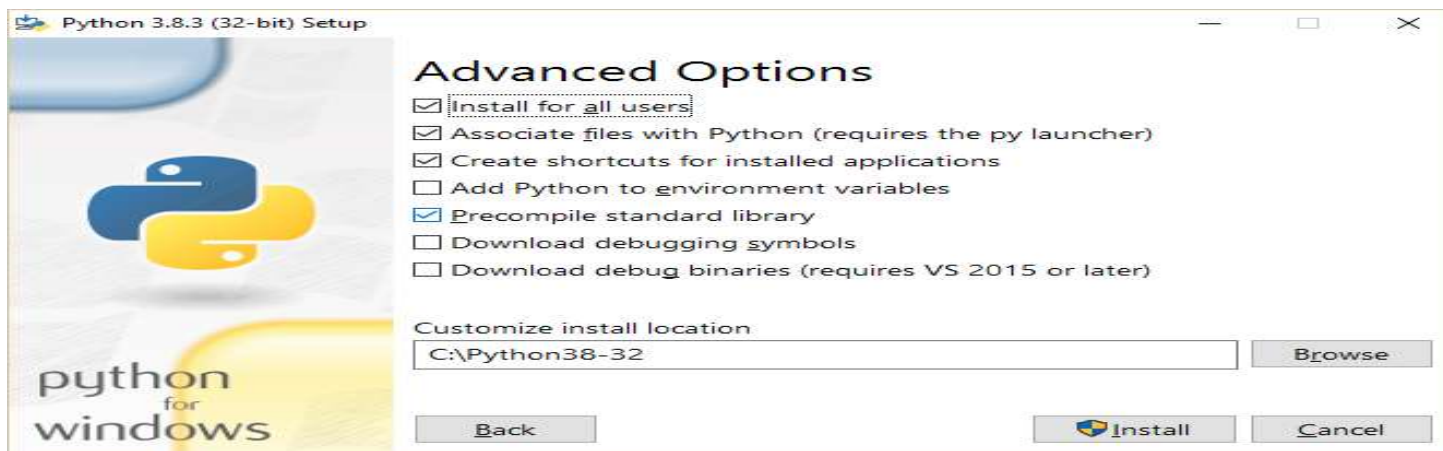
Installation of Python on Windows OS – cont.

- Python installer downloaded on to your system - Python-3.8.3.exe
- Click this icon and a installer screen shown as below



Installation of Python on Windows OS – cont.

- Default option will also work, it automatically install the software at the location, it is showing.
- We can select the customize installation option, to install the software at the user specified location. - C:\Python38-32



Setting Environment Variable

- To add the Python directory to the path for a particular session in Windows –
 - At the command prompt, type
 - path %path%;C:\Python38-32 and press Enter.
 - Where C:\Python38-32 is the path of the Python directory.
- You can also select the – **“Add Python to Environment Variable”** , while installing the Python.

- After setting the path variable

C:\>python

Python 3.8.3 (tags/v3.8.3:6f8c832, May 13 2020, 22:20:19) [MSC v.1925 32 bit (Intel)] on win32

Type "help", "copyright", "credits" or "license" for more information.

>>>

Execution of a python program

- Python file first get compiled to give us byte code and that byte code is interpreted into machine language.
- This is performed by PVM.
- Execution:



- x.py x.pyc x.exe

Viewing the Byte code

- After the execution of a program, we can not find any .pyc file .
- We should specify the **dis module** while using python command.
- C:\> python -m dis x.py

```
C:\Users\ekata\Desktop\Python>python -m dis first.py
1      0 LOAD_CONST           0 (10)
      2 STORE_NAME           0 (a)

2      4 LOAD_CONST           0 (10)
      6 STORE_NAME           1 (b)

3      8 LOAD_NAME              2 (print)
     10 LOAD_CONST           1 ('Sum ')
     12 LOAD_NAME              0 (a)
     14 LOAD_NAME              1 (b)
     16 BINARY_ADD
     18 CALL_FUNCTION          2
     20 POP_TOP
     22 LOAD_CONST           2 (None)
     24 RETURN_VALUE
```

Byte code in an understandable format

- The **dis** command is known as “disassembler” that displays the byte code in an understandable format. The code represents 5 columns:

- 1.Line Number
- 2.offset position of byte code
- 3.name of byte code instruction
- 4.instruction's argument
- 5.constants or names (in brackets)

Python Programming

- Comments – Everything after # on a line is ignored.
- Blocks and Indentation – It follows the block structure and nested block structure with indentation.

ABC Block-1

ABC Block-2

ABC Block-3

- Lines – Statement separator is a semi colon, but needed only when there are more than one statement on a line.
- File Extension- Program have the File Extension “.py”.

Python Programming

Creating Variables

- Variables are containers for storing data values.
- Python has no command for declaring a variable.
- A variable is created at the time, first value assign to it.
 - `x= 5` # Numeric Variable
 - `y= "Ajay"` # Character Variable
 - `print(x)`
 - `print(y)`

Python Programming

Rules for Variable Naming

- A variable can have a short name (like x and y) or a more descriptive name (age, EmpName, total_volume).
- Rules for Python variables:
 - A variable name must start with a letter or the underscore character
 - A variable name cannot start with a number
 - A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _)
 - Variable names are case-sensitive (age, Age and AGE are three different variables)

Python Programming

Reserve Words – It can not be used as constant or any other identifier name.

and	exec	not	assert	finally	or	break
for	pass	class	from		continue	
global	raise	def	if	return	del	import
try	elif	in	while	else	is	with
except	yield	lambda				

List of the Python keywords

```
>>>help("keywords")
```

- False class from or None continue global
pass
- True def if raise and del import
return
- as elif in try assert else is
while
- async except lambda with await finally
nonlocal yield
- break for not

Another way of access to the Python keywords:

- `>>>import keyword`
- `>>> keyword.kwlist`
- `['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']`
- `>>> print(len(keyword.kwlist))#total keywords`

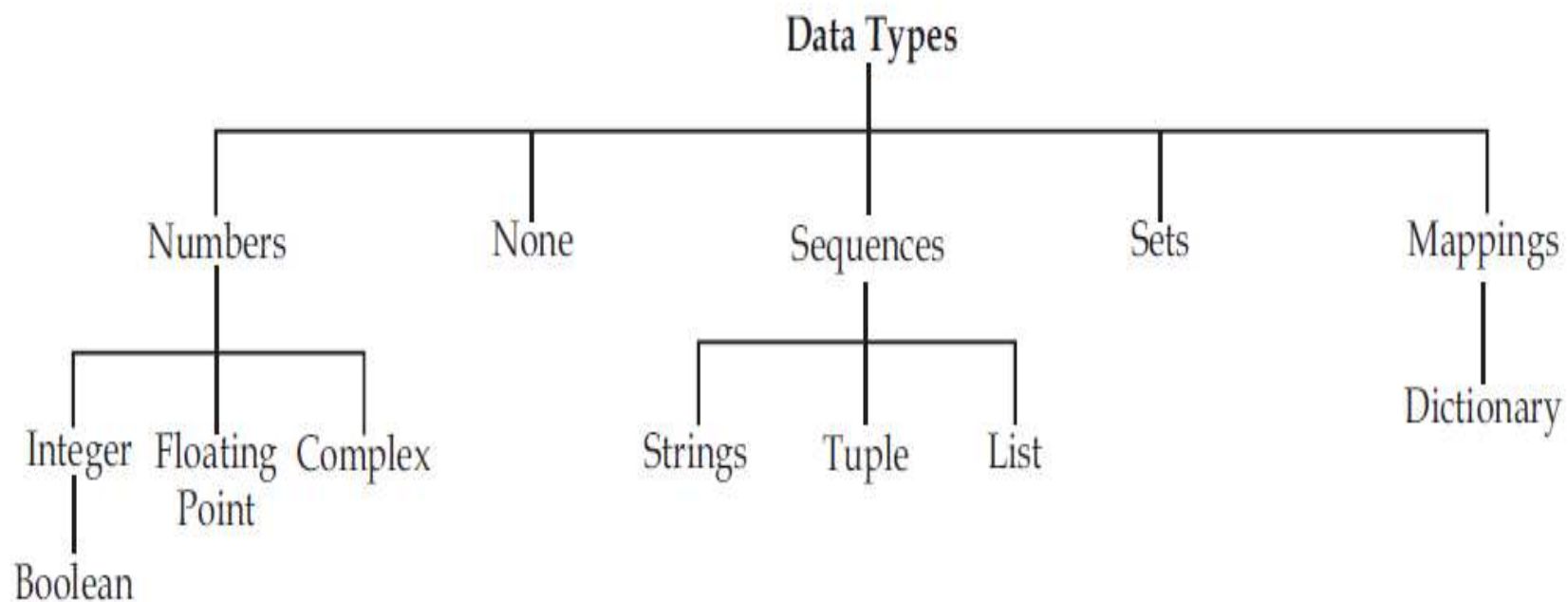
35

Python Programming

Basic Data Types

- The data stored in memory can be of many types. For example, a person's age is stored as a numeric value and his or her address is stored as alphanumeric characters.
- Python has five standard data types –
 - Numbers
 - String
 - List
 - Tuple
 - Dictionary

Data types: Hierarchical View



Python Programming

Python Identifiers

- It is a name used to identify a variable, function, class, module or other objects in Python program.
- An identifier starts with a letter (A to Z) or (a to z) or an underscore (_) followed by zero or more letters, underscores and digits (0 to 9).
- Python does not allow the characters - @, \$, and % .
- Valid Identifier- myvar, my_var , _my_var, myVar, MYVAR, myvar2

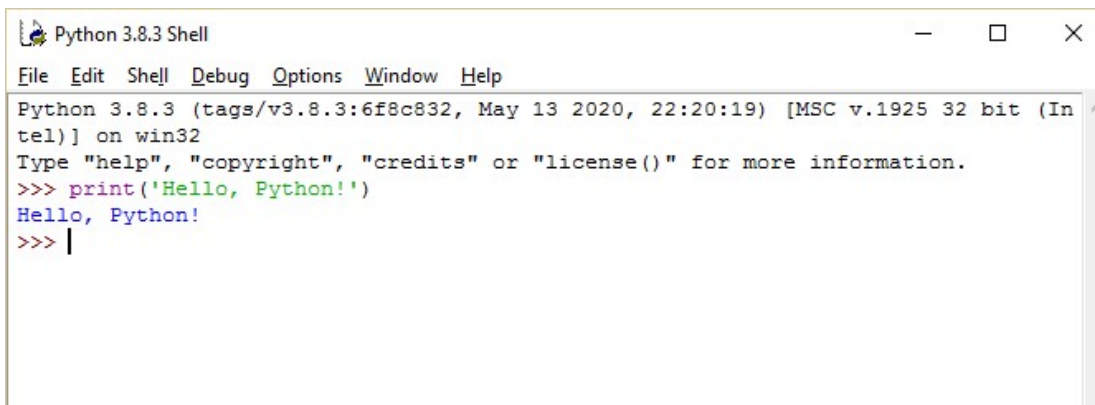
Python Programming

First Python Code

- Open the IDLE Python .

- Type
print('Hello, Python!')

- Press the Enter Key.



Python Programming

First Python Program

- Open the IDLE Python .
- Open the File-> New File

#program to print the sum of 2 number

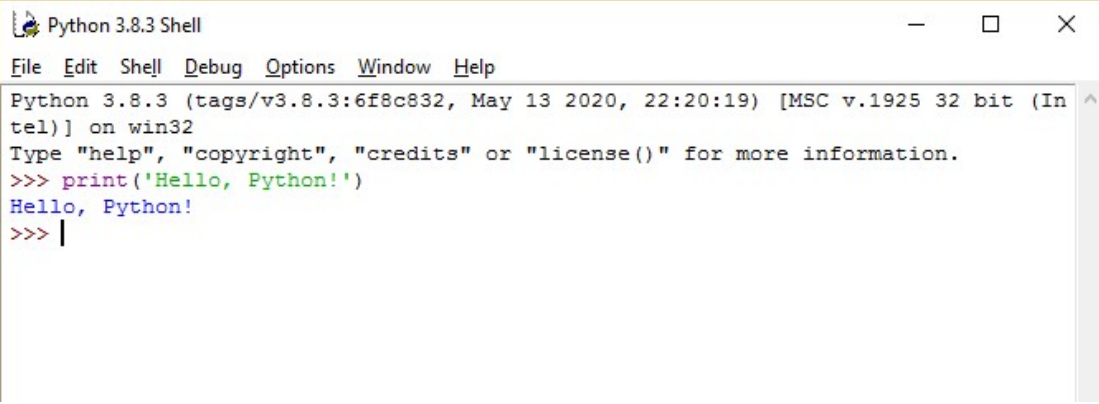
n1=10

n2=20

n3=n1+n2

print('Sum= ', n3)

- Save the file with “.py” extension.
- Click the Run-> Module.



References

- www.w3schools.com
- www.tutorialpoint.com
- Python Programming – Pearson – Taneja Kumar

Thank You
Any Query

Data Science using Python Programming

Unit-02: < Literals, Constants and Data Type>

Contents to be covered

1. Literals
2. Constants
3. Types of Literals
4. Python Program to demonstrate the Literals
5. Python Data Types
6. Python Program to demonstrate the Literals

Literals

- Literal is a raw data assigned to a variable or constant.
- Literals are immutable.
- Types of Literal
 - Text : str
 - Numeric : int, float, complex
 - Boolean Type : bool
 - Special : None
- Numeric Literal – 10 , 10.76 , 10+12j
- Text Literal – 'Mango' , "Mango" , ""Mango""
- Boolean Literal - True , False

Constants

- A constant is a type of variable whose value cannot be changed, during the execution of program.
- **In python, it does not prevent reassignment.**
- Constants are usually declared and assigned in a module.
- Module is a file containing variables, functions, etc which is imported to the main file.
- Usually, constants are written in all capital letters and underscores separating the words.

Types of Literals

- Numeric Literal
 - Numeric : int, float, complex
- Integer Literal
 - No fractional part allowed.
- Types of Integer Literal
 - Decimal Literal (base 10) - 10
 - Binary Literal (base 2) – **0b**1010
 - Octal Literal (base 8) – **0o**10
 - Hexadecimal Literal (base 16) - **0x**12c

```
a=10
b=0b10
c=0o10
d=0x10
print('Decimal ', a)
print('Binary ',b)
print('Octal ',c)
print('Hexadecimal ',d)
```

Output

```
Decimal 10
Binary 2
Octal 8
Hexadecimal 16
```

What is the output, if a=0x1C ?

Types of Literals

- Float Literal
 - Fractional part allowed.
- Types of Float Literal
 - Fixed Literal - 10.45
 - Scientific Literal – 10e2 , 10E2
 - 10e2 is equivalent to 10×10^2
 - 10e-2 is equivalent to 10×10^{-2}
- Complex Literal
 - It has real and imaginary part
 - 10+12j

Float Literal

10	E	2
Mantissa		Exponent
10	X	$10^2 = 1000$

Complex Literal

10	+	12j
Real		Imaginary

Types of Literals

Integer Literals

```
a=0b10  
b=0x10  
c=0o10
```

```
d=a+b+c  
print('Sum ', d)
```

Output

Sum 26

**What is the output,
if a=0x1c+0b1c ?**

Float Literals

```
a=10.45  
b=10E2  
c=10E-2
```

```
print('Fixed ', a)  
print('Scientific ',b)  
print('Scientific ',c)
```

Output

Fixed 10.45
Scientific 1000.0
Scientific 0.1

What is the output, if a=0.1E3 ?

Complex Literal

```
d=10 +12j  
print('Complex ', d)  
print('Real Part ', d.real)  
print('Imaginary Part ',d.imag)
```

Output

Complex (10+12j)
Real Part 10.0
Imaginary Part 12.0

Types of Literals

String Literal

- It is a sequence of characters surrounded by quotes.
- Quotes can be single, double, or triple quotes for a string.
- A string that is prefixed with an **r** or **R** before the opening quotes is a “raw” string .
- Backslash (\) allow the line to be continued as single line.
- Example

- `s1="This is Python"`
- `s2= "P"`
- `s3 = """This is a multiline string
with more than one line code."""`
- `s4=r"First \n Second"`
- `s5 ="This is \n Python"`

Character pairs start with backslash(\)

`\n` = Newline

`\r` = Carriage Return

`\t` = Tab

`\\` = \

`\u` =Marks the start of a Unicode character code

`\"` or `\'` = A double or single quote character **without** ending the current string.

Types of Literals

```
s1="This is Python"
s2= "P"
s3 = """This is a multiline string
with more than one line code."""
s4="First \n Second"
s5=r"First \n Second"
s6="This is \
    Python"
print("s1 ",s1)
print("s2 ",s2)
print("s3 ",s3)
print("s4 ",s4)
print("s5 ",s5)
print("s6 ",s6)
s1 = r'lang\tver\nPython\t3'
s2 = 'lang\\tver\\nPython\\t3'
print(s1)
print(s2)
```

Output

```
s1  This is Python
s2  P
s3  This is a multiline string
    with more than one line
    code.
s4  First
    Second
s5  First \n Second
s6  This is Python

lang\tver\nPython\t3
lang\tver\nPython\t3
```

Types of Literals

- **Boolean literals**

- It can have two values: True or False.
- True has numerical value 1.
- False has numerical value 0.
- Example
 - `x1 = (1 == True)`
 - `x2 = (1 == False)`
 - `x3 = True + 4`
 - `x4 = False + 10`

```
x1 = (1 == True)
x2 = (1 == False)
x3 = True + 4
x4 = False + 10
```

```
print("x1 is", x1)
print("x2 is", x2)
print("x3:", x3)
print("x4:", x4)
```

Output

```
x1 is True
x2 is False
x3: 5
x4: 10
```

Types of Literals

Special literals

- Python has one special literal i.e. **None**.
- It to specify that the variable has not been created.
- Example
 - `x = 10`
 - `y = None`
- Memory allocated to variable when the literal assigned to the variable for the first time.

```
x = 10  
y = None
```

```
print("x is", x)  
print("y is", y)
```

Output

```
x is 10  
y is None
```


Python Data Types

- Data type allows the type of data stored in memory.
- For example,
 - A person's age is stored as a numeric value and
 - Address is stored as alphanumeric characters.
- Data Types defines the operations possible on them and the storage method for each of them.
- Variables can store data of different types, and different types can do different things.
- Python has five standard data types –
 - Numbers
 - String
 - List
 - Tuple
 - Dictionary

Python Data Types

- Python has the following data types built-in under the categories:
 - Text Type : str
 - Numeric Types : int, float, complex
 - Sequence Types : list, tuple, range
 - Mapping Type : dict
 - Set Types : set
 - Boolean Type : bool

Example	Data Type
x = "Hello World"	Str
x = 20	Int
x = 20.5	Float
x = 1j	Complex
x = ["apple", "banana", "cherry"]	list
x = ("apple", "banana", "cherry")	Tuple
x = range(6)	Range
x = {"name" : "Ajay", "age" : 36}	Dict
x = {"apple", "banana", "cherry"}	Set
x = True	bool

Python Data Types

```
x1 = "Hello World"  
x2 = 20  
x3 = 20.5  
x4 = 1j  
x5 = True
```

```
print("String : ", x1)  
print("Integer : ", x2)  
print("Float : ", x3)  
print("Complex : ", x4)  
print("Bool : ", x5)
```

Output

```
String : Hello World  
Integer : 20  
Float : 20.5  
Complex : 1j
```

```
x6 = ["apple", "banana", "cherry"]  
x7 = ("apple", "banana", "cherry")  
x8 = range(6)  
x9 = {"name": "Ajay", "age": 36}  
x10 = {"apple", "banana", "cherry"}
```

```
print("List : ", x6)  
print("Tuple : ", x7)  
print("Range : ", x8)  
print("Dictionary : ", x9)  
print("Set : ", x10)
```

Output

```
List : ['apple', 'banana', 'cherry']  
Tuple : ('apple', 'banana', 'cherry')  
Range : range(0, 6)  
Dictionary : {'name': 'Ajay', 'age': 36}  
Set : {'banana', 'apple', 'cherry'}
```

Type Conversion

- Casting allows to specify a type on to a variable.

SNo.	Function	Description
1	int(x)	Converts x to an integer. base specifies the base if x is a string.
2	long(x)	Converts x to a long integer. base specifies the base if x is a string.
3	float(x)	Converts x to a floating-point number
4	complex(real [,imag])	Creates a complex number.
5	str(x)	Converts object x to a string representation.
6	tuple(s).	Converts s to a tuple
7	list(s)	Converts s to a list.
8	set(s)	Converts s to a set.
9	dict(d)	Creates a dictionary. d must be a sequence of (key,value) tuples.
10	chr(x)	Converts an integer to a character.

Type Conversion

Integers:

<code>x = int(1)</code>	<code># x will be 1</code>
<code>y = int(2.8)</code>	<code># y will be 2</code>
<code>z = int("3")</code>	<code># z will be 3</code>

Floats:

<code>x = float(1)</code>	<code># x will be 1.0</code>
<code>y = float(2.8)</code>	<code># y will be 2.8</code>
<code>z = float("3")</code>	<code># z will be 3.0</code>
<code>w = float("4.2")</code>	<code># w will be 4.2</code>

Strings:

<code>x = str("s1")</code>	<code># x will be 's1'</code>
<code>y = str(2)</code>	<code># y will be '2'</code>
<code>z = str(3.0)</code>	<code># z will be '3.0'</code>

References

- www.w3schools.com
- www.tutorialpoint.com
- Python Programming – Pearson – Taneja Kumar

Thank You
Any Query

Data Science using Python Programming

Unit-03: < Python Data Type and Operators>

Contents to be covered

1. Python Data Type- String , List , Tuple, Dictionary
2. Python Operators
3. Arithmetic operators
4. Assignment operators
5. Comparison operators
6. Logical operators

Python Data Type - String

- Strings in Python are identified as a contiguous set of characters represented in the quotation marks.
- Subsets of strings can be taken using the slice operator ([] and [:]) with indexes starting at 0 in the beginning of the string and negative index -1 at the end.
- The plus (+) sign is the string concatenation operator.
- The asterisk (*) is the repetition operator.

Python Data Type - String

H e l l o N I E L I T !
0 1 2 3 4 5 6 7 8 9 10 11 12

```
str = 'Hello NIELIT!'
print (str )           # Prints complete string
print (str[ 0])        # Prints first character of the string
print (str[ 2:5])       # Prints characters starting from 3rd to 5th
print (str[ 2:5:2])     # Prints characters starting from 3rd to 5th
print (str[ 2:] )       # Prints string starting from 3rd character
print ( str * 2 )       # Prints string two times
print (str + "GKP")    # Prints concatenated string
```

Output

```
Hello NIELIT!
H
llo
lo
llo NIELIT!
Hello NIELIT!Hello NIELIT!
Hello NIELIT!GKP
```

Python Data Type - List

- A list contains items separated by commas and enclosed within square brackets ([]).
- Difference between C array and List is that all the items belonging to a list can be of different data type.
- The values stored in a list can be accessed using the slice operator ([] and [:]) with indexes starting at 0 in the beginning of the list and working their way to end -1.
- The plus (+) sign is the list concatenation operator, and
- The asterisk (*) is the repetition operator.

Python Data Type - List

['abcd',	786 ,	2.23,	'john',	70.2]
0	1	2	3	4

```
x = [ 'abcd', 786 , 2.23, 'john', 70.2 ]
```

```
y = [123, 'john']
```

```
print(x)          # Prints complete list
```

```
print (x[0])      # Prints first element of the list
```

```
print (x[1:4])    # Prints elements starting from 2nd till 4th
```

```
print (x[1:4:2])  # Prints elements starting from 2nd till 4th
```

```
print (x[2:])     # Prints elements starting from 3rd element
```

```
print (y * 2)     # Prints list two times
```

```
print (x + y)     # Prints concatenated lists
```

Output

```
['abcd', 786, 2.23, 'john', 70.2]
```

```
abcd
```

```
[786, 2.23, 'john']
```

```
[786, 'john']
```

```
[2.23, 'john', 70.2]
```

```
[123, 'john', 123, 'john']
```

```
['abcd', 786, 2.23, 'john', 70.2, 123, 'john']
```

['abcd',	786 ,	2.23,	'john',	70.2]
0	1	2	3	4

Python Data Type - Tuple

- A tuple is similar to the list.
- A tuple consists of a number of values separated by commas. Tuples are enclosed within parentheses.
- The plus (+) sign is the list concatenation operator, and
- The asterisk (*) is the repetition operator.
- The main differences between lists and tuples are:
 - Lists are enclosed in brackets ([]) and their elements and size can be changed.
 - While tuples are enclosed in parentheses (()) and cannot be updated.
 - Tuples is an read-only lists.

Python Data Type - Tuple

```
( 'abcd', 786, 2.23, 'john', 70.2 )  
      0      1      2      3      4
```

```
x = ( 'abcd', 786 , 2.23, 'john', 70.2 )
```

```
y = (123, 'john')
```

```
print(x)      # Prints complete tuple
```

```
print (x[0])   # Prints first element of the tuple
```

```
print (x[1:3]) # Prints elements starting from 2nd till 3rd
```

```
print (x[2:])  # Prints elements starting from 3rd element
```

```
print (y * 2)  # Prints list two times
```

```
print (x + y)  # Prints concatenated tuple
```

Output

```
('abcd', 786, 2.23, 'john', 70.2)
```

```
abcd
```

```
(786, 2.23)
```

```
(2.23, 'john', 70.2)
```

```
(123, 'john', 123, 'john')
```

```
('abcd', 786, 2.23, 'john', 70.2, 123, 'john')
```

Python Data Type - Dictionary

- Python's dictionary is a **key : value** pair.
- A **dictionary key** are numbers or strings.
- **Values** can be any arbitrary Python object.
- Dictionaries are enclosed by curly braces { } and
- Values can be assigned and accessed using square braces [].

Python Data Type - Dictionary

```
d = {'name': 'john', 'code':6734, 'dept': 'sales', 23:45}
```

```
print ( d )           # Prints complete dictionary
```

```
print ( d.keys() )    # Prints all the keys
```

```
print ( d.values() )  # Prints all the values
```

```
print(d["name"])
```

```
print(d[23])
```

Output

```
{'name': 'john', 'code': 6734, 'dept': 'sales', 23: 45}
```

```
dict_keys( ['name', 'code', 'dept', 23] )
```

```
dict_values( ['john', 6734, 'sales', 45] )
```

```
john
```

```
45
```

Python Operators

- Operators are used to perform operations on variables.
- Python divides the operators in the following groups:
 - 1. Arithmetic operators
 - 2. Assignment operators
 - 3. Comparison operators
 - 4. Logical operators
 - 5. Identity operators
 - 6. Membership operators
 - 7. Bitwise operators

Arithmetic Operator

- Arithmetic operators are used with numeric values to perform common mathematical operations.

Operator	Name	Example
+	Addition	$x + y$
-	Subtraction	$x - y$
*	Multiplication	$x * y$
/	Division	x / y
%	Modulus	$x \% y$
**	Exponentiation	$x ** y$
//	Floor division	$x // y$

Arithmetic Operator

```
a = 21
b = 10
c = a + b
print("Line 1 - Value of c is ", c)
c = a - b
print("Line 2 - Value of c is ", c)
c = a * b
print("Line 3 - Value of c is ", c)
c = a / b
print ("Line 4 - Value of c is ", c)
```

Output

```
Line 1 - Value of c is 31
Line 2 - Value of c is 11
Line 3 - Value of c is 210
Line 4 - Value of c is 2.1
```

```
a = 21
b = 10
c = a % b
print("Line 5 - Value of c is ", c)
a = 2
b = 3
c = a**b
print("Line 6 - Value of c is ", c)
a = 10
b = 3
c = a//b
print ("Line 7 - Value of c is ", c)
```

Output

```
Line 5 - Value of c is 1
Line 6 - Value of c is 8
Line 7 - Value of c is 3
```

Assignment Operator

- Assignment operators are used to assign values to variables.

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
//=	x //= 3	x = x // 3
**=	x **= 3	x = x ** 3
&=	x &= 3	x = x & 3
=	x = 3	x = x 3
^=	x ^= 3	x = x ^ 3
>>=	x >>= 3	x = x >> 3
<<=	x <<= 3	x = x << 3

Assignment Operator

```
a = 21
b = 10
c = 0
c = a + b
print ("Line 1 - Value of c is ", c)
c += a
print ("Line 2 - Value of c is ", c)
```

```
c=10
c *= a
print ("Line 3 - Value of c is ", c)
c /= a
print ("Line 4 - Value of c is ", c)
c = 2
c %= a
print ("Line 5 - Value of c is ", c)
```

Output

```
Line 1 - Value of c is 31
Line 2 - Value of c is 52
Line 3 - Value of c is 210
Line 4 - Value of c is 10.0
Line 5 - Value of c is 2
```

```
c=10
a=2
c **= a
print ("Line 6 - Value of c is ", c)
c /= a
print ("Line 7 - Value of c is ", c)
```

```
c=20
c >>= 2
print ("Line 8 - Value of c is ", c)
```

```
c=20
c <<= 1
print ("Line 9 - Value of c is ", c)
```

Output

```
Line 6 - Value of c is 100
Line 7 - Value of c is 50
Line 8 - Value of c is 5
Line 9 - Value of c is 40
```

Comparison Operators

Comparison operators are used to compare two values.

Operator	Name	Example
==	Equal	$x == y$
!=	Not equal	$x != y$
>	Greater than	$x > y$
<	Less than	$x < y$
>=	Greater than or equal to	$x >= y$
<=	Less than or equal to	$x <= y$

Comparison Operators

```
a = 21  
b = 10
```

```
if ( a == b ):  
    print ("Line 1 - a is equal to b")  
else:  
    print ("Line 1 - a is not equal to b")
```

```
if ( a != b ):  
    print ("Line 2 - a is not equal to b")  
else:  
    print ("Line 2 - a is equal to b")
```

```
if ( a < b ):  
    print ("Line 3 - a is less than b")  
else:  
    print ("Line 3 - a is not less than b")
```

Output

Line 1 - a is not equal to b
Line 2 - a is not equal to b
Line 3 - a is not less than b

```
a = 5;  
b = 20;
```

```
if ( a <= b ):  
    print ("Line 4- a is either less than or equal to b")  
else:  
    print ("Line 4 - a is neither less than nor equal to b")
```

```
if ( b >= a ):  
    print ("Line 5 - b is either greater than or equal to b")  
else:  
    print ("Line 5 - b is neither greater than nor equal to b")
```

Output

Line 4 - a is either less than or equal to b
Line 5 - b is either greater than or equal to b

Logical Operators

Logical operators are used to combine conditional statements.

Operator	Description	Example
and	Returns True if both statements are true	$x < 5$ and $x < 10$
or	Returns True if one of the statements is true	$x < 5$ or $x < 4$
not	Reverse the result, returns False if the result is true	not($x < 5$ and $x < 10$)

E1	E2	and	or
False	False	False	False
False	True	False	True
True	False	False	True
True	True	True	True

Logical Operators

```
#Python program to demonstrate
```

```
# logical and operator
```

```
a = 10
```

```
b = 10
```

```
c = -10
```

```
if a > 0 and b > 0 :
```

```
    print("The numbers are greater than 0")
```

```
if a > 0 and b > 0 and c > 0 :
```

```
    print("The numbers are greater than 0")
```

```
else:
```

```
    print("Atleast one number is not greater than 0")
```

Output

The numbers are greater than 0

Atleast one number is not greater than 0

```
a = 10
```

```
b = 12
```

```
c = 0
```

```
if a and b and c :
```

```
    print("All the numbers have boolean value as True")
```

```
else:
```

```
    print("Atleast one number has boolean value as False")
```

Output

Atleast one number has boolean value as False

Logical Operators

```
a = 10
```

```
if not a:
```

```
    print("Boolean value of a is True")
```

```
if not (a%3 == 0 or a%5 == 0):
```

```
    print("10 is not divisible by either 3 or 5")
```

```
else:
```

```
    print("10 is divisible by either 3 or 5")
```

Output

```
10 is divisible by either 3 or 5
```

```
# First If Statement
```

```
not a    = not True    = False
```

```
# Second If Statement
```

```
a % 3 = 10 % 3 = 1    ( 1==0 ) False
```

```
a % 5 = 10 % 5 = 0    ( 0==0 ) True
```

```
not (False or True ) = not ( True ) = False
```

References

- www.w3schools.com
- www.tutorialpoint.com
- Python Programming – Pearson – Taneja Kumar

Thank You
Any Query

Data Science using Python Programming

Unit-04: < Python Operators and Control Structure>

<Shivlok Singh>

<S.T.O.>

Contents to be covered

1. Identity Operators
2. Membership Operators
3. Bitwise Operator
4. Introduction to Control Structure
5. IF Statement

Membership Operators

- Membership operators are used to test if a sequence is presented in an object.

Operator	Description	Example
in	Returns True if a sequence with the specified value is present in the object	x in y
not in	Returns True if a sequence with the specified value is not present in the object	x not in y

Membership Operators

Example-1

```
a = 10  
b = 20  
list = [1, 2, 3, 4, 5 ]
```

```
if ( a in list ):  
    print ("Line 1 - a is available in the given list")  
else:  
    print ("Line 1 - a is not available in the given list")
```

Output

Line 1 - a is not available in the given list

Example-2

```
a = 10  
b = 20  
list = [1, 2, 3, 4, 5 ]
```

```
if ( b not in list ):  
    print ("Line 2 - b is not available in the given list")  
else:  
    print ("Line 2 - b is available in the given list")
```

Output

Line 2 - b is not available in the given list

Membership Operators

Example-3

```
a = 2
list = [1, 2, 3, 4, 5 ]

if ( a in list ):
    print ("Line 3 - a is available in the given list")
else :
    print ("Line 3 - a is not available in the given list")
```

Output

Line 3 - a is available in the given list

Example-4

```
a = 2
dict = {1, 2, 3, 4, 5 }
tup= ( 2,4,6,8,10)

if ( a in dict and a in tup ):
    print ("a is available in the given dict and tuple")
else :
    print ("a is not available in the given dict and tuple")
```

Output

a is available in the given dict and tuple

Identity Operators

- Identity operators are used to compare the objects, for
 - if they are equal or not.
 - If they are equal, means both are same object, with the same memory location.

Operator	Description	Example
is	Returns True if both variables are the same object	x is y
is not	Returns True if both variables are not the same object	x is not y

Identity Operators

#In Python, if 2 variable has the same value, then they
#refer the same memory space.

```
a = 20
```

```
b = 20
```

```
if ( a is b ):
```

```
    print ("Line 1 - a and b have same identity")
```

```
else:
```

```
    print ("Line 1 - a and b do not have same identity")
```

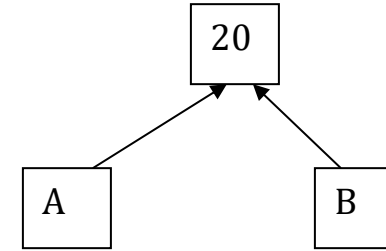
```
print("Id of a= ", id(a)," b= " ,id(b))
```

```
if ( id(a) == id(b) ):
```

```
    print ("Line 2 - a and b have same identity.")
```

```
else:
```

```
    print ("Line 2 - a and b do not have same identity")
```



Output

Line 1 - a and b have same identity

Id of a= 1529084128 b= 1529084128

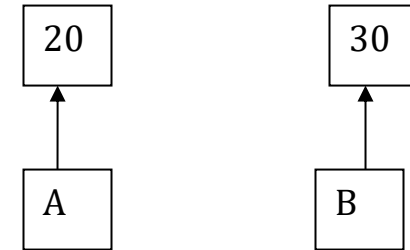
Line 2 - a and b have same identity.

Identity Operators

#In Python, if 2 variable has the same value, then they
#refer the same memory space, otherwise different

```
a = 20  
b = 30
```

```
if ( a is b ):  
    print ("Line 3 - a and b have same identity")  
else:  
    print ("Line 3 - a and b do not have same identity")  
  
print("Id of a= ", id(a)," b= " ,id(b))  
  
if ( a is not b ):  
    print ("Line 4 - a and b do not have same identity")  
else:  
    print ("Line 4 - a and b have same identity")
```



Output

Line 3 - a and b do not have same identity
Id of a= 1529084128 b= 1529084288
Line 4 - a and b do not have same identity

Bitwise Operators

- Bitwise operators are used to compare (binary) numbers.

Operator	Name	Description
&	AND	Sets each bit to 1 if both bits are 1
	OR	Sets each bit to 1 if one of two bits is 1
^	XOR	Sets each bit to 1 if only one of two bits is 1
~	NOT	Inverts all the bits
<<	Zero fill left shift	Shift left by pushing zeros in from the right and let the leftmost bits fall off
>>	Signed right shift	Shift right by pushing copies of the leftmost bit in from the left, and let the rightmost bits fall off

Bitwise Operator Chart

A	B	&		^
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

Bitwise Operators

```
a = 60          # 60 = 0011 1100
b = 13          # 13 = 0000 1101
c = 0
```

```
c = a & b        # 12 = 0000 1100
print ("Line 1 - Value of c is ", c)
```

```
c = a | b        # 61 = 0011 1101
print ("Line 2 - Value of c is ", c)
```

```
c = a ^ b        # 49 = 0011 0001
print ("Line 3 - Value of c is ", c)
```

```
c = ~a           # -61 = 1100 0011
print ("Line 4 - Value of c is ", c)
```

Bitwise &

```
A  0011 1100
B  0000 1101
-----
```

```
C  0000 1100
-----
```

Bitwise |

```
0011 1100
0000 1101
-----
```

```
0011 1101
-----
```

Bitwise ^

```
A  0011 1100
B  0000 1101
-----
```

```
C  0011 0001
-----
```

Bitwise ~

```
0011 1100
-----
```

```
1100 0011
-----
```

Output

```
Line 1 - Value of c is 12
Line 2 - Value of c is 61
Line 3 - Value of c is 49
Line 4 - Value of c is -61
```


1100 0011 is the binary representation of a negative integer, on 8 bits.

- The first bit (the leftmost) indicates the sign,
- 1 = negative, 0 = positive.
- 1100 0011 is the binary representation of a negative integer, on 8 bits.
- **2. Get the binary representation in one's complement**
- Run this step only if the number is negative *
- Note: $11 - 1 = 10$; $10 - 1 = 1$; $1 - 0 = 1$; $1 - 1 = 0$.
- Subtract 1 from the initial binary number.
- $1100\ 0011 - 1 = 1100\ 0010$
- $1100\ 0010 = 1 * 2^7 + 1 * 2^6 + 0 * 2^5 + 0 * 2^4 + 0 * 2^3 + 0 * 2^2 + 1 * 2^1 + 0 * 2^0$
 - $0 + 0 + 32 + 16 + 8 + 4 + 0 + 1 = -61$

Bitwise Operators

```
a = 10          # 10 = 0000 1010
```

```
c = a << 2      # 40 = 0010 1000  
print ("Line 5 - Value of c is ", c)
```

```
c = a >> 2      # 2 = 0000 0010  
print ("Line 6 - Value of c is ", c)
```

Output

```
Line 5 - Value of c is 40  
Line 6 - Value of c is 2
```

Python Control Statement

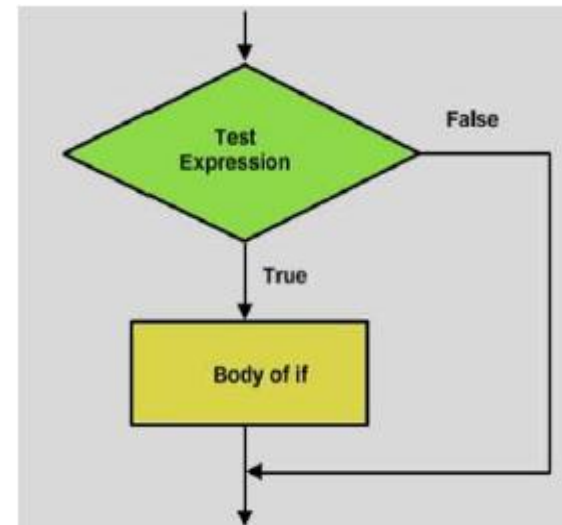
- A control statement modifies the order of statement execution.
- A control statement can cause other program statements
 - to execute multiple times
 - not to execute at all
 - depending on the circumstances.
- Types of Control Statement
 - Branching Statement :- used to select one of the alternative statement.
 - if Statement
 - If....else Statement.
 - If....elif...else Statement
 - Looping or Iterative Statement :- used to repeat the statement till the condition is true.
 - for loop
 - while loop

IF Statement

- This statement evaluates an expression and directs program execution depending on the result of evaluation.
- It controls the multiple statements through the use of a compound statement or block.
- Block is a group of two or more statements indented at the same level.

IF Syntax

**if expression :
statements**



IF Statement

```
var1 = 100
if var1:
    print ("1- Got a true expression value")
    print (var1)
```

```
var2 = 0
if var2:
    print ("2- Got a true expression value")
    print (var2)
```

```
print ("Good bye!")
```

Output

```
1- Got a true expression value
100
Good bye!
```

IF ... Else Statement

- The if..else statement evaluates test expression and will execute body of if only when test condition is **True**.
- If the condition is **False**, body of else is executed. Indentation is used to separate the blocks.
- It controls the multiple statements through the use of a compound statement or block.
- Block is a group of two or more statements indented at the same level.

IF ... Else Statement

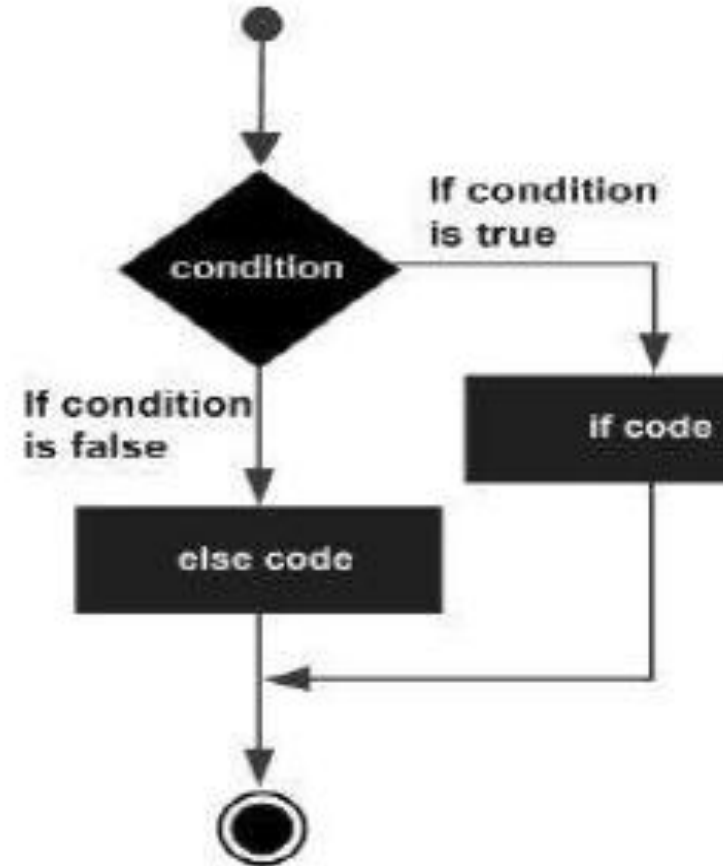
- **Syntax:-**

If condition:

Statement1

else :

Statement2



IF...Else Statement

#Example -1

```
var1 = 100
```

```
if var1:
```

```
    print ("1- Got a true expression value")
```

```
    print (var1)
```

```
else:
```

```
    print ("1- Got a false expression value")
```

```
    print (var1)
```

```
print ("Good bye!")
```

Output

```
1- Got a true expression value
```

```
100
```

```
Good bye!
```

#Example -2

```
var2 = 0
```

```
if var2:
```

```
    print ("2- Got a true expression value")
```

```
    print (var2)
```

```
else:
```

```
    print ("2 Got a false expression value")
```

```
    print (var2)
```

```
print ("Good bye!")
```

Output

```
2- Got a false expression value
```

```
0
```

```
Good bye!
```


If...elif...else Statement

- The elif statement allows you to check multiple expressions for TRUE and execute a block of code as soon as one of the conditions evaluates to TRUE.

- **Syntax**

```
if expression1 :  
    statement(s)
```

```
elif expression2 :  
    statement(s)
```

```
elif expression3 :  
    statement(s)
```

```
else :  
    statement(s)
```

IF...Else Statement

```
var = 100
if var == 200:
    print ("1 Got a true expression value")
    print (var)
elif var == 150:
    print ("2 Got a true expression value")
    print (var)
elif var == 100:
    print ("3 Got a true expression value")
    print (var)
else:
    print ("4 Got a false expression value")
    print (var)
print "Good bye!"
```

Output

```
3 Got a true expression value
100
Good bye!
```

References

- www.w3schools.com
- www.tutorialpoint.com
- Python Programming – Pearson – Taneja Kumar

Thank You
Any Query

Data Science using Python Programming

Unit-05: < Python Control Structure – While Loop>

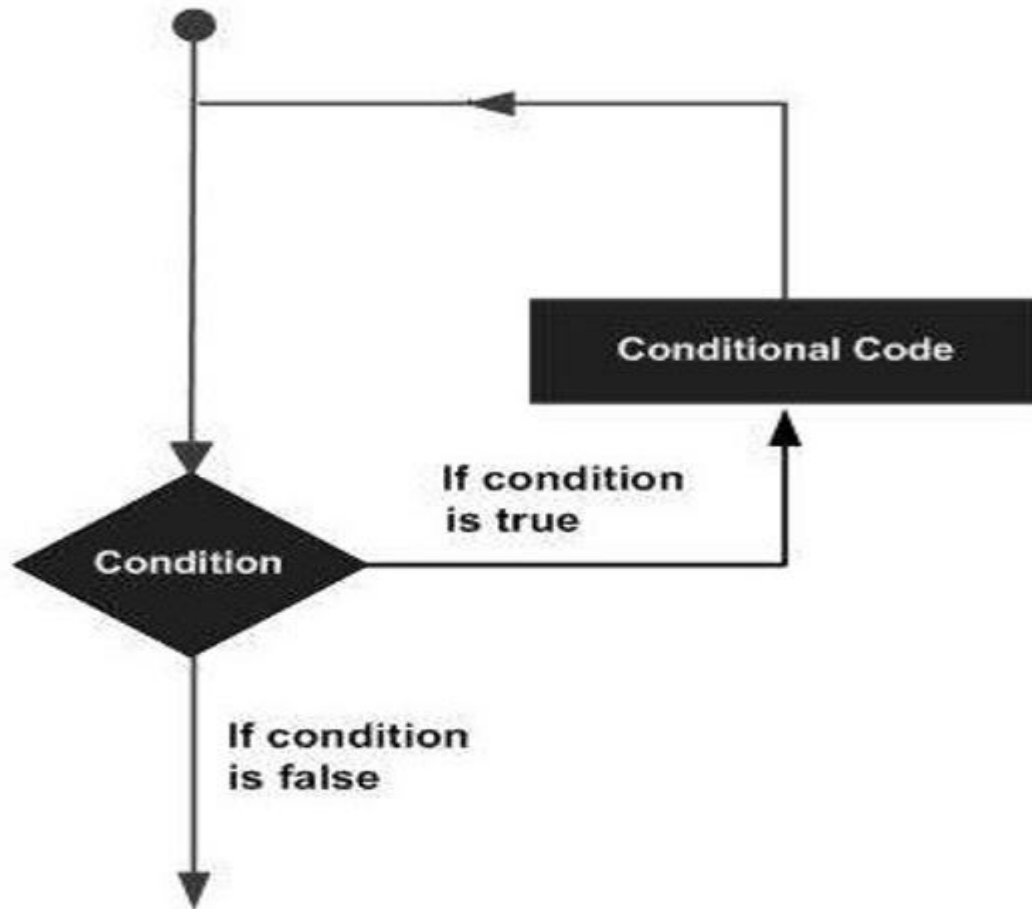
Contents to be covered

1. Loop Control Structure
2. While Loop
3. Break & Continue statement
4. Else and Pass statement
5. Input and Output Statement

Loop Control Structure

- A loop statement allows us to execute a statement or group of statements multiple times.
- Loop Types
 1. **while loop** : Repeats a statement or group of statements while a given condition is TRUE. It tests the condition before executing the loop body.
 2. **for loop** : Executes a sequence of statements multiple times and abbreviates the code that manages the loop variable. It repeats the loop till the items not exhausted in the sequence or range.
 3. **nested loops** : You can use one or more loop inside any another while, for or do..while loop.

Loop Control Structure



**Repeats the statements
till the condition is TRUE.**

**Statement is the
indented code.**

While Structure

- A while loop statement repeatedly executes a statement as long as a given condition is true.
- Syntax

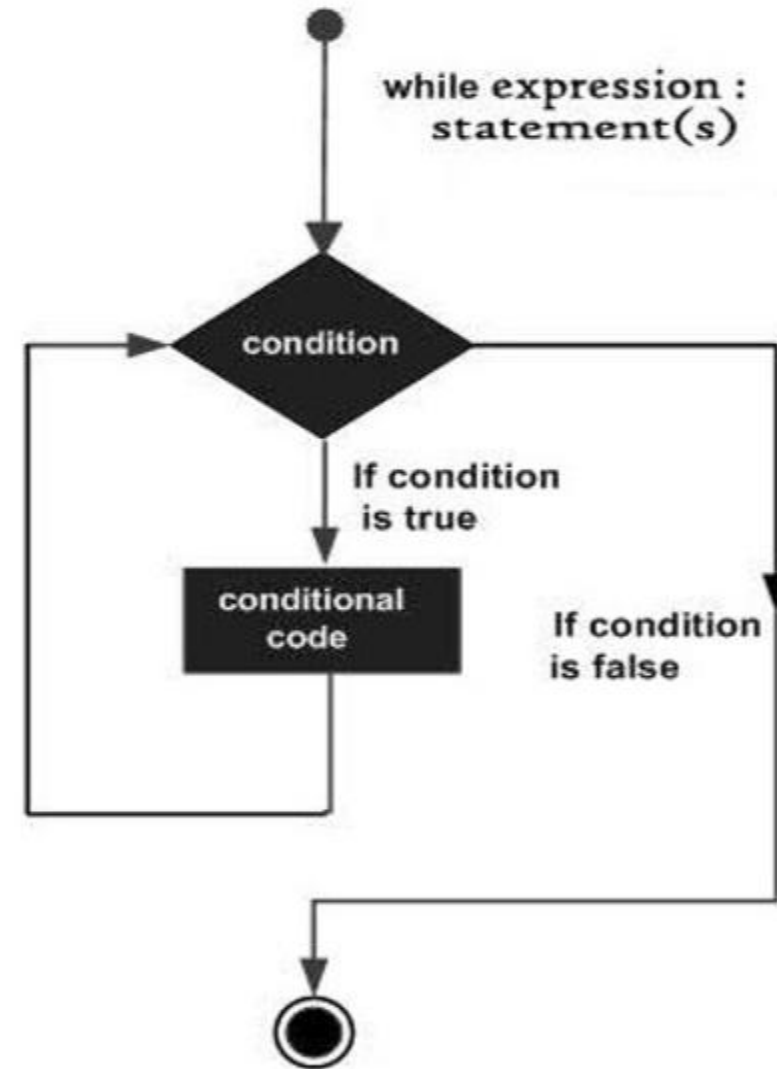
while expression :
statement(s)

- Statement(s) may be a single statement or a block of statements.
- The condition may be any expression, and **True** is any non-zero value.
- The loop iterates while the condition is **True**.
- Python uses indentation as its method of grouping statements.

While Structure

**Repeats the statements
till the condition is TRUE.**

**Statement is the
indented code.**



While Structure

Program to print 1 to 10

```
count = 1
```

```
while (count <= 10) :
```

```
    print ('The count is:', count)
```

```
    count = count + 1
```

Output

The count is: 0

The count is: 1

The count is: 2

The count is: 3

The count is: 4

The count is: 5

The count is: 6

The count is: 7

The count is: 8

The count is: 9

The count is: 10

While Structure

Program to print 1,3,5,7,9

```
count = 1
while (count < 10):
    print ( 'The count is:', count)
    count = count + 2
```

Output

The count is: 1
The count is: 3
The count is: 5
The count is: 9

Program to print sum of 1 to 10

```
count = 1
sum = 0
while (count <= 10) :
    sum=sum + count =1+2+3
    count = count + 1
print ( 'The sum is:', sum)
```

Output

The sum is: 55

While Structure

Program to factorial of number

```
count = 1
fact = 1
while (count <= 5) :
    fact = fact * count
    count = count + 1
print ('The fact is:', fact)
```

Output

The fact is: 120

Think for

1. Program to print 10 to 1.
eg 10 9 8 7 62 1
2. Program to print reverse of 125.

Using else Statement with While Loops

- If the else statement is used with a while loop, the else statement is executed when the condition becomes false.
- Syntax

while expression :

statement(s)

else :

statement(s)

While Structure - Using else Statement

Program to print sum of 1 to 10

```
count = 1
sum = 0
while (count <= 10) :
    sum=sum + count
    count = count + 1
else :
    print ( 'The sum is:', sum)
```

Output

The sum is: 55

Program to print Message

```
count = 0
while count < 5:
    print (count, " - Inside Loop")
    count = count + 1
else:
    print (count, " - Out of Loop")
```

Output

0 - Inside Loop
1 - Inside Loop
2 - Inside Loop
3 - Inside Loop
4 - Inside Loop
5 - Out of Loop

Pass Statement

- The **pass statement** is used as a placeholder for future code.
- **Pass statement** is an Empty Statement.
- It do nothing.
- Syntax `n=1`

while expression :

Pass

else :

statement(s)

Using pass Statement

Program to print message if
number is less the 5.

n=4

if n<5:

 pass

else:

 print("Greater than 5")

We use the pass statement,

- 1) when the logic of processing is currently unknown.
- 2) When nothing to be done.

While Loop and Break Statement

- With the **break** statement, allows to stop the loop even if the while condition is true.
- Control of the program flows to the statement immediately after the body of the loop.
- If break statement is inside a nested loop (loop inside another loop), break will terminate the innermost loop.
- Syntax

while expression :

statement(s)

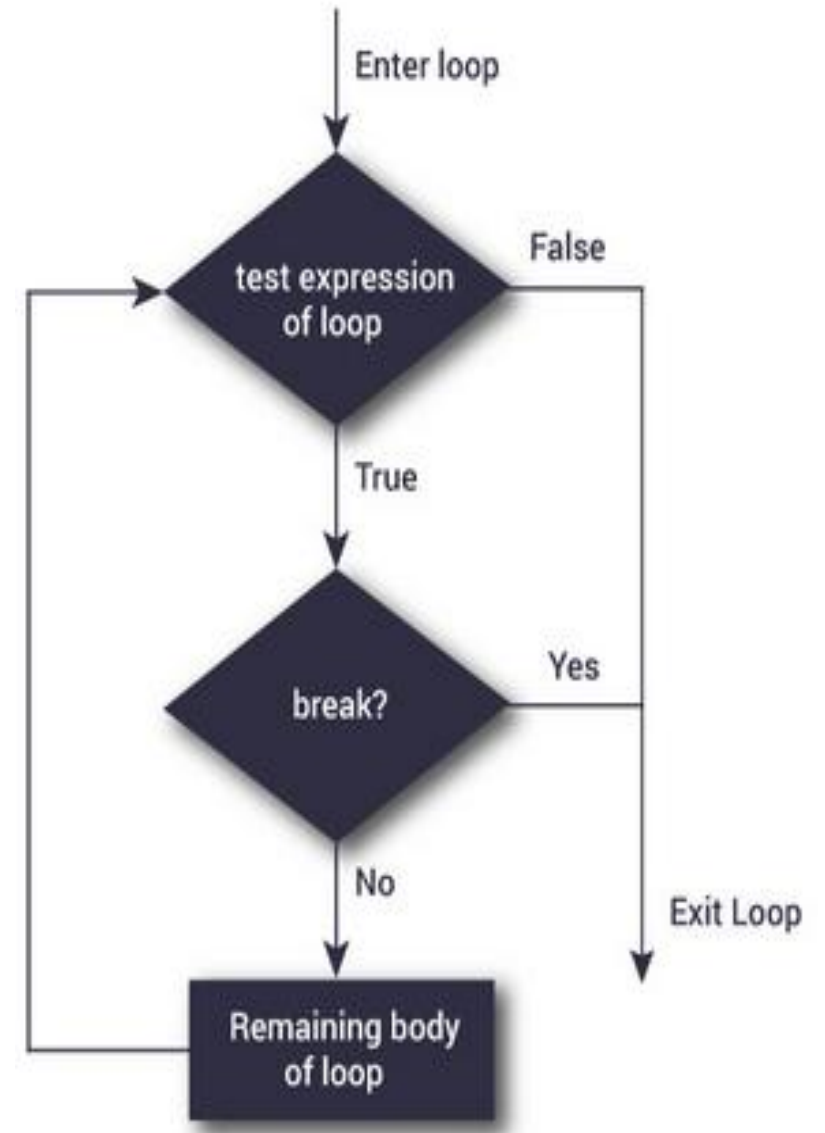
break

Break Statement

```
count=1  
while (count <= 10) :  
    count = count + 1  
    if count == 4 :  
        break  
    print ( count )
```

This loop will repeats 10 times in normal condition.

Due to **Break statement**, control jumps out of the loop, when count equal to 4.



Break Statement

Program to print 1 to 5

```
i = 0
while i <=5:
    i = i + 1
    if i == 3:
        break
    print( i )
```

Output

1
2

While Loop and Continue Statement

- With the **continue** statement, we can skip the rest of the code inside a loop for the current iteration only.
- Loop does not terminate but continues on with the next iteration.
- Control of the program remains inside the body of the loop.
- Syntax

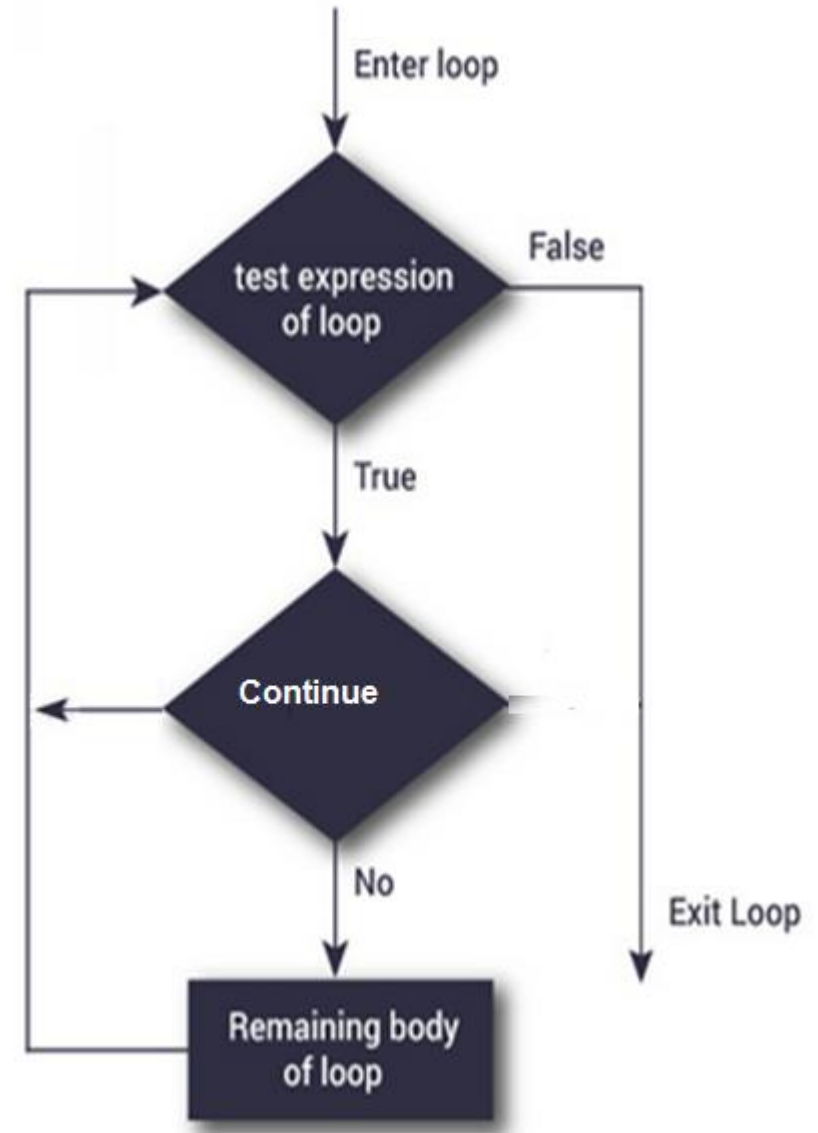
while expression :
statement(s)
continue

Continue Statement

```
count=1
while (count <= 10) :
    count = count + 1
    if count == 4 :
        continue
    print ( count )
```

This loop will repeats 10 times in normal condition and print 1 to 10.

Due to **Continue statement**, control skips loop body , when count equal to 4.



Continue Statement

Program to print 1 to 5

```
i = 0
while i < 6:
    i = i + 1
    if i == 3:
        continue
    print( i )
```

Output

1
2
4
5
6

References

- www.w3schools.com
- www.tutorialpoint.com
- Python Programming – Pearson – Taneja Kumar

Thank You
Any Query

Data Science using Python Programming

Unit-06: < Python Control Structure – For Loop>

<Shivlok Singh>

<S.T.O.>

Contents to be covered

1. For Loop
2. Else Statement
3. Break & Continue Statement
4. range() function
5. Enumerate() function

For Loop Structure

- A for loop is used for iterating the item from the sequence (list, tuple, dictionary, set, string) or range.
- Syntax

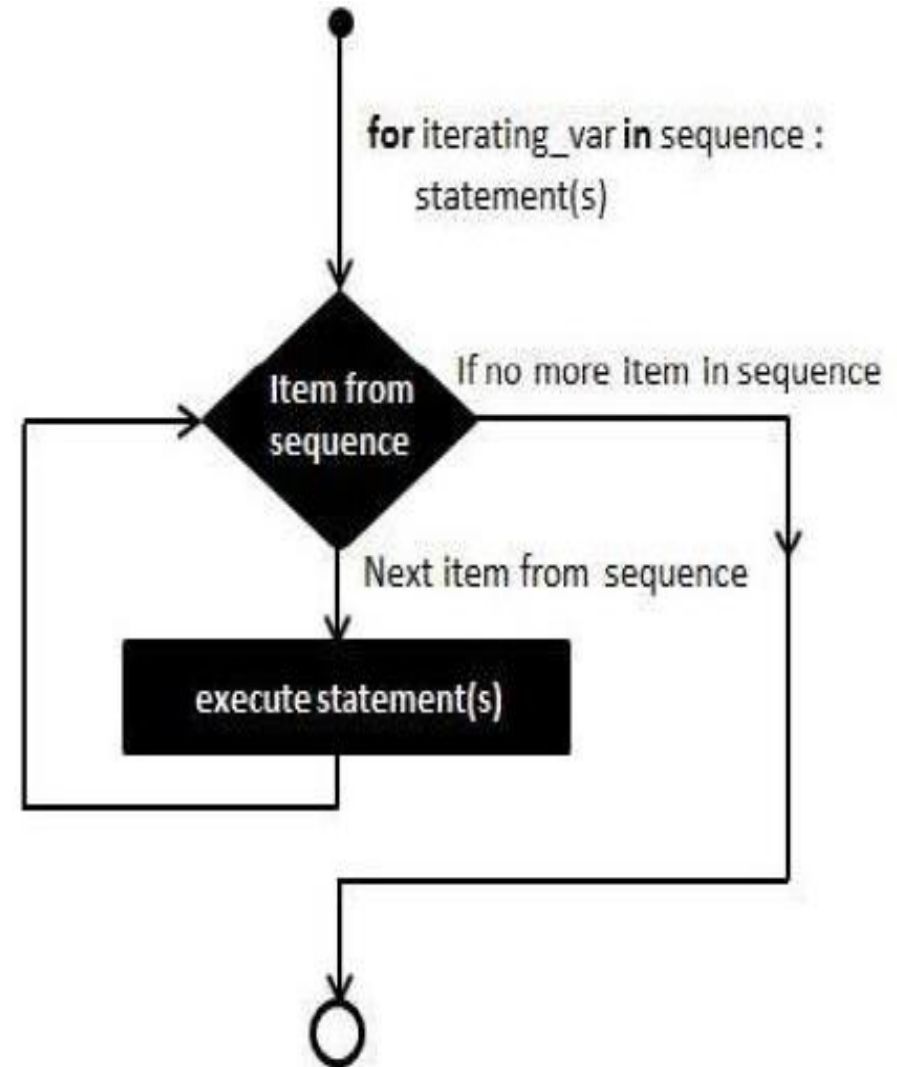
**for var in sequence or range :
statement(s)**

- For loop repeats till the item in the sequence not exhausted.
- Statement(s) may be a single statement or a block of statements.
- Python uses indentation as its method of grouping statements.

For Loop Structure

Repeats the statements till the items from the sequence not exhausted.

Statements is the indented code.



For Loop Structure

Program to print 1 to 5

```
for i in (1,2,3,4,5) :  
    print ( 'The count is:', i )
```

Output

The count is: 1
The count is: 2
The count is: 3
The count is: 4
The count is: 5

**# Program to print the character
#from the string**

```
for i in 'PYTHON' :  
    print ( i )
```

Output

P
Y
T
H
O
N

For Loop Structure

Program to count the vowels

```
count=0
```

```
for i in "Python Programming":
```

```
    if i in ('a', 'e', 'i', 'o', 'u') :
```

```
        count=count+1
```

```
print ('The count is:', count)
```

Output

The count is: 4

Program to count the even no

```
count=0
```

```
for i in (1 ,2 ,3, 4, 5,6):
```

```
    if i%2==0:
```

```
        count=count+1
```

```
print ('The count is:', count)
```

Output

The count is: 3

For Loop Structure

Think for

- 1. Program to count the characters other than Vowels.**
- 2. Program to count the numbers divisible by 3.**
- 3. Program to count the numbers divisible by 3 and 5.**

Using else Statement with For Loops

- If the else statement is used with a for loop, the else statement is executed when the condition becomes false.
- Syntax

```
for var in Sequence :  
    statement(s)  
  
else :  
    statement(s)
```

Using else Statement

Program to count the even.

```
count=0
for i in (1,2,3,4,5):
    if i % 2==0 :
        count=count+1
else :
    print ( 'The count is:', count)
```

Output

The count is: 2

Program to count the vowels

```
count=0
for i in "Python Programming":
    if i in ('a','i','e','o','u') :
        count=count+1
else :
    print ('The count is:', count)
```

Output

The count is: 4

For Loop and Break Statement

- With the **break** statement, allows to stop the loop even if the sequence in for loop not exhausted.
- Control of the program flows to the statement immediately after the body of the loop.
- If break statement is inside a nested loop (loop inside another loop), break will terminate the innermost loop.
- Syntax

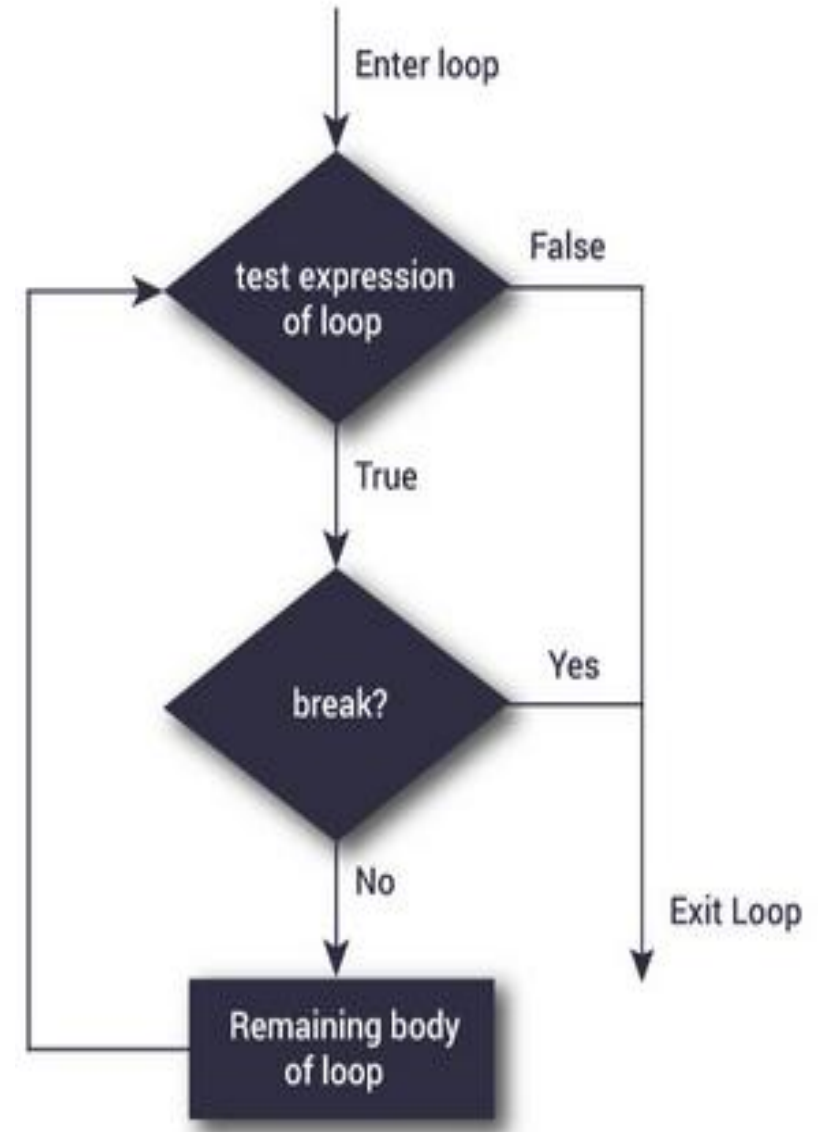
for var in sequence :
statement(s)
break

Break Statement

```
count=0
for i in [1,2,3,4,5,6]:
    count=count+1
    if count==3:
        break
print("Count : ", count)
```

This loop will repeat 6 times in normal condition.

Due to **Break statement**, control jumps out of the loop, when count equal to 3.



Break Statement

Program to use Break

```
count=0
for c in "python":
    if c in ( 'a' , 'e', 'i', 'o', 'u') :
        break
    print(c)
    count=count+1

print("Count ", count)
```

Output

p
y
t
h

Count 4

For Loop and Continue Statement

- With the **continue** statement, we can skip the rest of the code inside a loop for the current iteration only.
- Loop does not terminate but continues on with the next iteration.
- Control of the program remains inside the body of the loop.
- Syntax

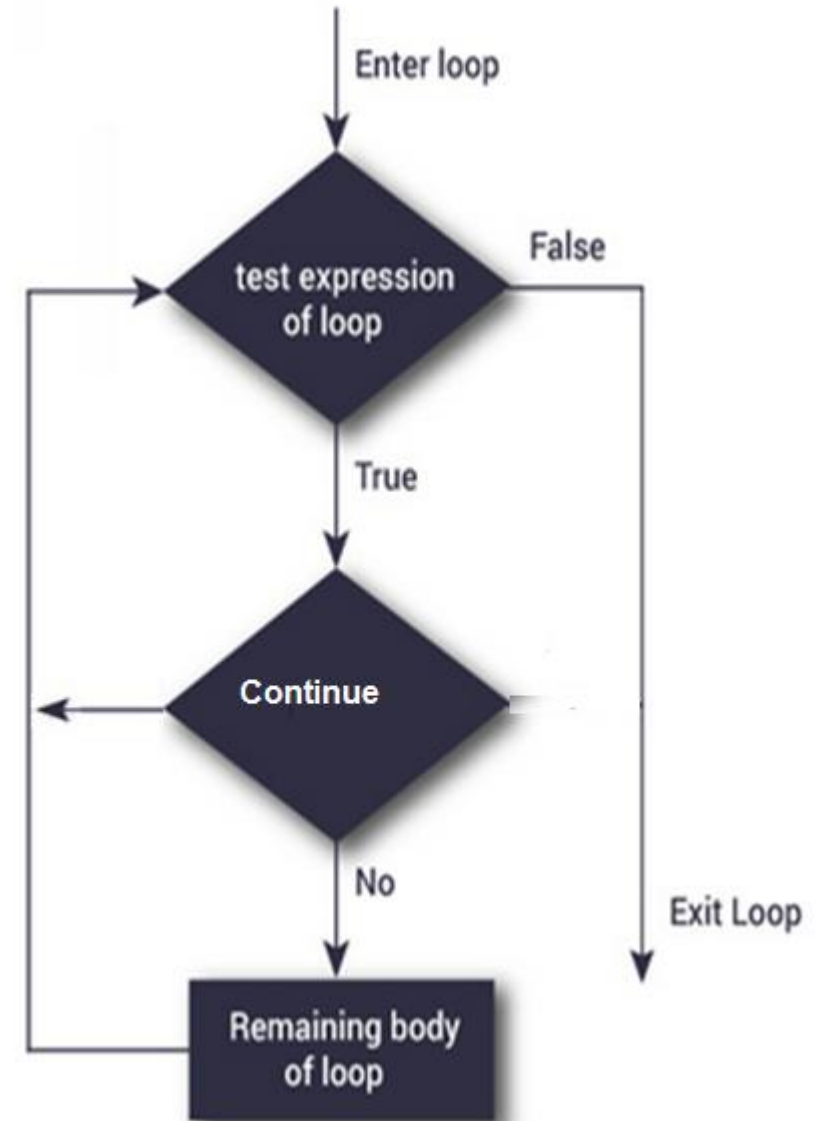
```
for var in sequence :  
    statement(s)  
    continue
```

Continue Statement

```
count=0
for i in [1,2,3,4,5]:
    count=count+1
    if count==3:
        continue
    print("Count:",count)
```

This loop will repeat 5 times in normal condition and print 1 to 5.

Due to **Continue statement**, control skips loop body, when count equal to 3.



Continue Statement

Program to use Continue

```
count=0
for c in "python":
    if c in ( 'a' , 'e', 'i', 'o', 'u') :
        continue
    print(c)
    count=count+1

print("Count ", count)
```

Output

p
y
t
h
n

Count 5

Range Function

- It is a in-built function.
- The range() function generates the sequence of numbers.
- The sequence starts from 0 by default, and increments by 1 (by default), and ends at a 1 before specified number.
- Syntax :

range (start, stop, step)

- start [Optional] - position to start. Default is 0
- stop [Required] - position to end. (**stop – 1**)
- step [Optional] - specifying the increment. Default is 1

Range Function

Example

- | | |
|---------------------------------------|-------------------------------------|
| 1. <code>x = range(3, 6)</code> | 3, 4, 5 |
| 2. <code>x = range(3, 20, 2)</code> | 3, 5, 7, 9, 11, 13, 15, 17, 19 |
| 3. <code>x = range(10)</code> | 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 |
| 4. <code>x = range(10, 1, -1)</code> | 10, 9, 8, 7, 6, 5, 4, 3, 2 |
| 5. <code>x = range(-10, -1, 1)</code> | -10, -9, -8, -7, -6, -5, -4, -3, -2 |
| 6. <code>x = range(10, 1, -2)</code> | 10, 8, 6, 4, 2 |

Range Function

#Program to print 1 to 5

```
r=range( 1, 5)
```

```
for x in r :  
    print( x)
```

Output

1
2
3
4

#Program to print 1 to 10

```
r=range(1, 11 , 1)
```

```
sum=0
```

```
for x in r :  
    sum=sum + x
```

```
print("sum ", sum)
```

Output

Sum = 55

Range Function

#Program to print odd number 1-10

```
r=range( 1, 10, 2)
```

```
for x in r :  
    print( x)
```

Output

1
3
5
7
9

Program for factorial of number

```
count = 1
```

```
fact = 1
```

```
for x in range(1, 6, 1) :
```

```
    fact = fact * x
```

```
print ('The fact is:', fact)
```

Output

The fact is: = 120

Enumerate with for loop

- The `enumerate( )` is an in-built function in Python that allows the user to convert the sequences(list, tuple, dictionary) in enumeration list.

- Example

```
Months = ["Jan", "Feb" , "Mar", "April", "May", "June"]  
for i, m in enumerate (Months):  
    print (i , m)
```

Output : 0 Jan , 1 Feb, 2 Mar , 3 April, 4 May, 5 June

Enumerate() Function

#Program to print odd numbers

```
r=range(1,10,2)
for i , m in enumerate(r):
    print("Odd Number at ", i, " is " , m)
```

Output

```
Odd Number at 0 is 1
Odd Number at 1 is 3
Odd Number at 2 is 5
Odd Number at 3 is 7
Odd Number at 4 is 9
```

#Program to print odd numbers

```
r=range(1,10,2)
for i , m in enumerate(r):
    print("Odd Number at ", i+1 , " is " , m)
```

Output

```
Odd Number at 1 is 1
Odd Number at 2 is 3
Odd Number at 3 is 5
Odd Number at 4 is 7
Odd Number at 5 is 9
```

References

- www.w3schools.com
- www.tutorialpoint.com
- Python Programming – Pearson – Taneja Kumar

Thank You
Any Query

Data Science using Python Programming

Unit-07: < Nested Loops and String Handling>

Contents to be covered

1. Nested While Loop
2. Nested For Loop
3. Strings and its manipulation
4. String built-in Function
5. String Formatting Operator

Nested While Loop

- Python allows using one loop inside another while or for loop.
- Syntax for nested While Loop

while expression:

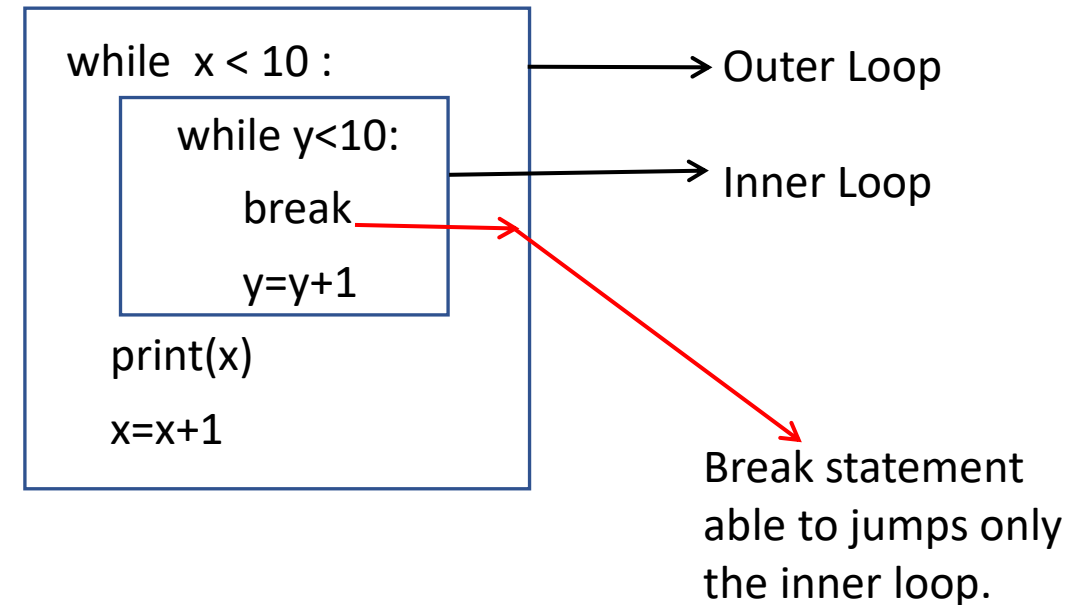
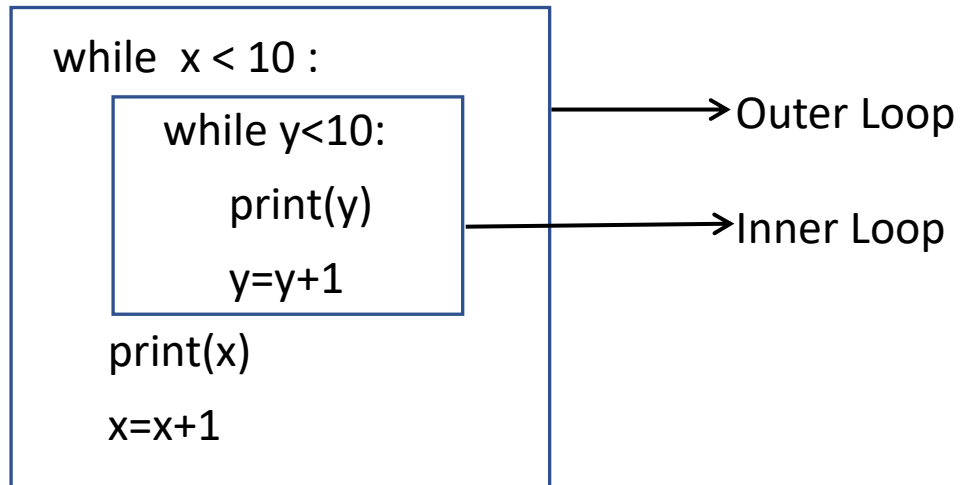
while expression:

statement(s)

statement(s)

- If break statement is used, it will jump out of the internal loop only.

Nested While Loop



Nested While Loop

#Program uses a nested for loop to find #the prime numbers from 2 to 100

#To display prime number from 2 to 10

```
i=2
f=0
while(i<=10):
    j=2
    f=0
    while(j<i):
        if i%j==0:
            print("not prime",i)
            f=1
            break

    j=j+1
    i=i+1
    if(f==1):
        print("number is prime",i)
```

Program uses a nested-for loop to
display multiplication tables from 1-10.

```
for i in range(1,11):
    for j in range(1,11):
        k = i *j
        print (k, end=' ')
    print( )
```

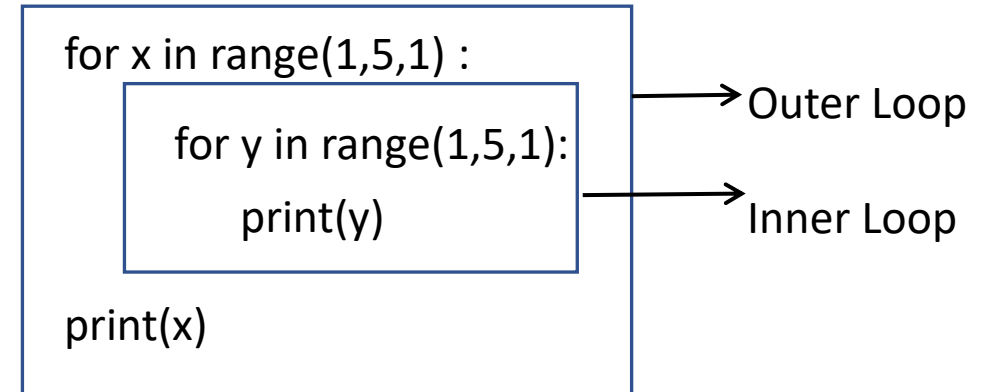
Output

It prints the table of 1 to 10 upto 10 multiples.

Nested For Loop Loop

- Python allows using one loop inside another while or for loop.
- Syntax for nested for Loop

**for var in sequence :
 for var in sequence :
 statements(s)
statements(s)**



- If break statement is used, it will jump out of the internal loop only.

Nested for Loop

#Program to print the pattern

```
for i in range(1,6):  
    for j in range(i) :  
        print("*", end=' ')  
    print()
```

Output

```
*  
**  
***  
****  
*****
```

Explanation

range(1,6) - generate the number 1,2,3,4,5

i=1 range(i) - generate 0

i=2 range(i) - generate 0,1

i=3 range(i) - generate 0,1,2

i=4 range(i) - generate 0,1,2,3

i=5 range(i) - generate 0,1,2,3,4

Nested for Loop

```
p=5
for i in range(1,6):
    print(" ",end="")
    for j in range(i) :
        print("*", end=' ')
    print()
    p=p-1
p=1
for i in range(5,0,-1):
    print(" "*p,end="")
    for j in range(i) :
        print("*", end=' ')
    print()
    p=p+1
```

```
      *
     * *
    * * *
   * * * *
  * * * * *
 * * * * *
* * * * *
 * * * *
  * * *
   * *
    *
```

Nested for Loop

#Program to print the length of words

```
for w in ("Python", "Programming") :  
    len=0  
    for c in w :  
        len=len+1  
    print('Length of Word : ', len)
```

Output

Length of Word : 6
Length of Word : 11

Explanation

("Python", "Programming") is a sequence.

Iteration-1

w - "Python"

c - represent each character in word

Iteration-2

w - "Programming"

c - represent each character in word

Strings

- Strings are arrays of bytes representing unicode characters.
- A single character is a string with a length of 1.
- Strings are enclosed characters in single quotes or double quotes.
- Strings are immutable so the characters of the string cannot be changed after it is created.
- Creating String
 - `var1 = 'Hello World!'`
 - `var2 = "Python Programming"`
 - `var3 = "P"`

Strings

- Access Items – To access substrings, we use the square brackets for slicing along with the index or indices.

- Example

```
var1 = 'Hello World!'
```

```
var2 = "Python Programming"
```

```
print ("var1[0]: ", var1[0])
```

```
---- var1[0]: H
```

```
print ("var2[1:5]: ", var2[1:5])
```

```
---- var2[1:5]: ytho
```

- Negative Index- It means from the end, -1 refers to the last item, -2 refers to second last item.

```
var2 = "Python Programming"
```

```
print ("last Character: ", var2[-1])
```

```
---- last Character: g
```

String A	S	A	V	E		E	A	R	T	H
Positive Index	0	1	2	3	4	5	6	7	8	9
Negative Index	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

Strings Manipulation

1. Range of Indexes (Slicing)

```
var2 = "Python Programming"  
print ("var2[2:5]: ", var2[2:6])  
print ("var2[2:5]: ", var2[2:6:2])
```

Output : thon
Output : to

2. Negative Indexes

```
var2 = "Python Programming"  
print ("var: ", var2[-4:-1])  
print ("var: ", var2[-4:-1:2])
```

Output : min
Output : mn

3. Updating Strings

```
var2 = "Python Programming"  
var2="Hello "+ var2[:6]  
print(var2)
```

Output : Hello Python

4. Loop Through a String

```
var2 = "Python Programming"  
for x in list:  
    print(x)
```

5. Check if Item String

```
txt = "The rain in Spain stays mainly in the plain"  
x = "ain" in txt  
print(x) Output True  
  
x = "ain" not in txt  
print(x) Output False
```

6. Length of String

```
var="Python Programming"  
print(len(var))
```

Strings - Functions

1. String Concatenation

```
a = "Hello"  
b = "World"  
c = a + b  
print(c)                # Hello World
```

2 Repeat the string with *

```
a = "Hello "  
print(c * 3)             # Hello Hello Hello
```

3. **strip()** - removes any whitespace from the start or end.

```
a = " Hello, World! "  
print(a.strip())         # "Hello, World!"
```

4. **lower()** - convert the string in lower case.

```
a = "Hello, World! "  
print(a.lower())         # "hello, world!"
```

5. **upper()** - convert the string in lower case.

```
a = "Hello, World! "  
print(a.upper())         # "HELLO, WORLD!"
```

6. **replace()** - replaces a string with another string.

```
a = "Hello, World!"  
print(a.replace("H", "J"))    # "Jello, World!"
```

7. **split()** - splits the string into substrings at the separator.

```
a = "Hello, World!"  
print(a.split(","))          # [ 'Hello', ' World!' ]
```

8. **format()** - format value and place at the placeholders {} in string.

```
quantity = 3  
itemno = 567  
price = 49.95  
myorder = "I want to pay {2} dollars for {0} pieces of item {1}. "  
print(myorder.format(quantity, itemno, price))
```

Output

I want to pay 49.95 dollars for 3 pieces of item 567.

Strings – Formatting Operator

String Formatting Operator

```
print ("My name is %s and weight is %d kg!" % ('Ajay', 21))
```

Output : My name is Ajay and weight is 21 kg!

Sr.No.	Format Symbol & Conversion
1	%c character
2	%s string conversion via <u>str()</u> prior to formatting
3	%i signed decimal integer
4	%d signed decimal integer
5	%u unsigned decimal integer
6	%o octal integer
7	%x hexadecimal integer (lowercase letters)
8	%X hexadecimal integer (<u>UPPERcase</u> letters)
9	%e exponential notation (with lowercase 'e')
10	%E exponential notation (with <u>UPPERcase</u> 'E')
11	%f floating point real number
12	%g the shorter of %f and %e
13	%G the shorter of %f and %E

Triple Quotes

Python's triple quotes allows strings to span multiple lines, including NEWLINES, TABs, and any other special characters.

The syntax for triple quotes consists of three consecutive **single or double** quotes.

```
a = " " "Hello,
      Python,
      Programming" " "
print(a)
```

```
a = ' ' 'Hello,
      Python,
      Programming' ' '
print(a)
```

Strings

#Program to check whether the string is a palindrome or not.

```
str=input("Enter the String :")
l=len(str)
p=l-1
index=0
while (index < p) :
    if( str[index]==str[p] ) :
        index=index+1
        p=p -1
    else :
        print ("String is not a palindrome" )
        break
else :
    print ("String is a Palindrome" )
```

Output

```
Enter the String : ABCBA
String is a Palindrome
```

#Program to count no of 'p' in the string pineapple.

```
word = 'pineapple'
count = 0
for letter in word:
    if letter == 'p':
        count = count + 1
print(' Total : ' ,count)
```

Output

```
Total : 3
```


Strings

#Program to check the negative sentence.

```
str=input("Enter the Sentence:")

if ( 'not' in str):
    print('It is negative sentence')
else:
    print ('It is positive sentence' )
```

Output

Enter the Sentence : Ajay do not like the mobile
It is negative sentence

#Program to reverse the string pineapple.

```
word = 'pineapple'
rev=""

for letter in word:
    rev=letter+ rev

print(' Reverse : ' , rev)
```

Output

Reverse: elppaenip

```
print( word [ : : -1] )
```

References

- www.w3schools.com
- www.tutorialpoint.com
- Python Programming – Pearson – Taneja Kumar

Thank You
Any Query

Data Science using Python Programming

Unit-08: < Python Sequences- List, Tuple, Set>

Contents to be covered

1. Python Sequences
2. List and its manipulation
3. Tuple and its manipulation
4. Set and its manipulation

Python Sequence

Python Collections (Arrays)

There are four collection data types in the Python programming language:

1. **List** - A Collection which is ordered and changeable. Allows duplicate members.
2. **Tuple** - A collection which is ordered and unchangeable. Allows duplicate members.
3. **Set** - A collection which is unordered and unindexed. No duplicate members.
4. **Dictionary** - A collection which is unordered, changeable and indexed. No duplicate members.

List

- A list is a collection which is ordered and changeable.
- The list is a **datatype** available in Python which allows a list of comma-separated values (items) between square brackets.
- Items in a list need not be of the same type.
- Example
 - `list1 = ['physics', 'chemistry', 1997, 2000]`
 - `list2 = [1, 2, 3, 4, 5]`
 - `list3 = ["a", "b", "c", "d"]`
- `List( )` – convert the object to List Type.
 - `list = list("Python") ---- ['P', 'y', 't', 'h', 'o', 'n']`
 - `list2=list( (2,4,5) ) ---- [2, 4, 5]`

List

- **Access Items** – To access item, we use the square brackets for slicing along with the index or indices.

- **Example**

```
list1 = ['physics', 'chemistry', 1997, 2000]
```

```
list2 = [1, 2, 3, 4, 5, 6, 7]
```

```
print ("list1[0]: ", list1[0])
```

```
---- list1[0]: physics
```

```
print ("list2[1:5]: ", list2[1:5])
```

```
---- list2[1:5]: [2, 3, 4, 5]
```

- **Negative Index-** It means from the end, -1 refers to the last item, -2 refers to second last item.

```
list1 = ['physics', 'chemistry', 1997, 2000]
```

```
print ("list1[3]: ", list1[-1])
```

```
----- list1[3]: 2000
```

List Manipulation

1. Range of Indexes (Slicing)

```
list1 = ['physics', 'chemistry', 1997, 2000]
print ("list1[1:3]: ", list1[1:3])      ---- ['chemistry', 1997]
print ("list1[1:5:2]: ", list1[1:5:2])  ---- ['chemistry', 2000]
```

2. Negative Indexes

```
list1 = ['physics', 'chemistry', 1997, 2000]
print ("list1[-3:-1]: ", list1[-3:-1])  ---- ['chemistry', 1997]
print ("list1[-3:-1:2]: ", list1[-3:-1:2]) ---- ['chemistry']
```

3. Loop Through a List

```
list = ['physics', 'chemistry', 1997, 2000]
for x in list:
    print(x)
```

4. Updating List

```
list = ['physics', 'chemistry', 1997, 2000]
list[2] = 2001
print ("New value available at index 2 : ", list[2])
list.append('Maths')
print("New value available at index 2 : ", list)
list.insert(1,'Hindi')
print("New value available at index 2 : ", list)
```

Output

New value available at index 2 : 2001

New value available at index 2 :

['physics', 'chemistry', 2001, 2000, 'Maths']

New value available at index 2 :

['physics', 'Hindi', 'chemistry', 2001, 2000, 'Maths']

List - Operation

5. Check if Item in List

```
list = ["apple", "banana", "cherry"]  
if "apple" in list:  
    print("Yes, 'apple' is in the fruits list")
```

6. Length of List

```
list = ["apple", "banana", "cherry"]  
print(len(list))
```

7. List Concatenation

```
a = [1, 2, 3]  
b = [4, 5, 6]  
c = a + b  
print(c)                # [1, 2, 3, 4, 5, 6]
```

8. Repeat the List with *

```
a = [1, 2, 3]  
print(a * 3)            # [1, 2, 3, 1, 2, 3, 1, 2, 3]
```

9. remove() - remove the item.

```
a = [1, 2, 3]  
print(a.remove(2))      # [1, 3]
```

10. pop() - remove the item at index.

```
a = [1, 2, 3]  
print(a.pop(2))        # [1, 2]
```

11. clear() - removes all the items.

```
a = [1, 2, 3]  
print(a.clear())       # None
```

12. max() - maximum from the list.

```
a = [1, 2, 3]  
print(max(a))          # 3
```

13. del - removes the object.

```
a = [1, 2, 3]  
del a  
print(a)               # Error
```

Tuple

- A tuple is a collection which is ordered and unchangeable (immutable).
- The tuple is a **datatype** in Python which allows comma-separated values (items). Optionally, you can put these comma-separated values between parentheses also.
- Items in a tuple need not be of the same type.
- Example
 - `t1 = ( 'physics', 'chemistry', 1997, 2000)`
 - `t2 = (1, 2, 3, 4, 5 )`
 - `t3 = "a", "b", "c", "d"`
- `Tuple( )` – convert the object to TupleType.
 - `t1 = tuple("Python")` ----- `('P', 'y', 't', 'h', 'o', 'n')`
 - `t2=tuple( [ 2,4,5 ] )` ----- `(2, 4, 5)`

Tuple

- **Access Items** – To access item, we use the square brackets for slicing along with the index or indices.

- **Example**

```
tup1= ( 'physics', 'chemistry', 1997, 2000 )
```

```
tup2 = ( 1, 2, 3, 4, 5, 6, 7 )
```

```
print ("tuple1[0]: ", tup1[0])
```

```
---- tuple1[0]: physics
```

```
print ("tuple2[1:5]: ", tup2[1:5])
```

```
---- tuple2[1:5]: (2, 3, 4, 5)
```

- **Negative Index-** It means from the end, -1 refers to the last item, -2 refers to second last item.

```
tup1 = ( 'physics', 'chemistry', 1997, 2000 )
```

```
print ("tuple1[3]: ", tup1[-1])
```

```
----- tuple1[3]: 2000
```

Tuple

1. Range of Indexes (Slicing)

```
tup1= ('physics', 'chemistry', 1997, 2000)
print ("tup1[1:3]: ", tup1[1:3])      ---- ('chemistry', 1997 )
print ("tup1[1:5:2]: ", tup1[1:5:2])  ---- ('chemistry', 2000)
```

2. Negative Indexes

```
tup1 = ( 'physics', 'chemistry', 1997, 2000 )
print ("tuple1[-3:-1]: ", tup1[-3:-1])  ----- ('chemistry', 1997)
print ("tuple1[-3:-1:2]: ", tup1[-3:-1:2])  ----- ('chemistry')
```

3. Loop Through a Tuple

```
tup1 = ('physics', 'chemistry', 1997, 2000)
for x in tup1:
    print(x)
```

4. Updating Tuple

```
tup1 = (12, 34.56)
tup2 = ('abc', 'xyz')
```

Following action is not valid for tuples

```
# tup1[0] = 100
```

create a new tuple

```
tup3 = tup1 + tup2
print (tup3)
```

add new value in tuple

```
tup1 = tup1 + (60, )
print (tup1)
```

Output

```
(12, 34.56, 'abc', 'xyz')
(12, 34.56, 60)
```

Tuple

4. Method to update the values in Tuple

```
x = ("apple", "banana", "cherry" )  
y = list(x)  
y[1] = "kiwi"  
x = tuple(y)  
print(x)
```

Output

```
("apple", "kiwi", "cherry")
```

Tuple Concepts

```
#Empty Tuple  
t1= ( )  
print(type(t1))
```

```
# a tuple with one item  
t2 = ("apple" , )  
print(type(t2))
```

```
#NOT a tuple  
t3 = ("apple")  
print(type(t3))
```

Tuple

6. Check if Item in Tuple

```
tup= ( "apple", "banana", "cherry" )  
if "apple" in tup :  
    print("Yes, 'apple' is in the fruits list")
```

7. Length of Tuple

```
tup= ("apple", "banana", "cherry")  
print( len(tup) )
```

8. Tuple Concatenation

```
a = (1, 2, 3 )  
b = ( 4, 5, 6 )  
c = a + b  
print(c)                # (1, 2, 3, 4, 5, 6 )
```

9. Repeat the List with *

```
a = ( 1, 2, 3 )  
print(a * 3)                # (1, 2, 3, 1, 2, 3, 1, 2, 3 )
```

10. del - Tuples can be deleted only. Removing item not allowed.

```
a = (1, 2, 3 )  
del a  
print( a )                # Error
```

11. index(item) - Gives the index of item.

```
a = (1, 2, 3 )  
print(a.index(2) )        # 1
```

12. max() - maximum from the tuple.

```
a = ( 1, 2, 3 )  
print(max(a))            # 3
```


Tuple

**#Write a program to input 'n' numbers and
store it in tuple.**

```
t=tuple( )  
n=input("Enter any number : ")  
n=int(n)  
print (" enter all numbers one after other : " )  
for i in range(n):  
    a=input("enter number : ")  
    t=t+(a,)  
  
print ("output is" )  
print ( t )
```

Output

```
Enter any number : 5  
enter all numbers one after other  
enter number : 10  
enter number : 20  
enter number : 30  
enter number : 40  
enter number : 50
```

```
output is  
( '10', '20', '30', '40', '50' )
```

SET

- A Set is a collection which is unordered and unindexed.
- The Set is a **datatype** which allows a list of comma-separated values (items) between **Curly brackets**.
- Items in a list need not be of the same type.
- Example
 - `set1 = { 'physics', 'chemistry', 1997, 2000 }`
 - `set2 = { 1, 2, 3, 4, 5 }`
 - `set3 = { "a", "b", "c", "d" }`
- `set( )` – convert the object to Set Type.
 - `s1 = set("Python")` ---- `{'P', 'y', 't', 'h', 'o', 'n' }`
 - `s2 = set( (2,4,5) )` ---- `{ 2, 4, 5 }`
- The sets allows the mathematical operations like union, intersection, difference and complement etc

SET

- Access Items – Individual items not accessible in SET. Access all the items together.

- Example

```
set1 = { 'physics', 'chemistry', 1997, 2000 }
```

```
set2 = { 1, 2, 3, 4, 5, 6, 7 }
```

```
print ("set1: ", set1)
```

```
print ("set1[1]: ", set1[1] )      # Index not allowed in Set
```

```
print ("tset2[3]: ", set2[-1] )    # Negative Index not allowed in Set
```

- Negative Indexing , Slicing , Updating , + and * not allowed on SETs
 - # set1[0] = 100
 - # set3 = set1 + set2

SET

1. **add()** - Add One Item in SET.

```
s = { 10, 20, 30, 40 }  
s.add(60 )  
print( s)                # { 10, 20, 30 ,40 , 60 }
```

2. **update(list)** - Add more the one Item in SET.

```
s = { 10, 20 }  
s = s.update( [50,60] )  
print( s)                # { 10, 20, 50 , 60 }  
s = s.update( 'ABC' )  
print( s)                # { 10, 20, 50 , 60 , 'A', 'B', 'C' }
```

3. **Loop Through a SET**

```
s1 = { 'physics', 'chemistry', 1997, 2000 }  
for x in s1 :  
    print(x)
```

4. **Length of SET**

```
s= { "apple", "banana", "cherry" }  
print(len(s))
```

5. **Check if Item in SET**

```
s= { "apple", "banana", "cherry" }  
if "apple" in s:  
    print("Yes, 'apple' is in the fruits SET")
```

6. **remove()** - remove the item.

```
a = { 1, 2, 3 }  
print(a.remove(2 ))      # [1, 3]
```

7. **pop()** - remove the last item in unordered SET .

```
a = { 1, 2, 3 }  
print(a.pop( ))          # [1, 3]
```

8. **clear()** and **del** - removes all the items.

```
a = [1, 2, 3]  
print(a.clear( ))        # None
```

9. **max()** - maximum from the SET.

```
a = { 1, 2, 3 }  
print(max(a))            # 3
```

SET

1. Union of Sets

- The union operation produces a new set containing all the distinct elements from both the sets.
- **union()** or **|** used for union operation on two set.

Example

DaysA = {"Mon", "Tue", "**Wed**" }

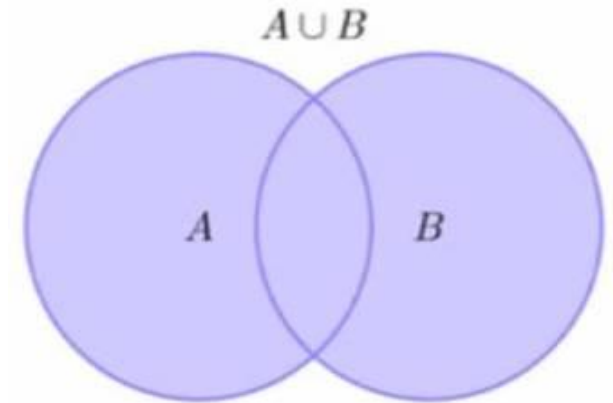
DaysB = { "**Wed**", "Thu", "Fri", "Sat", "Sun" }

D1 = DaysA | DaysB

D2 = DaysA.union(DaysB)

print(D1) ----- {'Thu', 'Sat', 'Tue', 'Sun', 'Mon', 'Fri', 'Wed'}

print(D2) ----- {'Sat', 'Thu', 'Tue', 'Sun', 'Mon', 'Wed', 'Fri'}



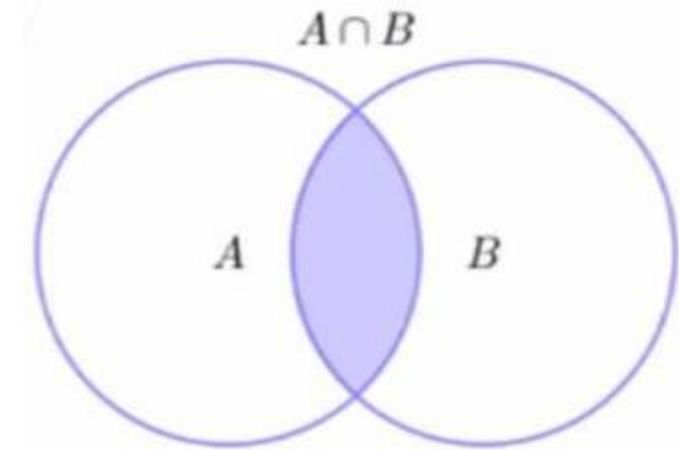
SET

2. Intersection of Sets

- The intersection operation produces a new set containing only the common elements from both the sets.
- **intersection()** or **&** used for intersection operation on two set.

Example

```
DaysA = {"Mon", "Tue", "Wed" }  
DaysB = { "Wed", "Thu", "Fri", "Sat", "Sun" }  
D1 = DaysA & DaysB  
D2 = DaysA.intersection( DaysB )  
print(D1)          ----- { 'Wed' }  
print(D2)          ----- { 'Wed' }
```



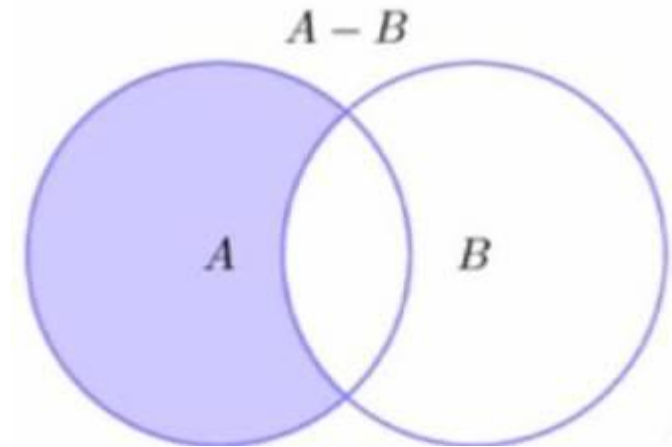
SET

3. Difference of Sets

- The difference operation produces a new set containing only the elements from the first set and none from the second set.
- **difference()** or **-** used for difference operation on two set.

Example

```
DaysA = {"Mon", "Tue", "Wed" }  
DaysB = { "Wed", "Thu", "Fri", "Sat", "Sun" }  
D1 = DaysA - DaysB  
D2 = DaysA.difference( DaysB )  
print(D1)          ----- {'Mon', 'Tue'}  
print(D2)          ----- {'Mon', 'Tue'}
```



SET

4. Disjoint of Sets

- This method generated TRUE if none of the items are present in both sets.
- **isdisjoint()** used for disjoint on two set.

Example

DaysA = {"Mon", "Tue", "**Wed**" }

DaysB = { "**Wed**", "Thu", "Fri", "Sat", "Sun" }

D1 = DaysA.isdisjoint (DaysB)

print(D1) ----- False

SET

5. Compare Set

- The **issubset()** method returns True if all items in the set exists in the specified set, otherwise it returns False. $\leq$ can also be used for subset.
- The **issuperset()** method returns True if all items in the specified set exists in the original set, otherwise it returns False. $\geq$ can also be used for superset

Example

```
DaysA = {"Mon", "Tue", "Wed" }
```

```
DaysB = {"Mon", "Tue", "Wed" ,"Thu","Fri","Sat","Sun" }
```

```
Subset = DaysA <= DaysB
```

```
Superset = DaysB >= DaysA
```

```
print(Subset)          ----- True
```

```
print(Superset)        ----- True
```

SET

6. Symmetric difference

- This method returns a set that contains all items from both set, but removes the items that are present in both sets.

Example

DaysA = {"Mon", "Tue", "Wed" }

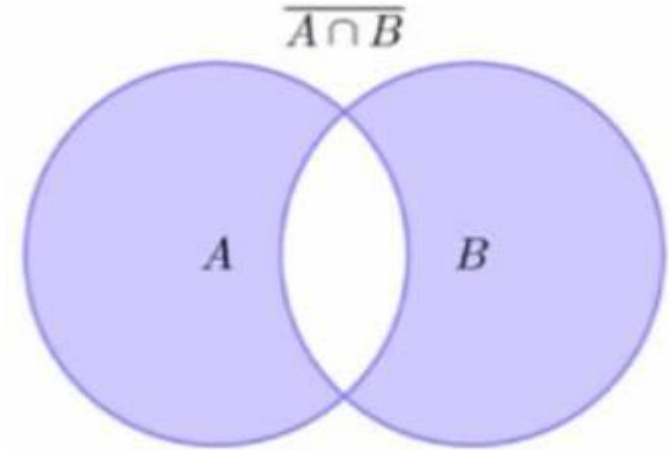
DaysB = {"Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun" }

```
D=DaysA.symmetric_difference(DaysB)
```

```
print(D)
```

Output

```
{'Sat', 'Fri', 'Sun', 'Thu'}
```



References

- www.w3schools.com
- www.tutorialpoint.com
- Python Programming – Pearson – Taneja Kumar

Thank You
Any Query

Data Science using Python Programming

Unit-09: < Dictionary Manipulation and Function >

Contents to be covered

1. Dictionary
2. Basic Operation on dictionary
3. Function
4. Built-in Function
5. Defining User Defined Function

Dictionary

- A dictionary is a collection which is unordered, changeable and indexed.
- Dictionaries are written with curly brackets, and they have keys and values.
- Each key is separated from its value by a colon (:)
- **Keys** are unique and **Values** may not be.
- The values of a dictionary can be of any immutable data type, like strings, numbers, or tuples.
- **dictionary()** – convert the object to dictionary Type.

Dictionary

- **Example**

- dict1 = {
 "Name" : "Ajay",
 "Course" : "Data Science",
 "Duration" : 100
}
- dict2 = {'Name': 'Zara', 'Age' : 7, 'Class' : 'First' }
- dict3 = { } **#Empty Dictionary**

Dictionary

- Access Items
 - To access item, we use the square brackets with the key name as index.
 - **get()** method can also used to get the value of key.
- Example

```
dict1 = { 'Name': 'Zara', 'Age': 7, 'Class': 'First' }  
print ( dict1 )                # {'Name': 'Zara', 'Age': 7, 'Class': 'First'}  
print ( dict1['Age'] )         # 7  
x= dict1['Class']  
print ( x )                    # First  
x= dict1.get('Name')  
print ( x )                    # Zara
```

Dictionary Manipulation

1. Updating Dictionary

```
dict1 = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}  
dict1['Age'] = 10  
print ( dict1['Age'] )           # 10
```

2. Adding new item in Dictionary

```
dict1 = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}  
dict1['Height'] = 4  
print ( dict1 )
```

3. Check if Item in Dictionary

```
dict1 = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}  
if "Age" in dict1:  
    print("Yes, 'Age' is the key in the Dictionary")
```

Think

1. What is the output ?

```
d= {'Name': 'Ajay', 'Age': 7, 'Class': 'First'}  
d['Age']= d['Age'] + 5  
Print(d['Age'])
```

2. What is the code to check the name of student is "Ajay" in dictionary.

```
d= {'Name': 'Ajay', 'Age': 7, 'Class': 'First'}
```

Dictionary - Operation

4. Looping through a Dictionary

```
dict1 = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}
```

#Print all key names in the dictionary, one by one

```
for x in dict1 :
```

```
    print(x)
```

#Print all *values* in the dictionary, one by one

```
for x in dict1:
```

```
    print(dict1[x])
```

using the `values( )` function

```
for x in dict1.values( ):
```

```
    print(x)
```

#Both *keys* and *values*, - `items()` function

```
for x , y in dict1.items( ):
```

```
    print(x , y)
```

Output

Name

Age

Class

Zara

7

First

Zara

7

First

Name Zara

Age 7

Class First

Dictionary - Operation

5. Removing Item – pop(key_name)

```
dict1 = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}  
dict1.pop('Name')  
print(dict1)           # {'Age': 7, 'Class': 'First' }
```

6. Removing Item – popitem () - Last item

```
dict1 = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}  
dict1.popitem()  
print(dict1)           # {'Name': 'Zara', 'Age': 7}
```

7. Remove Item - clear() - removes all the items.

```
dict1 = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}  
dict1.clear()  
print(dict1)           # { }
```

8. del - removes the object.

```
dict1 = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}  
del dict1  
print(dict1)           # Error
```

9. Length of Dictionary

```
dict1 = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}  
print(len(dict1))      # 3
```

10. max() – Maximum of Keys / Values

```
dict1 = {'N1': 67, 'N2': 71, 'N3': 69}  
print( max(dict1))      # N3  
print( max(dict1.values( ))) # 71  
print( max(dict1.keys( ))) # N3
```

Dictionary - Operation

11. Copy a Dictionary

We cannot copy a dictionary simply by typing **dict2 = dict1**, because: dict2 will only be a reference to dict1, and changes made in dict1 will automatically also be made in dict2.

```
dict1 = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}  
dict2=dict1  
dict2['Age'] = 30  
print('Dict1:', dict1)
```

Output

Dict1: {'Name': 'Zara', 'Age': 30, 'Class': 'First'}

Copy() - To make a copy, one way is to use the built-in Dictionary method **copy()**.

Dict() - Another way to make a copy is to use the built-in method or constructor **dict ()**.

```
dict1 = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}  
dict2= dict1.copy( )  
print(dict2)  
dict3=dict(dict1)  
print(dict2)
```

Output

```
{'Name': 'Zara', 'Age': 7, 'Class': 'First'}  
{'Name': 'Zara', 'Age': 7, 'Class': 'First'}
```

Dictionary - Operation

12. Nested Dictionary

```
myfamily = {  
    "child1" : {    "name" : "Emil",    "year" : 2004  },  
    "child2" : {    "name" : "Tobias", "year": 2007  },  
    "child3" : {    "name" : "Linus",   "year" : 2011  }  
    }  
    print(myfamily)
```

Output

```
{'child1': {'name': 'Emil', 'year': 2004}, 'child2': {'name':  
'Tobias', 'year': 2007}, 'child3': {'name': 'Linus', 'year':  
2011}}
```

What is the Output.

```
A={1:100,2:200,3:300,4:400,5:500}  
print (A.items( ) )  
print (A.keys( ) )  
print (A.values( ) )
```

Functions

- A function is a block of organized, reusable code that is used to perform a single, related action. Functions provide better modularity for application and a high degree of code reusing.
- **Types of functions**
 - **Predefined standard library functions** – such as input(), list(), str(), format() etc – These are the functions which already have a definition library files, so we just call them whenever there is a need to use them.
 - **User Defined functions** – The functions that we create in a program are known as user defined functions.
- **Syntax of a function**

```
def function_name( parameters ):
    "function_docstring"
    function_body
    return [expression]
```

Functions -

Built-in Python Functions

SNo	Function	Description
1.	<code>input()</code>	Allowing user input
2.	<code>print()</code>	Prints to the standard output device
3.	<code>int()</code>	Returns an integer number
4.	<code>float()</code>	Returns a floating point number
5.	<code>list()</code>	Returns a list
6.	<code>dict()</code>	Returns a dictionary (Array)
7.	<code>set()</code>	Returns a new set object
8.	<code>str()</code>	Returns a string object
9.	<code>tuple()</code>	Returns a tuple
10.	<code>type()</code>	Returns the type of an object

11.	<code>len()</code>	Returns the length of an object
12.	<code>format()</code>	Formats a specified value
13.	<code>abs()</code>	Returns the absolute value of a number
14.	<code>eval()</code>	Evaluates and executes an expression
15.	<code>round()</code>	Rounds a numbers
16.	<code>max()</code>	Returns the largest item in an iterable
17.	<code>min()</code>	Returns the smallest item in an iterable
18.	<code>oct()</code>	Converts a number into an octal
19.	<code>pow()</code>	Returns the value of x to the power of y
20.	<code>range()</code>	Returns a sequence of numbers, starting from 0 and increments by 1 (by default)

Functions – Built-in

1. eval() Function - evaluates the expression.

```
x= 'print(55)'
eval(x)
x= eval( '10+5' )
print(x)
```

Output-

```
55
5
```

2. type() Function - the type of the specified object.

```
a = ('apple', 'banana', 'cherry')
b = "Hello World"
c = 33
print(type(a) )           ---- <class 'tuple'>
print(type(b))            ---- <class 'str'>
print(type(c))            ---- <class 'int'>
```

3. range() Function - returns a sequence of numbers.

#sequence of numbers from 0 to 5

```
x = range(6)
for n in x :
    print(n)
```

#sequence of numbers from 3 to 5

```
x = range(3, 6)
for n in x :
    print(n)
```

#sequence of num from 3 to 19, increment by 2

```
x = range(3, 20, 2)
for n in x :
    print(n)
```

Functions – Built-in

4. format() Function - value into a specified format

`format(value, format)`

`x = format(0.5, '%')` ----- 50.000000%

`x = format(255, 'x')` ----- ff

Parameter	Description
<i>value</i>	A value of any format
<i>format</i>	The format you want to format the value into. Legal values: '<' - Left aligns the result (within the available space) '>' - Right aligns the result (within the available space) '^' - Center aligns the result (within the available space) '=' - Places the sign to the left most position '+' - Use a plus sign to indicate if the result is positive or negative '-' - Use a minus sign for negative values only ' ' - Use a leading space for positive numbers ',' - Use a comma as a thousand separator '_' - Use an underscore as a thousand separator 'b' - Binary format 'c' - Converts the value into the corresponding Unicode character 'd' - Decimal format 'e' - Scientific format, with a lower case e 'E' - Scientific format, with an upper case E 'f' - Fix point number format 'F' - Fix point number format, upper case 'g' - General format 'G' - General format (using a upper case E for scientific notations) 'o' - Octal format 'x' - Hex format, lower case 'X' - Hex format, upper case 'n' - Number format '%' - Percentage format

Functions

User Defined Function

1. A function definition is the actual function and start with the “**def**” keyword.
2. The definition contains the code that will be executed.
3. The first line of a function definition, called the function header, and specifies the parameter list.
4. A function header end with a colon.
5. The header is the function body, containing the statements that the function will perform.
6. The function body consists of indented statements.
7. A **return** statement should be included to specify the returning a value.

8. Example

```
def hello( ) :  
    print("Hello How Are You ! ")
```

Functions

Parameters

- The parameter list consists of none or more parameters.
- Parameters are called arguments, if the function is called.
- Parameter can be mandatory or optional.
- The optional parameters (zero or more) must follow the mandatory parameters.
- **Example**

```
def AddOne( x) :  
    return x+1
```

```
n=5
```

```
print( AddOne(n) )      ---- Output is 6
```

Functions

Return Values

- If a function return a value, use the return statement.

```
def my_function(x):  
    return 5 * x
```

```
print(my_function(3))  
print(my_function(5))
```

The pass Statement

- A function definitions cannot be empty, but if you for some reason have a function definition with no content, put in the pass statement to avoid getting an error.

```
def myfunction( ):  
    pass  
a=10  
print("Value=", a)
```

Functions

Program : Write a function to calculate the sum of 2 no and print the result.

```
def sum(a , b):  
    "function to calculate the sum of 2 no"  
    r=a+b  
    print("sum =",r)
```

```
a=10  
b=20  
r=sum(a,b)
```

Program : Write a function to calculate the sum of 2 no and return the result.

```
def sum(a , b):  
    "function to calculate the sum of 2 no"  
    r=a+b  
    return(r)
```

```
a=10  
b=20  
r=sum(a,b)  
  
print("sum =",r)
```

Functions

Program : Write a function to find the maximum of 2 no and return the result.

```
def MAX(a,b):  
    "function to calculate the sum of 2 no"  
    if a>b :  
        r=a  
    else:  
        r=b  
    return(r)
```

```
a=10  
b=20  
r=MAX(a,b)  
print("Maximum =",r)
```

Program : Write a function to calculate the factorial of number and return the result.

```
def fact(n):  
    "function to calculate the factorial of no"  
    f=1  
    i=1  
    while ( i<=n)  
        f=f* i  
        i=i+1  
    return(f)
```

```
a=5  
r=fact(a)  
print("fact=",r)
```

Functions

Maintaining reference of the passed object and appending values in the same object .

Program : Write a function to modify the items in the existing list.

```
def changeme( mylist ):  
    "This changes a passed list into this function"  
    print ("Values inside the function before change: ", mylist )  
    mylist[2]=50  
    print ("Values inside the function after change: ", mylist )  
    return
```

```
# Now you can call changeme function  
mylist = [10,20,30]  
changeme( mylist )  
print ("Values outside the function: ", mylist)
```

Output

Values inside the function before change:
[10, 20, 30]

Values inside the function after change:
[10, 20, 50]

Values outside the function: [10, 20, 50]

Functions

Argument is being passed by reference and the reference is being overwritten inside the called function

Program : Write a function to modify the items in the existing list.

```
def changeme( mylist ):
```

```
    "This changes a passed list into this function"
```

```
    mylist = [1,2,3,4]
```

```
    # This would assi new reference in mylist
```

```
    print ("Values inside the function: ", mylist)
```

```
    return
```

```
# Now you can call changeme function
```

```
mylist = [10,20,30]
```

```
changeme( mylist )
```

```
print ("Values outside the function: ", mylist)
```

Output

Values inside the function: [1, 2, 3, 4]

Values outside the function: [10, 20, 30]

References

- www.w3schools.com
- www.tutorialpoint.com
- Python Programming – Pearson – Taneja Kumar

Thank You
Any Query

Data Science using Python Programming

Unit-10: < Function Arguments Type>

Contents to be covered

1. Function Arguments
2. Required arguments
3. Keyword arguments(**kwargs**)
4. Default arguments
5. Variable-length arguments or Arbitrary Arguments(***args**)
6. Arbitrary Keyword Arguments (****kwargs**)

Arguments or Parameters

- Information can be passed into functions as arguments.
- Arguments are specified after the function name, inside the parentheses.
- We can add many arguments separating them with a comma.

- Example

```
def printme( name, age )
```

```
def sum( a, b, c )
```

Required arguments

- Required arguments are the arguments passed to a function in correct positional order.
- The number of arguments in the function call should match exactly with the function definition.

- **# Function definition**

```
def printme( name, age ):  
    "This prints a passed string into this function"  
    print (name , age)
```

- **# Function Call**

```
printme("Ajay",30)
```

- To call the function **printme()**, it is definitely need to pass one argument, otherwise it gives a syntax error.

Keyword Arguments (*kwargs*)

- Keyword arguments in a function call, allows to identifies the arguments by the parameter name.
- This allows us to skip arguments or place them out of order. The interpreter is able to use the keywords provided to match the values with parameters.

- **# Function definition**

```
def printme( name, age ):  
    "This prints a passed string into this function"  
    print (name , age)
```

- **# Function Call**

```
printme(age=30, name="Ajay")
```


Default Arguments

- A default argument is an argument that assumes a default value if a value is not provided in the function call for that argument.

Function definition

```
def printme( name, age=40 ):  
    "This prints a passed string into this function"  
    print (name , age)
```

#Function Call

```
printme(age=30, name="Ajay")  
printme(name="Ajay")
```

Output

Ajay 30

Ajay 40

Variable-length Arguments or Arbitrary Arguments(*args)

- If you do not know how many arguments that will be passed into your function, add an asterisk (*) before the parameter name in the function definition.
- This variable name holds the values of all non keyword variable arguments.
- The function will receive a *tuple* of arguments, and can access the items accordingly. This tuple remains empty if no additional arguments are specified during the function call.

Variable-length Arguments or Arbitrary Arguments(*args)

Example -1

```
def my_function(*kids):  
    print("The youngest child is " + kids[2])  
  
my_function("Emil", "Tobias", "Linus")
```

Output- The youngest child is Linus

Example -2

```
def printno( arg1, *vartuple ):  
    "This prints a variable passed arguments"  
    print ("Output is: ")  
    print (arg1)  
  
    for var in vartuple:  
        print (var)  
  
printno( 10 )#vartuple is empty  
printno( 70, 60, 50 )
```

Output is:

10

70

60

50

Arbitrary Keyword Arguments (****kwargs**)

- If you do not know how many keyword arguments that will be passed into your function, add two asterisk: ****** before the parameter name in the function definition.
- This way the function will receive a *dictionary* of arguments, and can access the items accordingly

- ```
def my_function(**kid):
 print("His last name is " + kid["lname"])
```

```
my_function(fname = "Ajay", lname = "Kumar")
```

# Recursion

- Python also accepts function recursion, which means a defined function can call itself.

```
def fact(k):
```

```
 "This function returns the factorial of a number"
```

```
 if(k > 0):
```

```
 f = k * fact(k - 1)
```

```
 else:
```

```
 f = 1
```

```
 return f
```

```
print("\nRecursion Example Results")
```

```
r=fact(5)
```

```
print("factorial=",r)
```

# The Anonymous Functions

- These functions are called anonymous because they are not declared in the standard manner by using the **def** keyword.
- You can use the **lambda** keyword to create small anonymous functions.
- Lambda forms can take any number of arguments but return just one value in the form of an expression. They cannot contain commands or multiple expressions.
- An anonymous function cannot be a direct call to print because lambda requires an expression.
- Lambda functions have their own local namespace and cannot access variables other than those in their parameter list and those in the global namespace.
- Syntax

**lambda [arg1 [,arg2,.....argn]]:expression**

# The Anonymous Functions

```
Function definition is here
sum = lambda arg1, arg2: arg1 + arg2
```

```
Now you can call sum as a function
print ("Value of total : ", sum(10, 20))
print ("Value of total : ", sum(20, 20))
```

```
#Fucntion Definition
SQR=lambda x: x * x
CUB=lambda x: x*x*x
```

```
n=3
print(SQR(n))
print(CUB(n))
```

## Output

```
9
27
```

# References

- [www.w3schools.com](http://www.w3schools.com)
- [www.tutorialpoint.com](http://www.tutorialpoint.com)
- Python Programming – Pearson – Taneja Kumar



Thank You  
Any Query

# Data Science using Python Programming

## Unit-11: < Python Modules and Packages>

# Contents to be covered

1. Scope and Namespace
2. LEGB Rule
3. Modules
4. Packages

# Scope and Namespace

## Scope

- The scope is a region in the program where variables and names are accessible.
- It also determines the visibility and lifetime of a variable in the program code.
- The names or objects which are accessible are called **in-scope**.
- The names or objects which are not accessible are called **out-of-scope**.
- The Python scope concept follows the LEGB (Local, Enclosing, Global and built-in) rule.
- The scope concept helps to avoid the name collision and use of global names across the programs.

# Scope and Namespace

## Names and Objects in Python

- When we assign the value to the variable for the first time, it creates the variable.
- Example
  - Assignment `X=value`
  - Import Operation `import module`
  - Function definitions `def fun( )`
  - Arguments in the function `def fun( x1, x2 , x3 )`
  - Class definition `class abc :`
- The location of the name assignment or definition used to associate it with a scope.
- If a variable assigned value in the function, it has a **local scope**.
- If a variable assigned value at the top level and outside all the functions, it has a **global scope**.

# Scope and Namespace

## Types of Namespace

1. **Built-in Namespace** : It includes functions and exception names that are built-in in the python.
2. **Global Namespace** : It includes the names from the modules that are imported in the code. It lasts till the end of program.
3. **Local Namespace** : It includes the names defined in the function. It is created when the function is called and ends when the value returned.

# Scope and Namespace

## LEGB Rule for Python Scope

- **Local (or function) scope** - This Python scope contains the names that you define inside the function.
- **Enclosing (or nonlocal) scope** - It is a special scope that only exists for nested functions.
- **Global (or module) scope** - It is the top-most scope in a Python program, script, or module.
- **Built-in scope** - This names that are built into Python and available from everywhere in your code.

# Scope and Namespace

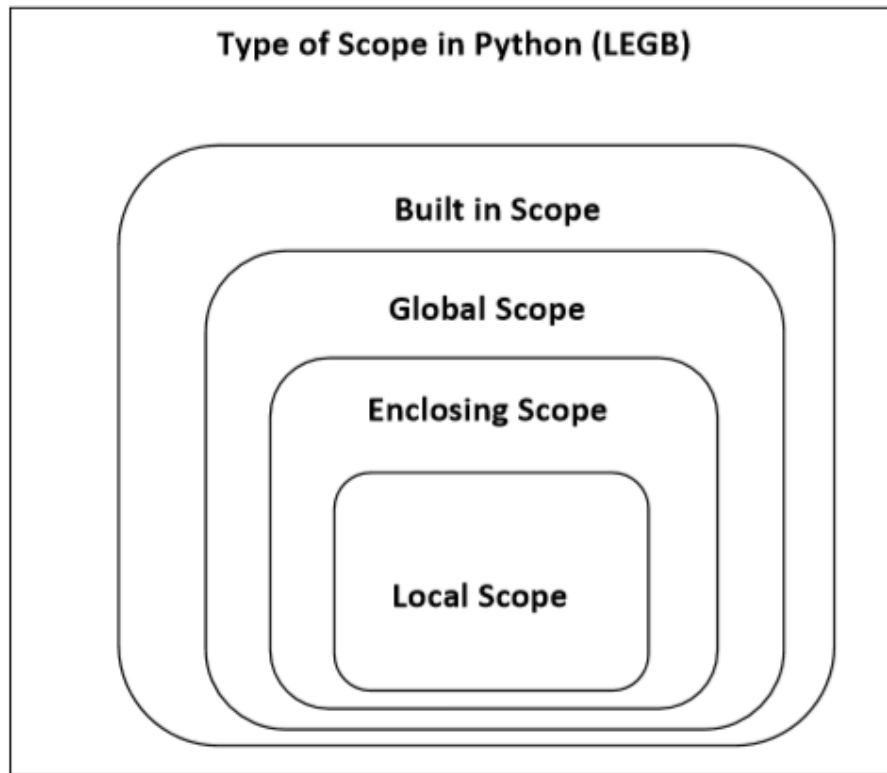
## LEGB Rule for Python Scope

- The LEGB stands for Local, Enclosing, Global and Built-in.
- Python resolves names using the LEGB rules.
- The LEGB rule is a kind of name lookup procedure, which determines the order in which Python looks up names.
- Whenever we use a name, such as a variable or a function name, Python searches through different scope levels (the local, enclosing, global, and built-in scope) to determine whether the name exists or not.
- If the name exists, then we always get the first occurrence of it. Otherwise, we get an error.



# Scope and Namespace

## LEGB Rule for Python Scope



### Example

#var1 is in the global namespace

```
var1 = 5
```

```
def ABC():
```

```
 # var2 is in the local namespace
```

```
 var2 = 6
```

```
 def XYZ():
```

```
 # var3 is in the nested local
```

```
 # namespace
```

```
 var3 = 7
```

**var1** is declared in the global namespace.

**var2** is inside the ABC( ).

**var3** has a local scope in XYZ( ).

# Modules

## Modules

- Module is a Python file containing a set of Python Statements and Functions.
- Module files has the “.py” extension. e.g. my\_abc.py
- Create a module -
  - File **my\_abc.py**

```
def show(name, age) :
 print("Name : " , name)
 print("Age : " , age)
```

```
Employee= { "name":"Ajay", "age":30, "Salary":20000 }
```

# Modules

## Import Module

- Import Statement help to provide the Module contents to the caller program code.
- Import module created a separate namespace, which contains all the objects define in the module.
- Module content like identifiers , functions , class or objects defined in the module can be accessed using the dot notation like

<ModuleName>.<FunctionName>.

eg. my\_abc.show( )

- **Syntax**

```
import Module_Name
```

```
import Module_Name as New_Name # to rename the module
```

# Modules

File **my\_abc.py**

```
def show(name, age) :
```

```
 print("Name : " , name)
```

```
 print("Age : " , age)
```

```
Employee= { "name":"Ajay",
 "age":30, "Salary":20000 }
```

```
import my_abc
```

```
print("\n Accessing function from Module\n")
```

```
my_abc.show("vikas",27)
```

```
print("\n Accessing Variable from Module\n")
```

```
print(my_abc.Employee["name"])
```

```
print(my_abc.Employee["age"])
```

```
print(my_abc.Employee["Salary"])
```

## Output

Accessing function from Module

Name : vikas

Age : 27

Accessing Variable from Module

Ajay

30

20000

# Modules

## Using the from ...import

- This is used to import the part of the module as required.
- If the module is imported using the “from..import” syntax, then module name is not used before the function or variable imported.

- **Syntax**

from my\_abc import show

---- import the show

from my\_abc import Employee

---- import the Employee

from my\_abc import \*

---- import all object from Module

- For example if **show( )** is imported from **my\_abc** module, we can't use **my\_abc.show( )**, instead use **show( )** only.

# Modules

File **my\_abc.py**

```
def show(name, age) :
```

```
 print("Name : " , name)
```

```
 print("Age : ", age)
```

```
Employee= { "name":"Ajay",
 "age":30, "Salary":20000 }
```

```
from my_abc import show
```

```
print("\n Accessing function from Module\n")
show("vikas", 27)
```

```
print("\n Accessing Variable from Module\n")
print(Employee["name"]) # This is not accessible, as not imported.
```

## Output

```
Accessing function from Module
Name : vikas
Age : 27
```

```
Accessing Variable from Module
Traceback (most recent call last): File "C:/Python38-32/m1.py",
line 7, in <module> print(Employee["name"]) NameError: name
'Employee' is not defined
```

# Modules

File **my\_abc.py**

```
def show(name, age) :
```

```
 print("Name : ", name)
```

```
 print("Age : ", age)
```

```
Employee= { "name":"Ajay",
 "age":30, "Salary":20000 }
```

```
from my_abc import *
```

```
print("\n Accessing function from Module\n")
show("vikas", 27)
```

```
print("\n Accessing Variable from Module\n")
print(Employee["name"])
```

## Output

```
Accessing function from Module
Name : vikas
Age : 27
```

```
Accessing Variable from Module
Ajay
```

# Packages

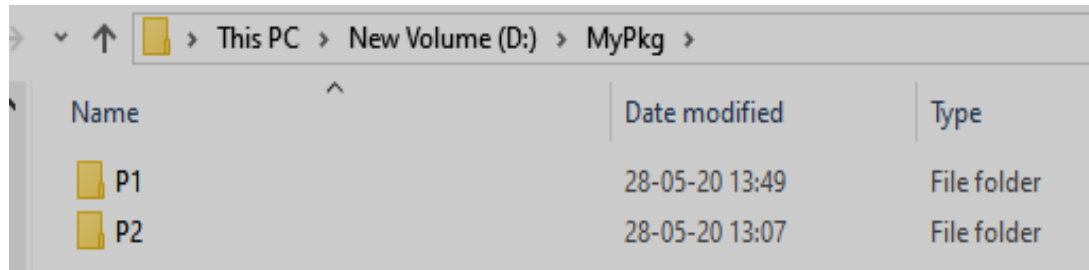
- A package is a well-organized modules.
- The directory is a package [or sub package] and files are modules.
- Package is a place for similar modules, and help in grouping and organizing.
- Package directory or sub-directory must contain a file “**`__init__.py`**” used as initialization code. It can be an empty file. However, it new version of python, this is not compulsory.
- Packages help avoid collisions between module names and modules help avoid collision between variable names.
- Packages are helpful to manage the large projects having lots of modules.



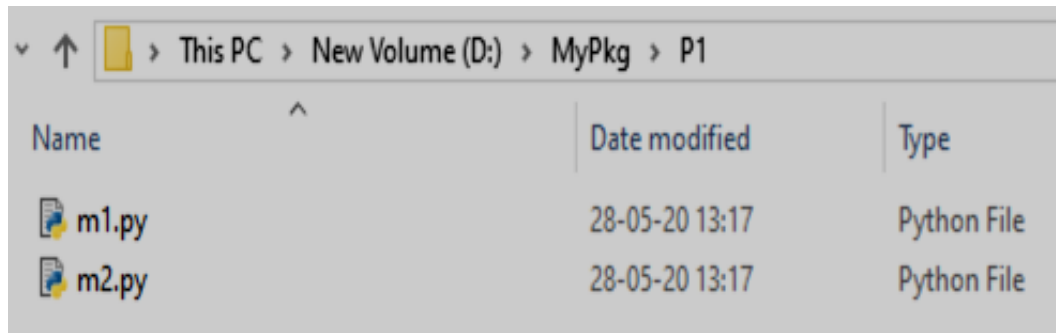
# Packages

To create a Package – **MyPkg** perform the following steps –

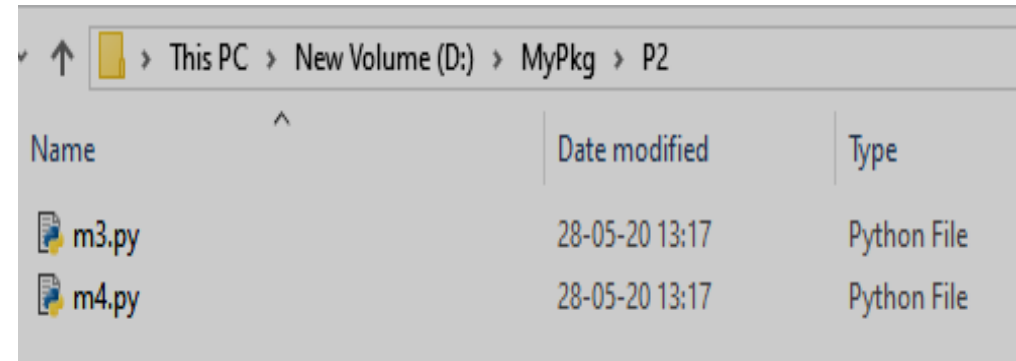
1. Create a folder on “D:\MyPkg”
2. Create a sub folder in D:\MyPkg – P1 & P2



3. Create a module **m1.py** , **m2.py** in D:\MyPkg\P1 sub folder.



4. Create a module m3.py and m4.py in D:\MyPkg\P2 sub folder.



5. Write the Content of the Module as given.

# Packages

## Module M1.py

```
Employee={ "name":"Ajay M1", "age":27,
"salary":30000}
```

```
def show(name):
 print("Name :", name)
```

## Module M2.py

```
Employee={ "name":"Vijay M2", "age":27,
"salary":30000}
```

```
def sum(x , y):
 return(x+y)
```

## Module M3.py

```
def show(name, age):
 print("Name :", name)
 print("Age :", age)
```

```
Employee={ "name":"Ajay M3", "age":27,
"salary":30000}
```

## Module M4.py

```
def mul(x,y):
 print("sum : ", x*y)
```

```
Employee={ "name":"Vijay M4", "age":27,
"salary":30000}
```

# Packages

## Import Package

- Import Statement help to provide the Module content in the Package to the caller program code.
- Import Package creates a separate namespace for each sub package, which contains all the objects define in the module of Package.
- We can provide the path for package using **sys.path.append("Package Path")** in the code or modify the **PYTHONPATH** environment variable.

- **Syntax**

```
import Package_Name.Module_Name
```

# Packages

```
import sys
sys.path.append("D:\\")
import MyPkg.P1.m1
import MyPkg.P1.m2

print("Objects in P1 - m1 module ")
print(dir(MyPkg.P1.m1))

print("Objects in P1 - m2 module ")
print(dir(MyPkg.P1.m2))

print("Accessing P1 - m1 module ")
MyPkg.P1.m1.show("Ajay")

print("Accessing P1 - m1 module ")
print(MyPkg.P1.m1.Employee["name"])

print("Accessing P1 - m2 module ")
print("Sum : ", MyPkg.P1.m2.sum(20,20))

print("Accessing P1 - m2 module ")
print(MyPkg.P1.m2.Employee["name"])
```

## Output

Objects in **P1 - m1** module

```
['Employee', '__builtins__', '__cached__', '__doc__', '__file__',
 '__loader__', '__name__', '__package__', '__spec__', 'show']
```

Objects in **P1 - m2** module

```
['Employee', '__builtins__', '__cached__', '__doc__', '__file__',
 '__loader__', '__name__', '__package__', '__spec__', 'sum']
```

Accessing P1 - m1 module

Name : Ajay

Accessing P1 - m1 module

Ajay M1

Accessing P1 - m2 module

Sum : 40

Accessing P1 - m2 module

Vijay M2

# Packages

## Using the from ...import

- If the module from the package is imported using the “**from..import**” syntax, then package name is not used before the function or variable imported, as applicable in DOT Notation.
- Syntax
  - from Package\_Name import Module\_Name**
  - from Package\_Name import \***
- e.g. if **show( )** is imported from **P1.m1** module, we use **show( )** instead of **P1.m1.show( )**.
- For module , import \* allows importing all the objects defined in the module.
- For Package, import \* does not import anything. It will allow importing, only if the **\_\_init\_\_.py** file is present in the folder and **\_\_all\_\_** is defined in it.
- Content of **\_\_init\_\_.py** file present in the “D:\MyPkg\P1” folder  
**\_\_all\_\_=["m1","m2"]**

# Packages

## Example-1

```
import sys
sys.path.append("D:\\")

from MyPkg.P1.m1 import show
from MyPkg.P1.m2 import sum

print("Objects loaded in Memory")
print(dir())

print("Accessing P1 - m1 module ")
show("Ajay")

print("Accessing P1 - m2 module ")
print("Sum :", sum(20,20))

print("\n Accessing Variable from Module\n")
print(Employee["name"])
This is not accessible, as not imported.
```

## Output

### Objects loaded in Memory

```
['_annotations_', '__builtins__', '__doc__', '__file__',
 '__loader__', '__name__', '__package__', '__spec__', 'show',
 'sum', 'sys']
```

### Accessing P1 - m1 module

Name : Ajay

### Accessing P1 - m2 module

Sum : 40

### Accessing P1 - m1 module

Traceback (most recent call last):

File "D:\Python38\p3.py", line 17, in <module>

print( Employee["name"])

NameError: name 'Employee' is not defined

# Packages

## Example-2

```
import sys
sys.path.append("D:\\")

from MyPkg.P1.m1 import Employee
from MyPkg.P1.m2 import Employee

print("Objects loaded in Memory")
print(dir())

print("Accessing P1 - m1 module ")
print(Employee["name"])

print("Accessing P1 - m2 module ")
print(Employee["name"])
```

## Output

Objects loaded in Memory

```
['Employee', '__annotations__', '__builtins__', '__doc__',
 '__file__', '__loader__', '__name__', '__package__', '__spec__',
 'sys']
```

Accessing **P1 - m1** module  
Vijay M2

Accessing **P1 - m2** module  
Vijay M2

In this, **Employee** object is present in both the modules (**m1 and m2**), so the **Employee** object from the module-m2 replaces **Employee** from the module-m1. Thus, output is same in above case. **To resolve this issue, follow the following example**

# Packages

## Example-3

```
import sys
sys.path.append("D:\\")

from MyPkg.P1.m1 import Employee as E1
from MyPkg.P1.m2 import Employee as E2

print("Objects loaded in Memory")
print(dir())

print("Accessing P1 - m1 module ")
print(E1["name"])

print("Accessing P1 - m2 module ")
print(E2["name"])
```

## Output

Objects loaded in Memory

```
['E1', 'E2', '__annotations__', '__builtins__', '__doc__',
 '__file__', '__loader__', '__name__', '__package__',
 '__spec__', 'sys']
```

Accessing P1 - m1 module  
Ajay M1

Accessing P1 - m2 module  
Vijay M2



# Packages

Example-4

#use of `__init__.py` and `__all__`

```
import sys
sys.path.append("D:\\")

from MyPkg.P1 import *

print("Objects loaded in Memory")
print(dir())

print("Accessing P1 - m1 module ")
print(m1.Employee["name"])

print("Accessing P1 - m2 module ")
print(m2.Employee["name"])
```

## Output

Objects loaded in Memory

```
['__annotations__', '__builtins__', '__doc__', '__file__',
 '__loader__', '__name__', '__package__', '__spec__',
 'm1', 'm2', 'sys']
```

Accessing P1 - m1 module

Ajay M1

Accessing P1 - m2 module

Vijay M2

# Packages

## Example-5

#\_\_init\_\_.py file not present in D:\MyPkg\P2 folder

```
import sys
sys.path.append("D:\\")
```

```
from MyPkg.P2 import *
```

```
print("Objects loaded in Memory")
print(dir())
```

```
print("Accessing P1 - m1 module ")
print(m3.Employee["name"])
```

```
print("Accessing P1 - m2 module ")
print(m4.Employee["name"])
```

## Output

### Objects loaded in Memory

```
['__annotations__', '__builtins__', '__doc__', '__file__',
'__loader__', '__name__', '__package__', '__spec__',
'sys']
```

### Accessing P1 - m1 module

Traceback (most recent call last):

File "D:/Python38/p4.py", line 12, in <module>

print( m3.Employee["name"])

NameError: name 'm3' is not defined

# References

- [www.w3schools.com](http://www.w3schools.com)
- [www.tutorialpoint.com](http://www.tutorialpoint.com)
- Python Programming – Pearson – Taneja Kumar

Thank You  
Any Query

# Data Science using Python Programming

## Unit-12: < NumPy Module >

# Contents to be covered

1. NumPy Installation
2. Using NumPy
3. Arrays in NumPy
4. Creating Matrix - Empty , One, Zero
5. Aggregate Function on NumPy

# NumPy Module

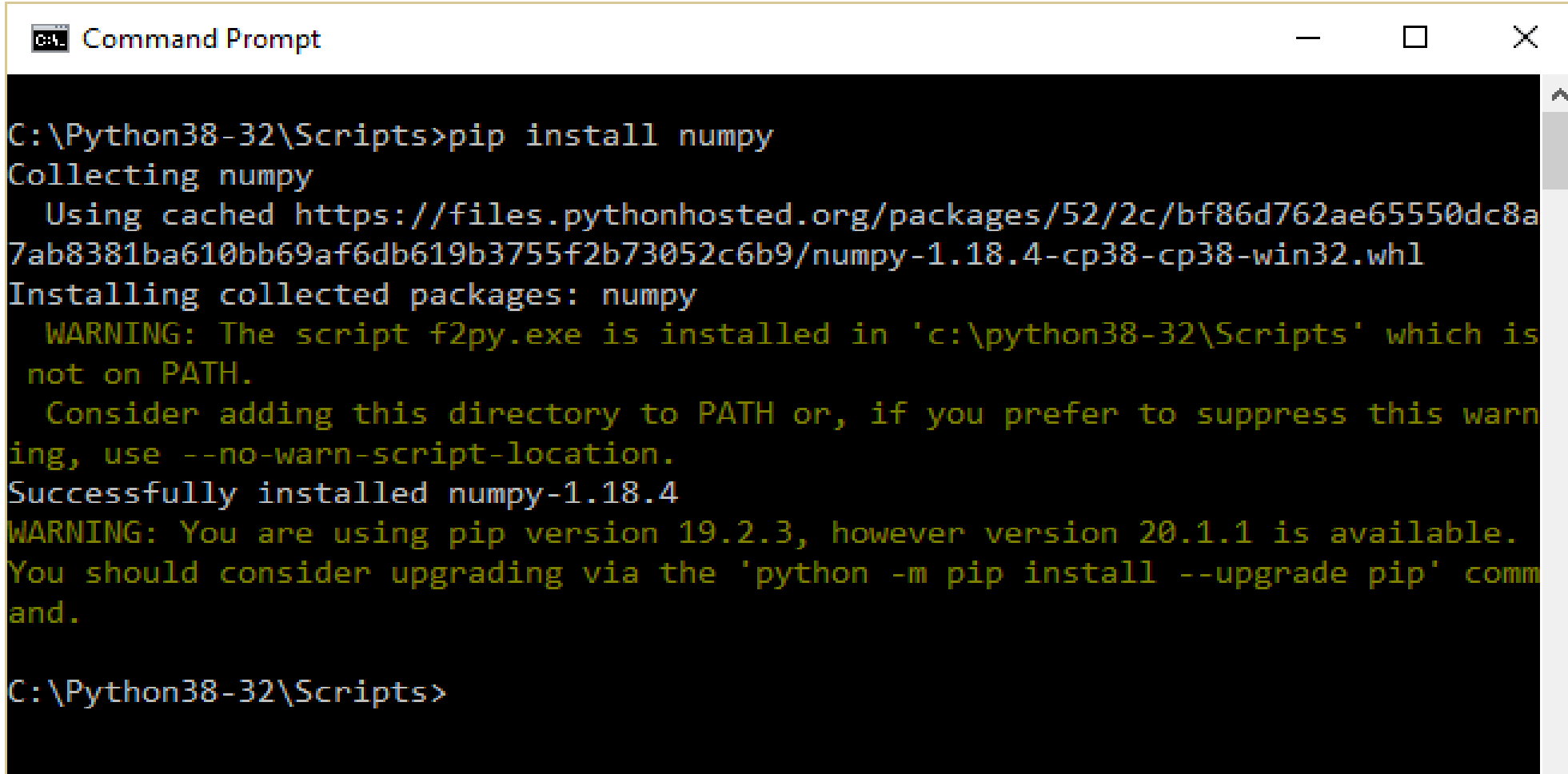
- NumPy is python package and it stands for Numerical Python.
- It was created in 2005 by Travis Oliphant. It is an open source project.
- It is core python library used working with arrays and for scientific computing.
- It also has inbuilt functions for working with linear algebra, Fourier transform, matrices and data science.
- It has an N-dimensional array object(**ndarray**) which is in the form of rows and columns.
- NumPy arrays are stored at one continuous place in memory, so it provides the fast access and processing. The NumPy follows the Locality of Reference.

# Installation of Numpy

- Numpy is not available as bundled package in Python. We can install it using PIP.
- Steps to install the NumPy in Python.
  - a) Go to command prompt -
    - a) Type cmd on RUN or click the command prompt
  - b) Change the folder to location where python installed.
    - a) `cd c:\Python38-32`
  - c) Move to the folder **Scripts** in **C:\python38-32**.
    - a) `c:\Python38-32> cd scripts`
    - b) `c:\Python38-32\Scripts>`
  - d) Type the **PIP install Numpy** and press the enter key. Keep the internet connection active during this process. It will download the package and install it automatically.
    - a) `c:\Python38-32\Scripts> pip install numpy`
  - e) Or **python -m pip install numpy**



# Installation of NumPy



```
C:\Python38-32\Scripts>pip install numpy
Collecting numpy
 Using cached https://files.pythonhosted.org/packages/52/2c/bf86d762ae65550dc8a7ab8381ba610bb69af6db619b3755f2b73052c6b9/numpy-1.18.4-cp38-cp38-win32.whl
Installing collected packages: numpy
 WARNING: The script f2py.exe is installed in 'c:\python38-32\Scripts' which is not on PATH.
 Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
Successfully installed numpy-1.18.4
WARNING: You are using pip version 19.2.3, however version 20.1.1 is available.
You should consider upgrading via the 'python -m pip install --upgrade pip' command.

C:\Python38-32\Scripts>
```

# Using of NumPy

## Example

```
import numpy as np
arr1=np.array(1)
arr2=np.array([1,2,3,4,5])
print('Numpy Version : ', np.__version__)
print('Array 1 Values : ', arr1)
print('Array 2 Values : ', arr2)
print('Array 1 Data Type : ', type(arr1))
print('Array 2 Data Type : ', type(arr2))
```

## Output

```
Numpy Version : 1.18.4
Array 1 Values : 1
Array 2 Values : [1 2 3 4 5]
Array 1 Data Type : <class 'numpy.ndarray'>
Array 2 Data Type : <class 'numpy.ndarray'>
```

# Dimension in NumPy

- A dimension in arrays is one level of array depth.
- Types of Arrays
  - 0-D arrays :** It has a scalar element in an array. Each value in an array is a 0-D array.  
e.g. `numpy.array( 10 )`
  - 1-D arrays :** An array that has 0-D arrays as its elements is called uni-dimensional or 1-D array.  
e.g. `numpy.array( [ 30 , 40 ] )`
  - 2-D arrays :** An array that has 1-D arrays as its elements is called a 2-D array.  
e.g. `numpy.array( [ [ 30 , 40 ] , [ 50 , 60 ] ] )`
  - 3-D arrays :** An array that has 2-D arrays (matrices) as its elements is called 3-D array.  
e.g. `numpy.array( [ [ [ 30 , 40 ] , [ 50 , 60 ] ] ,  
                  [ [ 70 , 80 ] , [ 90 , 10 ] ]  
                  ] )`
- `<array_name>.ndim` used to know the dimension of the array.

# Dimension in NumPy

## Example

### # 1D Array

```
import numpy as np
arr1=np.array(1)
arr2=np.array([1,2,3,4,5])
print('Array -1')
print(arr1)
print('Array Dim :', arr1.ndim)
print('Array -2')
print(arr2)
print('Array Dim :', arr2.ndim)
```

## Output

```
Array -1
1
Array Dim : 0

Array -2
[1 2 3 4 5]
Array Dim : 1
```

## Example

### # 2D Array

```
import numpy
arr3=numpy.array([[1,2,3],
[4,5,6]])

print('Array -3')
print(arr3)
print('Array Dim :', arr3.ndim)
```

## Output

```
Array -3
[[1 2 3]
 [4 5 6]]
Array Dim : 2
```

## Example

### # 3D Array

```
import numpy

arr4=numpy.array(
 [[[1,2,3], [4,5,6]],
 [[1,2,3], [4,5,6]]
])

print('Array -4')
print(arr4)
print('Array Dim :', arr4.ndim)
```

## Output

```
Array -4
[[[1 2 3]
 [4 5 6]]
 [[1 2 3]
 [4 5 6]]]
Array Dim : 3
```

# Access Array in NumPy

|                |    |    |    |    |
|----------------|----|----|----|----|
| Array a        | 1  | 2  | 3  | 4  |
| Index          | 0  | 1  | 2  | 3  |
| Negative Index | -4 | -3 | -2 | -1 |

## 1-D Array

```
import numpy as np
a = np.array([1, 2, 3, 4])
print('First Element : ', a[0])
print('Last Element : ', a[3])
print('Last Element : ', a[-1])
```

## 2-D Array

| Index | 0 | 1 |
|-------|---|---|
| 0     | 1 | 2 |
| 1     | 3 | 4 |

Index for First Row Values 1- (0, 0) 2- (0, 1)  
Index for Second Row Values 3- (1, 0) 4- (1, 1)

## Negative Index for the Elements

Index for First Row Values 1- (-2, -2) 2- (-2, -1)  
Index for Second Row Values 3- (-1, -2) 4- (-1, -1)

## 2-D Array

```
import numpy as np
a = np.array([[1, 2],[3, 4]])
print('Array Elements')
print(a)
```

```
print('Array Elements using Index')
print(a[0, 0] , a[0, 1])
print(a[1,0], a[1,1])
```

```
print('Array using Negative Index')
print(a[-2, -2], a[-2, -1])
print(a[-1,-2], a[-1,-1])
```

# Access Array in NumPy

## Slicing for Accessing Array Elements

We can access the elements of array by using the slicing [**start: end**] or [**start : end : step**].

|                |                                                                  |
|----------------|------------------------------------------------------------------|
| [ 1 : 4 ]      | means starting from 1 to 3 ( 4 is not included).                 |
| [ : 4 ]        | means starting from 0 to 3.                                      |
| [ 1: ]         | means starting from 1 upto end.                                  |
| [1 : 4 : 1 ]   | means 1 , 2 , 3 . step 1 indicates the change in the index value |
| [1 : 4 : 2 ]   | means 1, 3 . step value is 2.                                    |
| [3 : 0 : -1]   | means 3, 2, 1 . step value is -1.                                |
| [-1 : -3 : -1] | means -1, -2. step value is -1.                                  |

# Access Array in NumPy

## Example -1

### # Slicing on 1D Array

```
import numpy as np
a = np.array([1, 2, 3, 4])
print('Array Elements')
print(a)

print('Dimension of Array: ', a.ndim)
print('Array Elements using Slicing Index')

print('L-1 :', a[1:4])
print('L-2 :', a[:4])
print('L-3 :', a[1:])
print('L-4 :', a[:])
print('L-5 :', a[1:4:1])
print('L-6 :', a[1:4:2])
print('L-7 :', a[3:0:-1])
print('L-8 :', a[-1:-3:-1])
```

|                |    |    |    |    |
|----------------|----|----|----|----|
| Array a        | 1  | 2  | 3  | 4  |
| Index          | 0  | 1  | 2  | 3  |
| Negative Index | -4 | -3 | -2 | -1 |

### Array Elements using Slicing Index

L-1 : [2 3 4]

L-2 : [1 2 3 4]

L-3 : [2 3 4]

L-4 : [1 2 3 4]

L-5 : [2 3 4]

L-6 : [2 4]

L-7 : [4 3 2]

L-8 : [4 3]

# Access Array in NumPy

## Example -2

### #Slicing on 2D Array

```
import numpy as np
```

```
a = np.array([[1, 2, 3,4,5] , [6,7,8,9,10]])
```

```
print('Array Elements')
```

```
print(a)
```

```
print('Dimension of Array: ', a.ndim)
```

```
print('Array Elements using Slicing Index')
```

```
print('L-1 :', a[1,1:4])
```

```
print('L-2 :', a[1, :4])
```

```
print('L-3 :', a[1, 1:])
```

```
print('L-4 :', a[1 , :])
```

```
print('L-5 :', a[1 , 1:4:1])
```

```
print('L-6 :', a[1 ,1:4:2])
```

```
print('L-7 :', a[1 , 3:0:-1])
```

```
print('L-8 :', a[1 , -1:-3:-1])
```

```
print('L-9 :', a[0:2 , 0:2])
```

2-D Array

| Index | 0 | 1 | 2 | 3 | 4  |
|-------|---|---|---|---|----|
| 0     | 1 | 2 | 3 | 4 | 5  |
| 1     | 6 | 7 | 8 | 9 | 10 |

### Array Elements using Slicing Index

L-1 : [7 8 9]

L-2 : [6 7 8 9]

L-3 : [ 7 8 9 10]

L-4 : [ 6 7 8 9 10]

L-5 : [7 8 9]

L-6 : [7 9]

L-7 : [9 8 7]

L-8 : [10 9]

L-9 : [ [1 2]

[6 7] ]



# Creating Matrix

1. **empty( )** - creates an uninitialized array of specified shape and **dtype**.

```
import numpy as np
x = np.empty([3,2], dtype = int)
print (x)
```

## Output

```
[[12846240 12859408]
 [12875144 12875480]
 [12875480 0]]
```

2. **ones( )**- New array of specified size and type, filled with ones. Default data type is float.

```
import numpy as np
x = np.ones([2,2])
print(x)
```

## Output

```
[[1. 1.]
 [1. 1.]]
```

3. **zeros( )**- New array of specified size, filled with zeros.

```
import numpy as np
x = np.zeros(5)
print(x)
x = np.zeros([3,3] , int)
print(x)
```

## Output

```
[0. 0. 0. 0. 0.]
```

```
[[0 0 0]
 [0 0 0]
 [0 0 0]]
```

# Axis of Matrix

## 1D Array

Axis=0 represent the columns

Axis=1 represent the rows

| Axis | 0 |   |   |
|------|---|---|---|
|      | 1 | 2 | 3 |

## 2D Array

Axis=0 represent the columns

Axis=1 represent the rows

| Axis | 0 |   |   |
|------|---|---|---|
| 1    | 1 | 2 | 3 |
|      | 4 | 5 | 6 |
|      | 7 | 8 | 9 |

# Aggregate Function

**Numpy** has fast built-in **aggregate and statistical** for working on arrays.

| <u>SNo</u> | Function        | Description                                   |
|------------|-----------------|-----------------------------------------------|
| 1          | np.sum(Arr)     | Find the sum of the array.                    |
| 2          | np.prod(Arr)    | Find the product of the values of array.      |
| 3          | np.mean(Arr)    | Find the mean of the array.                   |
| 4          | np.std(Arr)     | Find the standard deviation of the array.     |
| 5          | np.var(Arr)     | Find the variance of the data of array.       |
| 6          | np.min(Arr)     | Find the minimum from the array.              |
| 7          | np.max(Arr)     | Find the maximum from the array.              |
| 8          | np.argmin(Arr)  | Find the index of minimum value in the array. |
| 9          | np.argmax(Arr)  | Find the index of maximum value in the array. |
| 10         | np.median(Arr)  | Find the median of the values of the array.   |
| 11         | np.average(Arr) | Find the average of the values of the array.  |

# Aggregate Function

**1. sum( )** – calculate the sum of elements in array.

```
import numpy as np
```

```
A1=np.array([1,2,3,4,5])
```

```
A2=np.array([[1,2,3], [4,5,6]])
```

```
A3=np.array([[1,2,3], [4,5,6] , [7,8,9]])
```

## #1D Array – A1

```
print(np.sum(A1)) ---- 15
```

## #2D Array – A2

```
print(np.sum(A2)) ---- 21
```

```
print(np.sum(A2,0)) ---- [5 7 9]
```

```
print(np.sum(A2,1)) ---- [6 15]
```

## #2D Array – A3

```
print(np.sum(A3)) ---- 45
```

```
print(np.sum(A3,0)) ---- [12 15 18]
```

```
print(np.sum(A3,1)) ---- [6 15 24]
```

Array – A1

| Axis =0 |   |   |   |   |
|---------|---|---|---|---|
| 1       | 2 | 3 | 4 | 5 |

Array – A2

| Axis | 0 |   |   |
|------|---|---|---|
| 1    | 1 | 2 | 3 |
|      | 4 | 5 | 6 |

Array – A3

| Axis | 0 |   |   |
|------|---|---|---|
| 1    | 1 | 2 | 3 |
|      | 4 | 5 | 6 |
|      | 7 | 8 | 9 |

# Aggregate Function

**2. prod( )** – calculate the product of elements in array.

```
import numpy as np
```

```
A1=np.array([1,2,3,4,5])
```

```
A2=np.array([[1,2,3], [4,5,6]])
```

```
A3=np.array([[1,2,3], [4,5,6] , [7,8,9]])
```

## #1D Array – A1

```
print(np.prod(A1)) ---- 120
```

## #2D Array – A2

```
print(np.prod(A2)) ---- 720
```

```
print(np.prod(A2,0)) ---- [4 10 18]
```

```
print(np.prod(A2,1)) ---- [6 120]
```

## #2D Array – A3

```
print(np.prod(A3)) ---- 362880
```

```
print(np.prod(A3,0)) ---- [28 80 162]
```

```
print(np.prod(A3,1)) ---- [6 120 504]
```

Array – A1

| Axis =0 |   |   |   |   |
|---------|---|---|---|---|
| 1       | 2 | 3 | 4 | 5 |

Array – A2

| Axis | 0 |   |   |
|------|---|---|---|
| 1    | 1 | 2 | 3 |
|      | 4 | 5 | 6 |

Array – A3

| Axis | 0 |   |   |
|------|---|---|---|
| 1    | 1 | 2 | 3 |
|      | 4 | 5 | 6 |
|      | 7 | 8 | 9 |

# Aggregate Function

**3. max( )** – calculate the product of elements in array.

```
import numpy as np
```

```
A1=np.array([1,2,3,4,5])
```

```
A2=np.array([[1,2,3], [4,5,6]])
```

```
A3=np.array([[1,2,3], [4,5,6] , [7,8,9]])
```

## #1D Array – A1

```
print(np.max(A1)) ---- 5
```

## #2D Array – A2

```
print(np.max(A2)) ---- 6
```

```
print(np.max(A2, 0)) ---- [4 5 6]
```

```
print(np.max(A2, 1)) ---- [3 6]
```

## #2D Array – A3

```
print(np.max(A3)) ---- 9
```

```
print(np.max(A3, 0)) ---- [7 8 9]
```

```
print(np.max(A3, 1)) ---- [3 6 9]
```

Array – A1

| Axis =0 |   |   |   |   |
|---------|---|---|---|---|
| 1       | 2 | 3 | 4 | 5 |

Array – A2

| Axis | 0 |   |   |
|------|---|---|---|
| 1    | 1 | 2 | 3 |
|      | 4 | 5 | 6 |

Array – A3

| Axis | 0 |   |   |
|------|---|---|---|
| 1    | 1 | 2 | 3 |
|      | 4 | 5 | 6 |
|      | 7 | 8 | 9 |

# Aggregate Function

**4. argmax( )** – find the index of max elements in array.

```
import numpy as np
```

```
A1=np.array([1,2,3,4,5])
```

```
A2=np.array([[1,2,3], [4,5,6]])
```

```
A3=np.array([[1,2,3], [4,5,6] , [7,8,9]])
```

**#1D Array – A1**

```
print(np.argmax(A1)) ---- 4
```

**#2D Array – A2**

```
print(np.argmax(A2)) ---- 5
```

```
print(np.argmax(A2, 0)) ---- [1 1 1]
```

```
print(np.argmax(A2, 1)) ---- [2 2]
```

**#2D Array – A3**

```
print(np.argmax(A3)) ---- 8
```

```
print(np.argmax(A3, 0)) ---- [2 2 2]
```

```
print(np.argmax(A3, 1)) ---- [2 2 2]
```

Array – A1

| Axis =0 |   |   |   |   |
|---------|---|---|---|---|
| 1       | 2 | 3 | 4 | 5 |

Array – A2

| Axis | 0 |   |   |
|------|---|---|---|
| 1    | 1 | 2 | 3 |
|      | 4 | 5 | 6 |

Array – A3

| Axis | 0 |   |   |
|------|---|---|---|
| 1    | 1 | 2 | 3 |
|      | 4 | 5 | 6 |
|      | 7 | 8 | 9 |

# Aggregate Function

**5. mean( )** – find the mean of elements in array.

```
import numpy as np
```

```
A1=np.array([1,2,3,4,5])
```

```
A2=np.array([[1,2,3], [4,5,6]])
```

```
A3=np.array([[1,2,3], [4,5,6] , [7,8,9]])
```

## #1D Array – A1

```
print(np.mean(A1)) ---- 3.0
```

## #2D Array – A2

```
print(np.mean(A2)) ---- 3.5
```

```
print(np.mean(A2, 0)) ---- [2.5 3.5 4.5]
```

```
print(np.mean(A2, 1)) ---- [2. 5.]
```

## #2D Array – A3

```
print(np.mean(A3)) ---- 5.0
```

```
print(np.mean(A3, 0)) ---- [4. 5. 6.]
```

```
print(np.mean(A3, 1)) ---- [2. 5. 8.]
```

Array – A1

| Axis =0 |   |   |   |   |
|---------|---|---|---|---|
| 1       | 2 | 3 | 4 | 5 |

Array – A2

| Axis | 0 |   |   |
|------|---|---|---|
| 1    | 1 | 2 | 3 |
|      | 4 | 5 | 6 |

Array – A3

| Axis | 0 |   |   |
|------|---|---|---|
| 1    | 1 | 2 | 3 |
|      | 4 | 5 | 6 |
|      | 7 | 8 | 9 |



# Aggregate Function

**6. median( )** – find the median of elements in array.

```
import numpy as np
```

```
A1=np.array([1,2,3,4,5])
```

```
A2=np.array([[1,2,3], [4,5,6]])
```

```
A3=np.array([[1,2,3], [4,5,6] , [7,8,9]])
```

## #1D Array – A1

```
print(np.median(A1)) ---- 3.0
```

## #2D Array – A2

```
print(np.median(A2)) ---- 3.5
```

```
print(np.median(A2, 0)) ---- [2.5 3.5 4.5]
```

```
print(np.median(A2, 1)) ---- [2. 5.]
```

## #2D Array – A3

```
print(np.median(A3)) ---- 5.0
```

```
print(np.median(A3, 0)) ---- [4. 5. 6.]
```

```
print(np.median(A3, 1)) ---- [2. 5. 8.]
```

Array – A1

| Axis =0 |   |   |   |   |
|---------|---|---|---|---|
| 1       | 2 | 3 | 4 | 5 |

Array – A2

| Axis | 0 |   |   |
|------|---|---|---|
| 1    | 1 | 2 | 3 |
|      | 4 | 5 | 6 |

Array – A3

| Axis | 0 |   |   |
|------|---|---|---|
| 1    | 1 | 2 | 3 |
|      | 4 | 5 | 6 |
|      | 7 | 8 | 9 |

# Standard Deviation

- Values are: 600, 470, 170, 430, 300
- Find out the Mean, the Variance, and the Standard Deviation.
- Your first step is to find the Mean:
- Answer:
- $\text{Mean} = 600 + 470 + 170 + 430 + 300 / 5$
- $= 1970 / 5 = 394$
- so the mean (average) height is 394 mm.
- To calculate the Variance, take each difference, square it, and then average the result:

## Variance

$$\begin{aligned}\sigma^2 &= \frac{206^2 + 76^2 + (-224)^2 + 36^2 + (-94)^2}{5} \\ &= \frac{42436 + 5776 + 50176 + 1296 + 8836}{5} \\ &= \frac{108520}{5} \\ &= 21704\end{aligned}$$

So the Variance is **21,704**

# Standard Deviation

And the Standard Deviation is just the square root of Variance, so:

## Standard Deviation

$$\begin{aligned}\sigma &= \sqrt{21704} \\ &= 147.32... \\ &= \mathbf{147} \text{ (to the nearest mm)}\end{aligned}$$

# Aggregate Function

**7. std( )** – find the standard deviation of array.

```
import numpy as np
```

```
A1=np.array([1,2,3,4,5])
```

```
A2=np.array([[1,2,3], [4,5,6]])
```

```
A3=np.array([[1,2,3], [4,5,6] , [7,8,9]])
```

**#1D Array – A1**

```
print(np.std(A1)) ---- 1.414
```

**#2D Array – A2**

```
print(np.std(A2)) ---- 1.707
```

```
print(np.std(A2, 0)) ---- [1.5 1.5 1.5]
```

```
print(np.std(A2, 1)) ---- [0.816 0.816]
```

**#2D Array – A3**

```
print(np.std(A3)) ---- 2.5819
```

```
print(np.std(A3, 0)) ---- [2.449 2.449 2.449]
```

```
print(np.std(A3, 1)) ---- [0.816 0.816 0.816]
```

**Array – A1**

| Axis =0 |   |   |   |   |
|---------|---|---|---|---|
| 1       | 2 | 3 | 4 | 5 |

**Array – A2**

| Axis | 0 |   |   |
|------|---|---|---|
| 1    | 1 | 2 | 3 |
|      | 4 | 5 | 6 |

**Array – A3**

| Axis | 0 |   |   |
|------|---|---|---|
| 1    | 1 | 2 | 3 |
|      | 4 | 5 | 6 |
|      | 7 | 8 | 9 |

# Aggregate Function

**8. variance( )** – find the variance of elements in array. It is equal to square of standard deviation.

```
import numpy as np
```

```
A1=np.array([1,2,3,4,5])
```

```
A2=np.array([[1,2,3], [4,5,6]])
```

```
A3=np.array([[1,2,3], [4,5,6] , [7,8,9]])
```

**#1D Array – A1**

```
print(np.var(A1)) ---- 2.0
```

**#2D Array – A2**

```
print(np.var(A2)) ---- 2.916
```

```
print(np.var(A2, 0)) ---- [2.25 2.25 2.25]
```

```
print(np.var(A2, 1)) ---- [0.666 0.666]
```

**#2D Array – A3**

```
print(np.var(A3)) ---- 6.666
```

```
print(np.var(A3, 0)) ---- [6. 6. 6.]
```

```
print(np.var(A3, 1)) ---- [0.666 0.666 0.666]
```

**Array – A1**

| Axis =0 |   |   |   |   |
|---------|---|---|---|---|
| 1       | 2 | 3 | 4 | 5 |

**Array – A2**

| Axis | 0 |   |   |
|------|---|---|---|
| 1    | 1 | 2 | 3 |
|      | 4 | 5 | 6 |

**Array – A3**

| Axis | 0 |   |   |
|------|---|---|---|
| 1    | 1 | 2 | 3 |
|      | 4 | 5 | 6 |
|      | 7 | 8 | 9 |

# NumPy – arange( )

- The advantage of `numpy.arange( )` over the normal in-built `range( )` function is that it allows us to generate sequences of numbers that are not integers.
- `arange([start,] stop[, step,][, dtype])` : Returns an array with evenly spaced elements as per the interval. The interval mentioned is half opened i.e. [Start, Stop)
- Parameters :
  - `start` : [optional] start of interval range. By default `start = 0`
  - `stop` : end of interval range
  - `step` : [optional] step size of interval. By default `step size = 1`,

## Example

```
import numpy as np
print("A1\n", np.arange(4).reshape(2, 2), "\n")
print("A2\n", np.arange(4, 10), "\n")
print("A3\n", np.arange(4, 20, 3), "\n")

Printing all numbers from 1 to 2 in steps of 0.1
print(("A4\n", np.arange(1, 2, 0.1))
```

## Output

```
A1 [[0 1]
 [2 3]]
A2 [4 5 6 7 8 9]
A3 [4 7 10 13 16 19]
A4 [1. 1.1 1.2 1.3 1.4 1.5 1.6 1.7 1.8 1.9]
```

## NumPy – Random Numbers

### Example

```
import numpy as np
x=np.random.rand() # random number from 0-1
print(x) # 0.60494724

x=np.random.randint(100) # A number from 1-100
print(x) #82

x=np.random.choice([1,2,3,4,5])
print(x) # 3
```

# Arrays of Random Numbers

```
x=np.random.randint(100, size=(2,3))
print(x)
```

# array of 5 random number from 0-1

```
x=np.random.rand(2,3)
print(x)
```

```
x=np.random.choice([1,2,3,4,5], size=(2,3))
print(x)
```

### Output

```
[[54 52 59]
```

```
 [18 38 37]]
```

```
[[0.92394553 0.34195831 0.06910477]
```

```
 [0.87275751 0.07780082 0.21137941]]
```

```
[[4 1 5]
```

```
 [3 5 3]]
```

# References

- [www.w3schools.com](http://www.w3schools.com)
- [www.tutorialpoint.com](http://www.tutorialpoint.com)
- Python Programming – Pearson – Taneja Kumar



Thank You  
Any Query

# Data Science using Python Programming

## Unit-13: < NumPy Module – Copy , Filter, Searching , Sorting>

# Contents to be covered

1. NumPy Data Type
2. Array Copy & View
3. Shape & Reshape
4. Filtering Array
5. Searching & Sorting Array

# NumPy Data Types

Python provides the following data types :-

1. **strings** - text data enclosed in quote marks. eg. "ABCD"
2. **integer** - integer numbers. eg. 1,2,3,-1, -2, -3
3. **float** - real numbers. eg. 12.2, 12.42
4. **boolean** - True or False.
5. **complex** - a number in complex form. eg.  $2.0 + 2.0j$ ,  $2.5 + 1.5j$

**NumPy provides more data types, and referenced by one character**

i – integer where ( i1 = int8 , i2=int16, i3=int32 , i4=int64)

b - boolean

u - unsigned integer

f - float

c - complex float

m - timedelta

M - datetime

O - object

S - string

U - unicode string

V - fixed chunk of memory for other type ( void )

# NumPy Data Types

## dtype

1. NumPy array has property **dtype**.
2. **dtype** can be used to define the data type of the array , while creating.
3. **dtype** also returns the data type of the array elements.

## Example

```
a = np.array([1, 2, 3,4,5])
a = np.array([1, 2, 3,4,5], dtype='i2')
a = np.array([1, 2, 3,4,5], dtype='S')
a = np.array([1, 2, 3,4,5], dtype='f')
```

```
print('Data Type of Array: ', a.dtype)
```

## astype( )

1. NumPy array has inbuilt method **astype( )**.
2. **astype( )** allows to convert the data type of an existing array, to the specified data type.

## Example

```
a = np.array([1.1, 2.5, 0.6, 4.7, 5.3])
a=a.astype("i")
a=a.astype(bool)
```

```
[1.1 2.5 0.6 4.7 5.3]
```

```
[1 2 0 4 5]
```

```
[True True False True True]
```

# NumPy Data Types

## # Example for dtype( )

```
import numpy as np
a = np.array([1, 2, 3,4,5])
print('Values of Array')
print(a)
print('Data Type of Array: ', a.dtype)
a = np.array([1, 2, 3,4,5], dtype='i2')
print('Values of Array')
print(a)
print('Data Type of Array: ', a.dtype)
a = np.array([1, 2, 3,4,5], dtype='S')
print('Values of Array')
print(a)
print('Data Type of Array: ', a.dtype)
a = np.array([1, 2, 3,4,5], dtype='f')
print('Values of Array')
print(a)
print('Data Type of Array: ', a.dtype)
```

## Output

Values of Array

[1 2 3 4 5]

Data Type of Array: int32

Values of Array

[1 2 3 4 5]

Data Type of Array: int16

Values of Array

[b'1' b'2' b'3' b'4' b'5']

Data Type of Array: |S1

Values of Array

[1. 2. 3. 4. 5.]

Data Type of Array: float32

# NumPy Data Types

## # Example for astype( )

```
import numpy as np
a = np.array([1.1, 2.5, 0.6, 4.7, 5.3])
print('Values of Array')
print(a)
print('Data Type of Array: ', a.dtype)
```

```
a=a.astype("i")
print('Values of Array')
print(a)
print('Data Type of Array: ', a.dtype)
```

```
a=a.astype(bool)
print('Values of Array')
print(a)
print('Data Type of Array: ', a.dtype)
```

## Output

Values of Array  
[1.1 2.5 0.6 4.7 5.3]  
Data Type of Array: float64

Values of Array  
[1 2 0 4 5]  
Data Type of Array: int32

Values of Array  
[ True True False True True]  
Data Type of Array: bool

# NumPy- Shape & Reshape

## Shape and Reshape

- NumPy array has property “**shape**” which returns the tuple with count of elements in each dimension.
- For example – ( 2,4) means array has 2 rows , 4 column (values in each rows).
- NumPy array has inbuilt method “**reshape( )**” used to change the dimension of array.

- **Example**

- `a = np.array([1, 2, 3, 4,5, 6, 7, 8])`      --- shape - (8, )    Row -8 , Column-0
- `a= a.reshape(2,4)`      --- shape - (2, 4 ) Row -2 , Column-4
- `a = np.array([ [1, 2, 3,4], [5, 6, 7, 8] ])`      --- shape - (2, 4 ) Row -2 , Column-4
- `a= a.reshape(8)`      --- shape - (8, )    Row -8 , Column-0



# NumPy – Shape & Reshape

## # Example

```
import numpy as np
a = np.array([1, 2, 3, 4, 5, 6, 7, 8])
print('Array Before : ')
print(a)
print('shape of array :', a.shape)

a=a.reshape(2,4)
print('Array After : ')
print(a)
print('shape of array :', a.shape)

a = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])
print('Array Before : ')
print(a)
a=a.reshape(8)
print('Array After : ')
print(a)
print('shape of array :', a.shape)
```

## Output

### Array Before :

```
[1 2 3 4 5 6 7 8]
```

shape of array : (8,)

### Array After :

```
[[1 2 3 4]
```

```
[5 6 7 8]]
```

shape of array : (2, 4)

### Array Before :

```
[[1 2 3 4]
```

```
[5 6 7 8]]
```

### Array After :

```
[1 2 3 4 5 6 7 8]
```

shape of array : ( 8,)

# NumPy- Array Iterating

- NumPy allows the iterating of array values using the **for..loop**.
- For **1-D Array**

```
for row in n :
 print(row)
```
- For **2-D Array**

```
for row in n :
 for col in row :
 print(col)
```
- For **3-D Array**

```
for z in n:
 for row in z :
 for col in row :
 print(col)
```
- **nditer( )** method allows accessing each value in the array of 1-D, 2-D or 3-D.  

```
for x in np.nditer(n_arr) :
 print(x)
```
- **ndenumerate( )** method returns the pair (index , value) for each element of the array.  

```
for id , x in np.ndenumerate(n_arr) :
 print(id , x)
```

# NumPy – Array Iterating

## # Example

```
import numpy as np
a = np.array([[1, 2, 3, 4],[5, 6, 7, 8]])
print('Simple Loop - Print Rows')
for row in a:
 print(row)

print('Simple Loop - Print each value')
for row in a:
 for col in row:
 print(col)

print('nditer() - Print each value')
for val in np.nditer(a):
 print(val)

print('ndenumerate() - Print (id ,value)')
for id,val in np.ndenumerate(a):
 print(id,val)
```

## Output

Simple Loop - Print Rows

[1 2 3 4]

[5 6 7 8]

Simple Loop - Print each value

1 2 3 4 5 6 7 8

nditer( ) - Print each value

1 2 3 4 5 6 7 8

ndenumerate( ) - Print (id ,value)

(0, 0) 1

(0, 1) 2

(0, 2) 3

(0, 3) 4

(1, 0) 5

(1, 1) 6

(1, 2) 7

(1, 3) 8

# NumPy- Filtering ,Searching & Sorting

- `numpy.where( )` is used to search the value in the array and it return the tuple of index.
- **Example**
  - `np.where( arr==4)`
  - `np.where( arr%2 ==0)`
- Filtering is the process of extracting some value from the array based on certain conditions.
- **Example**
  - `arr=[10,11,12,13,14]`
  - `filter= (arr%2 ==0)`
  - `arr=arr[filter]`
- `Numpy.sort(array)` is used to arrange the value of the array passed.
- **Example**
  - `Np.sort(arr)`

# NumPy- Searching & Sorting

## # Example

```
import numpy as np
a = np.array([1,4,3,2,5,8,7,6])
print('Values of Array - A ')
print(a)

print('Sorted Array: ')
print(np.sort(a))

print('Search the Even Numbers')
x=np.where(a%2==0)
print(x)
```

## Output

```
Values of Array - A
[1 4 3 2 5 8 7 6]
Sorted Array:
[1 2 3 4 5 6 7 8]
Search the Even Numbers
(array([1, 3, 5, 7], dtype=int32),)
```

## #Example for Searching

```
t = np.array([[10,20,30,'red'],
 [21,31,41,'green'],
 [50,60,70,'brown'],
 [89,97,11,'green']])

row= np.where(t == 'green')
print(row)

rs= t[row]
print(rs)
```

## Output

```
(array([1, 3], dtype=int32), array([3, 3], dtype=int32))
['green' 'green']
```

# NumPy- Searching in 2D Array

## # Example

```
import numpy as np
a = np.array([[1,5,2,5], [5,7,6,5]])
```

```
print('Searching the Value')
filter=np.where(a==5)
print(filter)
x=a[filter]
print(x)
```

```
t = np.array([[10,20,30,'red'],
[21,'green',41,'green'],
[50,60,70,'brown'],
[89,97,11,'green']])
```

```
filter=np.where(t == 'green')
print(filter)
x=t[filter]
print(x)
```

```
[[1, 5, 2, 5],
 [5, 7, 6, 5]]
```

## Output

### Searching ing the Value

```
(array([0, 0, 1, 1], dtype=int32), array([1, 3, 0, 3],
dtype=int32))
```

```
[5 5 5 5]
```

```
(array([1, 1, 3], dtype=int32), array([1, 3, 3],
dtype=int32))
```

```
['green' 'green' 'green']
```

### Values found at this index

```
0 0 1 1
```

```
1 1 3
```

```
1 3 0 3
```

```
1 3 3
```

```
(0,1) (0,3) (1,0) (1,3)
```

```
(1,1) (1,3) (3,3)
```

# NumPy- Filtering

## # Example

```
import numpy as np
a = np.array([[1,5,2,5], [5,7,6,5]])
print('Filtering the Value')
filter=(a%2==0)
print(filter)
x=a[filter]
print(x)
```

```
t = np.array([[10,20,30,'red'],
[21,31,41,'green'],
[50,60,70,'brown'],
[89,97,11,'green']])
```

```
filter=(t == 'green')
print(filter)
x=t[filter]
print(x)
```

```
[[1, 5, 2, 5],
 [5, 7, 6, 5]]
```

## Output

### Filtering the Value

```
[[False False True False]
 [False False True False]]
[2 6]
```

```
[[False False False False]
 [False False False True]
 [False False False False]
 [False False False True]]
```

```
['green' 'green']
```

# NumPy – Joining and Splitting

## Joining Methods

- `.concatenate ( )` used to join to 2 arrays at the provided axis level. Default axis=0.
- `.hstack( )` used to horizontally join 2 arrays
- `.vstack( )` used to vertically join 2 arrays.

## Splitting Methods

- `numpy.hsplit ( )` split the array horizontally.
- `numpy.vsplit( )` split the array vertically.
- `numpy.array_split( array , count, axis=0 )` split the array into specified number.



# NumPy- Array Joining

## # Example

```
import numpy as np
a=np.array([[1,2,3] , [4,5,6]])
b=np.array([[11,12,13] , [14,15,16]])

c=np.concatenate((a , b), axis=0)
print('Array join at Axis=0')
print(c)

c=np.concatenate((a , b), axis=1)
print('Array join at Axis=1')
print(c)

c=np.vstack((a , b))
print('Array join vertically')
print(c)

c=np.hstack((a , b))
print('Array join Horizontally')
print(c)
```

## Output

Array join at Axis=0

```
[[1 2 3]
 [4 5 6]
 [11 12 13]
 [14 15 16]]
```

Array join at Axis=1

```
[[1 2 3 11 12 13]
 [4 5 6 14 15 16]]
```

Array join vertically

```
[[1 2 3]
 [4 5 6]
 [11 12 13]
 [14 15 16]]
```

Array join Horizontally

```
[[1 2 3 11 12 13]
 [4 5 6 14 15 16]]
```

# NumPy- Array Splitting

```
[[1, 2, 3],
 [4, 5, 6],
 [11, 12, 13],
 [14, 15, 16]]
```

```
import numpy as np
a=np.array([[1,2,3], [4,5,6], [11,12,13], [14,15,16]])
c=np.array_split(a, 2)
print('Array Split into 2 part')
print(c)
```

## Output

**Array Split into 2 part**  
array([[1, 2, 3],  
 [4, 5, 6]]),  
array([[11, 12, 13],  
 [14, 15, 16]])

```
c=np.array_split(a, 3, axis=0)
print('Array Split at Axis=0')
print(c)
```

## Array Split at Axis=0

array([[1, 2, 3], [4, 5, 6]]),  
array([[11, 12, 13]]), array([[14, 15, 16]])

```
c=np.array_split(a, 3, axis=1)
print('Array Split at Axis=1')
print(c)
```

## Array Split at Axis=1

array([[1], [4], [11], [14]]), array([[2], [5], [12], [15]]),  
array([[3], [6], [13], [16]])

```
c=np.hsplit(a , 3)
print('Array Split Horizontally')
print(c)
```

## Array Split Horizontally

array([[1], [4], [11], [14]]), array([[2], [5], [12], [15]]),  
array([[3], [6], [13], [16]])

```
c=np.vsplit(a,2)
print('Array Split vertically')
print(c)
```

## Array Split vertically

array([[1, 2, 3], [4, 5, 6]]),  
array([[11, 12, 13], [14, 15, 16]])

# References

- [www.w3schools.com](http://www.w3schools.com)
- [www.tutorialpoint.com](http://www.tutorialpoint.com)
- Python Programming – Pearson – Taneja Kumar

Thank You  
Any Query

# Data Science using Python Programming

## Unit-14: < Pandas Module >

# Contents to be covered

1. Pandas Installation
2. Using Pandas
3. Series – Creating and Accessing
4. Dataframe – Creating and Accessing

# Pandas Module

- Pandas is the module, which enhances the capabilities of Numpy, Scipy and Matplotlib.
- Pandas is not an alternative to Numpy, SciPy and Matplotlib.
- It is built on **top of Numpy**. This means that Numpy is required by pandas. Scipy and Matplotlib on the other hand are not required by pandas but they are extremely useful.
- Pandas is used for **data manipulation and analysis**.
- It provides special data structures and operations for the manipulation of numerical tables and time series.

# Installation of Pandas

- Pandas is not available as bundled package in Python. We can install it using PIP.
- Steps to install the pandas in Python.
  - a) Go to command prompt -
    - a) Type cmd on RUN or click the command prompt
  - b) Change the folder to location where python installed.
    - a) `cd c:\Python38-32`
  - c) Move to the folder **Scripts** in **C:\python38-32**.
    - a) `c:\Python38-32> cd scripts`
    - b) `c:\Python38-32\Scripts>`
  - d) Type the **PIP install Pandas** and press the enter key. Keep the internet connection active during this process. It will download the package and install it automatically.
    - a) `c:\Python38-32\Scripts> pip install pandas`
  - e) Or **Python -m pip install pandas**



# Installation of Pandas

```
C:\WINDOWS\system32\cmd.exe

C:\>pip install pandas
Collecting pandas
 Downloading https://files.pythonhosted.org/packages/57/70/0292ee0aad71fd662f196502da0d608b7fa82fa32d19e412c10b63b4b415/pandas-1.0.4-cp38-cp38-win32.whl (7.6MB)
 |████████████████████| 7.6MB 3.3MB/s
Requirement already satisfied: python-dateutil>=2.6.1 in c:\python38-32\lib\site-packages (from pandas) (2.8.1)
Collecting pytz>=2017.2 (from pandas)
 Downloading https://files.pythonhosted.org/packages/4f/a4/879454d49688e2fad93e59d7d4efda580b783c745fd2ec2a3adf87b0808d/pytz-2020.1-py2.py3-none-any.whl (510kB)
 |████████████████████| 512kB ...
Requirement already satisfied: numpy>=1.13.3 in c:\python38-32\lib\site-packages (from pandas) (1.18.4)
Requirement already satisfied: six>=1.5 in c:\python38-32\lib\site-packages (from python-dateutil>=2.6.1->pandas) (1.15.0)
Installing collected packages: pytz, pandas
Successfully installed pandas-1.0.4 pytz-2020.1
WARNING: You are using pip version 19.2.3, however version 20.1.1 is available.
You should consider upgrading via the 'python -m pip install --upgrade pip' command.

C:\>
```

# Data Structures

Pandas deals with the following two data structures –

- Series
- DataFrame

These data structures are built on top of Numpy array, which means they are fast.

| Data Structure | Dimensions | Description                                                    |
|----------------|------------|----------------------------------------------------------------|
| Series         | 1          | 1-D labelled homogeneous array.<br>size immutable.             |
| Data Frames    | 2          | 2-D labelled heterogeneous tabular structure .<br>size-mutable |

# Using of Pandas

## Example-1

```
import pandas as pd
S = pd.Series([11, 28, 72, 3, 5, 8])
print(S)
print(S.index)
print(S.values)
```

### Output

```
0 11
1 28
2 72
3 3
4 5
5 8
dtype: int64
RangeIndex(start=0, stop=6, step=1)
[11 28 72 3 5 8]
```

## Example-2

```
import pandas as pd
fruits = ['apple', 'orange', 'mango', 'pear']
quantities = [20, 30, 52, 15]
S = pd.Series(quantities, index=fruits)
print(S)
```

### Output

```
apple 20
orange 30
mango 52
pear 15
dtype: int64
```

## Example-3

```
import pandas as pd
data = {
 'Fruits': ['apple', 'orange', 'mango', 'pear'],
 'Quantities': [20, 30, 52, 15, 90] }
df = pd.DataFrame(data)
print(df)
```

### Output

|   | Fruits | Quantities |
|---|--------|------------|
| 0 | apple  | 20         |
| 1 | orange | 30         |
| 2 | mango  | 52         |
| 3 | pear   | 15         |

# Example

- `import pandas as pd`
- `S = pd.Series([11, 28, 72, 3, 5, 8])`
- `print(S)`
- `print(type(S)) #<class  
'pandas.core.series.Series'`
- `print(S.shape)#(6,)`
- `print(S.index)#RangeIndex(start=0,  
stop=6, step=1)`
- `print(S.values)#[11, 28, 72, 3, 5, 8]`
- `print(type(S.values))#<class  
'numpy.ndarray'>`
- `print(type(S.index))#<class  
'pandas.core.indexes.range.RangeIndex'>`

# Python Pandas - Series

Series is a one-dimensional labelled array capable of holding data of any type (integer, string, float, python objects, etc.). The axis labels are collectively called index.

A pandas Series can be created using the following constructor –

**pandas.Series( data, index, dtype, copy)**

The parameters of the constructor are as follows –

1. **Data:** data takes various forms like ndarray, list, constants
2. **Index:** Index values must be unique and hashable, same length as data. Default np.arange(n) if no index is passed.
3. **Dtype:** dtype is for data type. If None, data type will be inferred
4. **Copy:** Copy data. Default False

# Create an Empty Series

A basic series, which can be created is an Empty Series.

```
#import the pandas library and aliasing as pd
import pandas as pd
s = pd.Series()
print(s)
```

Its output is as follows –  
Series( [ ], dtype: float64 )

# Create a Series from ndarray

1. If data is an **ndarray**, then index passed must be of the same length.
2. If no index is passed, then by default index will be **range(n)** where **n** is array length, i.e., **[0,1,2,3.... range(len(array))-1]**.

## Example

**#import the pandas library and aliasing as pd**

```
import pandas as pd
```

```
import numpy as np
```

```
data = np.array(['a', 'b','c','d'])
```

```
s = pd.Series(data)
```

```
print(s)
```

```
0 a
1 b
2 c
3 d
dtype: object
```

# Create a Series from dict

1. A **dict** can be passed as input and if no index is specified, then the dictionary keys are taken in a sorted order to construct index.
2. If **index** is passed, the values in data corresponding to the labels in the index will be pulled out.

## Example

```
#import the pandas library and aliasing as pd
import pandas as pd
data = { 'a' : 0., 'b' : 1., 'c' : 2.}
s = pd.Series(data)
print(s)
```

## Output

```
a 0.0
b 1.0
c 2.0
dtype: float64
```



# Example

```
import pandas as pd
data = { 'a' : 0, 'b' : 1, 'c' : 2}
s = pd.Series(data)
print(s)
s1 = pd.Series(data,index=['a','b','c'])
print(s1)
s1 = pd.Series(data,index=[0,1,2])# pulled out
print(s1)
```

Output:

```
a 0
b 1
c 2
dtype: int64
a 0
b 1
c 2
dtype: int64
0 NaN
1 NaN
2 NaN
dtype: float64
```

# Create a Series from Scalar

1. If data is a scalar value, an index must be provided.
2. The value will be repeated to match the length of **index**.

**#import the pandas library and aliasing as pd**

```
import pandas as pd
```

```
s = pd.Series(5, index=[0, 1, 2, 3])
```

```
print(s)
```

## Output

```
0 5
```

```
1 5
```

```
2 5
```

```
3 5
```

```
dtype: int64
```

# Accessing Data from Series with Position

1. Data in the series can be accessed similar to that in an **ndarray**.
2. Retrieve the element by providing index. The Index starts from zero for the array, which means the first element is stored at zeroth position and so on.

## Example

```
import pandas as pd
s = pd.Series([1,2,3,4,5], index = ['a','b','c','d','e'])
#retrieve the first element
print(s)
print('By numeric index :', s[0])
print('By named index :', s['a'])
```

## Output

```
a 1
b 2
c 3
d 4
e 5
dtype: int64
```

```
By numeric index : 1
By named index : 1
```

# Retrieve Data Using Label (Index)

A Series is like a fixed-size **dict** in that you can get and set values by index label.

```
import pandas as pd
s = pd.Series([1,2,3,4,5],index = ['a', 'b', 'c', 'd', 'e'])
```

```
#retrieve a single element
print(s['a'])
```

```
a 20
b 2
c 3
d 4
e 5
dtype: int64
```

```
#update a single element
s['a']= 20
print(s)
```

```
#update a single element
s['a']= 'A'
print(s)
```

```
a A
b 2
c 3
d 4
e 5
dtype: object
```

# DataFrame

A Data frame is a two-dimensional data structure, i.e., data is aligned in a tabular fashion in rows and columns.

## Features of DataFrame

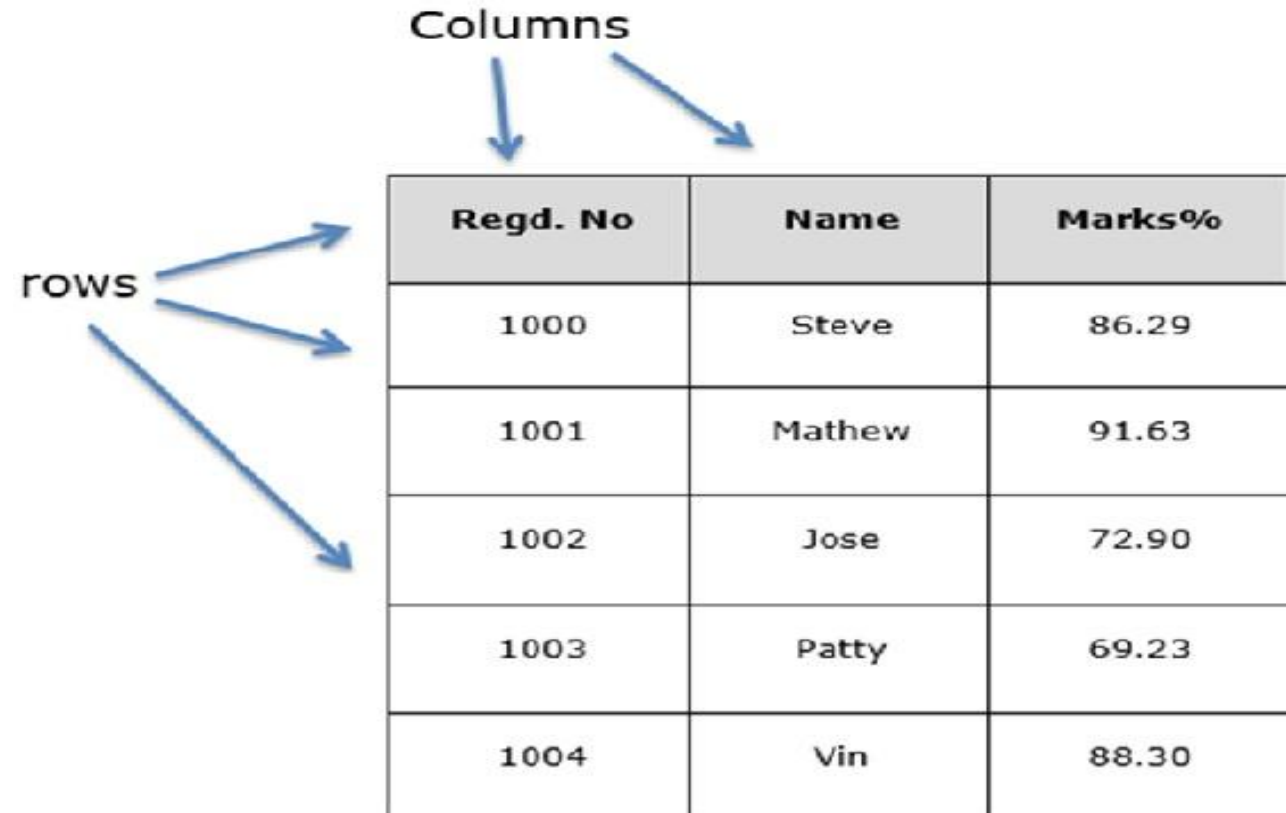
- Potentially columns are of different types
- Size – Mutable
- Labeled axes (rows and columns)
- Can Perform Arithmetic operations on rows and columns

# DataFrame

## Structure

We are creating a data frame with student's data.

You can think of it as an SQL table or a spreadsheet data representation.



| Columns  |        |        |
|----------|--------|--------|
| Regd. No | Name   | Marks% |
| 1000     | Steve  | 86.29  |
| 1001     | Mathew | 91.63  |
| 1002     | Jose   | 72.90  |
| 1003     | Patty  | 69.23  |
| 1004     | Vin    | 88.30  |

ROWS

# DataFrame

## pandas.DataFrame

A pandas DataFrame can be created using the following constructor –

`pandas.DataFrame( data, index, columns, dtype, copy)`

The parameters of the constructor are as follows –

1. **Data** - Data takes various forms like **ndarray**, series, map, lists, dict, constants and also another DataFrame.
2. **Index** - For the row labels, the Index to be used for the resulting frame is Optional.  
Default `np.arange(n)`, if no index is passed.
3. **Columns**- It is optional. Default is - `np.arange(n)`, if no index is passed.
4. **Dtype**- Data type of each column.
5. **Copy**- This command is used for copying of data, if the default is False.

# Create DataFrame

A pandas DataFrame can be created using various inputs like –

- Lists
- dict
- Series
- Numpy ndarrays
- Another DataFrame



# Create DataFrame

## Create an Empty DataFrame

A basic DataFrame, which can be created is an Empty Dataframe.

```
#import the pandas library and aliasing as pd
import pandas as pd
df = pd.DataFrame()
print(df)
```

## Output

Empty DataFrame

Columns: [ ]

Index: [ ]

# Create DataFrame

## Create a DataFrame from Lists

The DataFrame can be created using a single list or a list of lists.

```
import pandas as pd
data = [1,2,3,4,5]
df = pd.DataFrame(data)
print(df)
```

## Output

```
0 1
1 2
2 3
3 4
4 5
```

# Create DataFrame

## Create a DataFrame from Dict of ndarrays / Lists

1. All the **ndarrays** must be of same length. If index is passed, then the length of the index should equal to the length of the arrays.
2. If no index is passed, then by default, index will be range(n), where **n** is the array length.

### Example

```
import pandas as pd
data = {'Name':['Tom', 'Jack', 'Steve', 'Ricky'], 'Age':[28,34,29,42]}
df = pd.DataFrame(data)
print(df)
```

### Output

|   | Name  | Age |
|---|-------|-----|
| 0 | Tom   | 28  |
| 1 | Jack  | 34  |
| 2 | Steve | 29  |
| 3 | Ricky | 42  |

# Create DataFrame

## Create a DataFrame from List of Dicts

1. List of Dictionaries can be passed as input data to create a DataFrame.
2. The dictionary keys are by default taken as column names.
3. The following example shows how to create a DataFrame by passing a list of dictionaries.

```
import pandas as pd
data = [{'a': 1, 'b': 2}, {'a': 5, 'b': 10, 'c': 20}]
df = pd.DataFrame(data)
print(df)
```

## Output

|   | a | b  | c    |
|---|---|----|------|
| 0 | 1 | 2  | NaN  |
| 1 | 5 | 10 | 20.0 |

# Create DataFrame

## Create a DataFrame from Dict of Series

1. Dictionary of Series can be passed to form a DataFrame.
2. The resultant index is the union of all the series indexes passed.

### Example

```
import pandas as pd
d = { 'one' : pd.Series([1, 2, 3], index=['a', 'b', 'c']),
 'two' : pd.Series([1, 2, 3, 4], index=['a', 'b', 'c', 'd'])}
df = pd.DataFrame(d)
print(df)
```

### Output

|   | one | two |
|---|-----|-----|
| a | 1.0 | 1   |
| b | 2.0 | 2   |
| c | 3.0 | 3   |
| d | NaN | 4   |

# DataFrame

## Column Selection

Selecting a column from the DataFrame.

```
import pandas as pd
d = {'one' : pd.Series([1, 2, 3], index=['a', 'b', 'c']),
 'two' : pd.Series([1, 2, 3, 4], index=['a', 'b', 'c', 'd'])}
df = pd.DataFrame(d)
print(df ['one'])
```

## Output

```
a 1.0
b 2.0
c 3.0
d NaN
Name: one, dtype: float64
```

# DataFrame

## Column Addition

```
import pandas as pd
d = {'one' : pd.Series([1, 2, 3], index=['a', 'b', 'c']),
 'two' : pd.Series([1, 2, 3, 4], index=['a', 'b', 'c', 'd'])}
```

```
df = pd.DataFrame(d)
```

## Adding a new column to an existing DataFrame

```
print ("Adding a new column by passing as Series:")
df['three']=pd.Series([10,20,30], index=['a', 'b', 'c'])
print(df)
```

```
print ("Adding a new column using the existing columns in DataFrame:")
df['four']=df['one']+df['three']
```

```
print(df)
```

# DataFrame

**Column Deletion** - Columns can be deleted or popped.

## Example

```
import pandas as pd
d = {'one' : pd.Series([1, 2, 3], index=['a', 'b', 'c']),
 'two' : pd.Series([1, 2, 3, 4], index=['a', 'b', 'c', 'd']),
 'three' : pd.Series([10,20,30], index=['a','b','c'])}
df = pd.DataFrame(d)
print("Our dataframe is:")
print(df)

print ("Deleting the first column using DEL function:")
del df['one']
print(df)

print ("Deleting another column using POP function:")
df.pop('two')
print(df)
```

Our dataframe is:

|   | one | two | three |
|---|-----|-----|-------|
| a | 1.0 | 1   | 10.0  |
| b | 2.0 | 2   | 20.0  |
| c | 3.0 | 3   | 30.0  |
| d | NaN | 4   | NaN   |

Deleting the first column using DEL function:

|   | two | three |
|---|-----|-------|
| a | 1   | 10.0  |
| b | 2   | 20.0  |
| c | 3   | 30.0  |
| d | 4   | NaN   |

Deleting another column using POP function:

|   | three |
|---|-------|
| a | 10.0  |
| b | 20.0  |
| c | 30.0  |
| d | NaN   |



# DataFrame

## Row Selection

1. Pandas provide a unique method to retrieve rows from a Data frame.
2. **DataFrame.loc[ ]** method is used to retrieve rows from Pandas DataFrame.
3. Rows can also be selected by passing integer location to an **iloc[ ]** function.

```
import pandas as pd
d={'Name':["ram","shyam","uma","dinesh"],
 'Age':[28,34,29,42]}
df = pd.DataFrame(d, index= ['e1','e2','e3','e4'])
print(df)
retrieving row by loc method
first = df.loc['e1']
second = df.loc['e2']
first = df.iloc[2]
second = df.iloc[3]
```

Output:

```
Name Age
e1 ram 28
e2 shyam 34
e3 uma 29
e4 dinesh 42
Name ram
Age 28
Name shyam
Age 34
Name uma
Age 29
Name dinesh
Age 42
```

Name: e4, dtype: object

# DataFrame

**Row Addition-** In Order to add a Row in Pandas DataFrame, we can concat the old dataframe with new .

```
import pandas as pd
d = {'Name':['Tom', 'Jack', 'Steve', 'Ricky'], 'Age':[28,34,29,42]}
df = pd.DataFrame(d, index= ['e1','e2', 'e3','e4'])
print("Our dataframe is:")
print(df)
simply concatenate both dataframes
new_row = pd.DataFrame({'Name':'Ajay', 'Age':33},index =[0])
df = pd.concat([new_row, df])
print(df)
df = pd.concat([new_row, df]).reset_index(drop = True)
print(df)
df = pd.concat([new_row, df]).reset_index(drop = False)
print(df)
```

Output:

Name Age

e1 Tom 28

e2 Jack 34

e3 Steve 29

e4 Ricky 42

Name Age

0 Ajay 33

e1 Tom 28

e2 Jack 34

e3 Steve 29

e4 Ricky 42

Name Age

0 Ajay 33

1 Ajay 33

2 Tom 28

3 Jack 34

4 Steve 29

5 Ricky 42

index Name Age

0 0 Ajay 33

1 0 Ajay 33

2 1 Ajay 33

3 2 Tom 28

4 3 Jack 34

5 4 Steve 29

6 5 Ricky 42

# DataFrame

**Row Deletion:** In Order to delete a row in Pandas DataFrame, we can use the `drop()` method. Rows is deleted by dropping Rows by index label.

```
d = {'Name':['Tom', 'Jack', 'Steve', 'Ricky'],
 'Age':[28,34,29,42]}
```

```
df = pd.DataFrame(d, index= ['e1','e2', 'e3','e4'])
print("Our dataframe is:")
print(df)
```

```
dropping values
df.drop(['e1', 'e4'], inplace = True)
print(df)
```

Our dataframe is:

|    | Name  | Age |
|----|-------|-----|
| e1 | Tom   | 28  |
| e2 | Jack  | 34  |
| e3 | Steve | 29  |
| e4 | Ricky | 42  |

|    | Name  | Age |
|----|-------|-----|
| e2 | Jack  | 34  |
| e3 | Steve | 29  |

# Python Pandas - Sorting

Using the **sort\_index()** method, by passing the axis arguments and the order of sorting, DataFrame can be sorted. By default, sorting is done on row labels in ascending order.

```
import pandas as pd
```

```
RollNo=[101, 104, 103, 102]
```

```
d = { 'Name' : ['Ajay' , 'Vijay','Sanjay', 'Vikas'],
 'M1' : [52 , 61 ,57,65] }
```

```
df = pd.DataFrame(d, index=RollNo)
```

```
sorted_df=df.sort_index()
```

```
print(sorted_df)
```

## Unsorted

|     | Name   | M1 |
|-----|--------|----|
| 101 | Ajay   | 52 |
| 104 | Vijay  | 61 |
| 103 | Sanjay | 57 |
| 102 | Vikas  | 65 |

## Sorted

|     | Name   | M1 |
|-----|--------|----|
| 101 | Ajay   | 52 |
| 102 | Vikas  | 65 |
| 103 | Sanjay | 57 |
| 104 | Vijay  | 61 |

# Boolean Indexing in Pandas

1. In boolean indexing, we will select subsets of data based on the actual values of the data in the DataFrame and not on their row/column labels or integer locations.
2. In boolean indexing, we use a boolean vector to filter the data.
3. Boolean indexing is a type of indexing which uses actual values of the data in the DataFrame.
4. In boolean indexing, we can filter a data in four ways –
  1. Accessing a DataFrame with a boolean index
  2. Applying a boolean mask to a dataframe
  3. Masking data based on column value
  4. Masking data based on index value

# Accessing a DataFrame with a boolean index

In order to access a dataframe with a boolean index, we have to create a dataframe in which index of dataframe contains a boolean value that is “True” or “False”.

## Example

```
importing pandas as pd
```

```
import pandas as pd
```

```
dictionary of lists
```

```
dict = {'name':["aparna", "pankaj", "sudhir", "Geeku"],
 'degree': ["MBA", "BCA", "M.Tech", "MBA"],
 'score':[90, 40, 80, 98]}
```

```
df = pd.DataFrame(dict, index = [True, False, True, False])
```

```
print(df)
```

# Accessing a DataFrame with a boolean index

|       | name   | degree | score |
|-------|--------|--------|-------|
| True  | aparna | MBA    | 90    |
| False | pankaj | BCA    | 40    |
| True  | sudhir | M.Tech | 80    |
| False | Geeku  | MBA    | 98    |

1. Now we have created a dataframe with boolean index after that user can access a dataframe with the help of boolean index.
2. User can access a dataframe using functions that is **.loc[ ]**, **.iloc[ ]**

# Accessing a DataFrame with a boolean index

## Accessing a Dataframe with a boolean index using .loc[ ]

In order to access a dataframe with a boolean index using .loc[ ], we simply pass a boolean value (True or False) in a .loc[ ] function.

```
import pandas as pd
dict = {'name':["aparna", "pankaj", "sudhir", "Geeku"],
 'degree': ["MBA", "BCA", "M.Tech", "MBA"],
 'score':[90, 40, 80, 98]}

creating a dataframe with boolean index
df = pd.DataFrame(dict, index = [True, False, True, False])

accessing a dataframe using .loc[] function
print(df.loc[True])
```



# Accessing a DataFrame with a boolean index

|       | name   | degree | score |
|-------|--------|--------|-------|
| True  | aparna | MBA    | 90    |
| False | pankaj | BCA    | 40    |
| True  | sudhir | M.Tech | 80    |
| False | Geeku  | MBA    | 98    |

|      | name   | degree | score |
|------|--------|--------|-------|
| True | aparna | MBA    | 90    |
| True | sudhir | M.Tech | 80    |

# Accessing a DataFrame with a boolean index

## Accessing a Dataframe with a boolean index using `.iloc[ ]`

In order to access a dataframe using `.iloc[ ]`, we have to pass a boolean value (True or False) in a `iloc[ ]` function but `iloc[ ]` function accept only integer as argument so it will throw an error so we can only access a dataframe when we pass a integer in `iloc[ ]` function

```
import pandas as pd
dict = {'name':["aparna", "pankaj", "sudhir", "Geeku"],
 'degree': ["MBA", "BCA", "M.Tech", "MBA"],
 'score':[90, 40, 80, 98]}
df = pd.DataFrame(dict, index = [True, False, True, False])
print(df.iloc[1])
```

# Accessing a DataFrame with a boolean index

## Accessing a Dataframe with a boolean index using .iloc[]

```
import pandas as pd
dict = {'name':["aparna", "pankaj", "sudhir", "Geeku"],
 'degree': ["MBA", "BCA", "M.Tech", "MBA"],
 'score':[90, 40, 80, 98]}
df = pd.DataFrame(dict, index = [True, False, True, False])
print(df.iloc[1])
```

|       | name   | degree | score |
|-------|--------|--------|-------|
| True  | aparna | MBA    | 90    |
| False | pankaj | BCA    | 40    |
| True  | sudhir | M.Tech | 80    |
| False | Geeku  | MBA    | 98    |

```
name pankaj
degree BCA
score 40
dtype: object
```

# Accessing a DataFrame with a boolean index

```
import pandas as pd
dictionary of lists
dict = {'name':["aparna", "pankaj", "sudhir", "Geeku"],
 'degree': ["BCA", "BCA", "M.Tech", "BCA"],
 'score':[90, 40, 80, 98]}
```

```
creating a dataframe
```

```
df = pd.DataFrame(dict)
```

```
using a comparsion operator for filtering of data
```

```
row=df['degree'] == 'BCA'
```

```
print(row)
```

```
print(df[row])
```

|       | name   | degree | score |
|-------|--------|--------|-------|
| True  | aparna | BCA    | 90    |
| False | pankaj | BCA    | 40    |
| True  | sudhir | M.Tech | 80    |
| False | Geeku  | BCA    | 98    |

0 True

1 True

2 False

3 True

Name: degree, dtype: bool

|   | name   | degree | score |
|---|--------|--------|-------|
| 0 | aparna | BCA    | 90    |
| 1 | pankaj | BCA    | 40    |
| 3 | Geeku  | BCA    | 98    |

# References

- [www.w3schools.com](http://www.w3schools.com)
- [www.tutorialpoint.com](http://www.tutorialpoint.com)
- Python Programming – Pearson – Taneja Kumar

Thank You  
Any Query

# Data Science using Python Programming

## Unit-15: < Pandas Module >

# Contents to be covered

1. Pandas – Handling Missing Values
2. Working with Missing Data –  
( isnull( ), notnull( ), dropna( ), fillna( ), replace( ), interpolate( ) )
3. Iterating over Row and Items in DataFrame
4. Working with Text Data –  
( lower( ) , upper( ) , split( ) , replace( ) , concat( ) )



# Missing Values

Data in real world are rarely clean and homogeneous. They tend to be incomplete, noisy, and inconsistent and it is an important task of a Data scientist to pre-process the data by filling missing values.

## Sources of Missing Values

- User forgot to fill in a field.
- Data was lost while transferring manually from a legacy database.
- There was a programming error.
- Users chose not to fill out a field tied to their beliefs about how the results would be used or interpreted.

### Types of Missing Values

**Standard Missing Values** (Missing values could be: NaN, empty string)

**Non Standard Missing Values** (Missing values could be: ?, -1, -99, -999 and so on.)

# Working with Missing Data

In Pandas missing data is represented by two value:

- **None:** None is a Python singleton object that is often used for missing data in Python code.
- **NaN :** NaN (an acronym for Not a Number), is a special floating-point value recognized by all systems that use the standard IEEE floating-point representation

Pandas treat None and NaN as essentially interchangeable for indicating missing or null values. To facilitate this convention, there are several useful functions for detecting, removing, and replacing null values in Pandas DataFrame :

- `isnull( )`
- `notnull( )`
- `dropna( )`
- `fillna( )`
- `replace( )`
- `interpolate( )`

# Checking for missing values using isnull()

```
importing pandas as pd
import pandas as pd

importing numpy as np
import numpy as np

dictionary of lists
dict = {'First Score':[100, 90, np.nan, 95],
 'Second Score': [30, 45, 56, np.nan],
 'Third Score':[np.nan, 40, 80, 98]}

creating a dataframe from list
df = pd.DataFrame(dict)

using isnull() function
df.isnull()
```

|   | First Score | Second Score | Third Score |
|---|-------------|--------------|-------------|
| 0 | 100.0       | 30.0         | NaN         |
| 1 | 90.0        | 45.0         | 40.0        |
| 2 | NaN         | 56.0         | 80.0        |
| 3 | 95.0        | NaN          | 98.0        |

## Output:

|   | First Score | Second Score | Third Score |
|---|-------------|--------------|-------------|
| 0 | False       | False        | True        |
| 1 | False       | False        | False       |
| 2 | True        | False        | False       |
| 3 | False       | True         | False       |

## Checking for missing values using notnull()

```
importing pandas as pd
import pandas as pd

importing numpy as np
import numpy as np

dictionary of lists
dict = {'First Score':[100, 90, np.nan, 95],
 'Second Score': [30, 45, 56, np.nan],
 'Third Score':[np.nan, 40, 80, 98]}

creating a dataframe using dictionary
df = pd.DataFrame(dict)

using notnull() function
df.notnull()
```

```
First Score Second Score Third Score
0 100.0 30.0 NaN
1 90.0 45.0 40.0
2 NaN 56.0 80.0
3 95.0 NaN 98.0
```

### Output:

|   | First Score | Second Score | Third Score |
|---|-------------|--------------|-------------|
| 0 | True        | True         | False       |
| 1 | True        | True         | True        |
| 2 | False       | True         | True        |
| 3 | True        | False        | True        |

## Filling missing values using fillna()

```
importing pandas as pd
import pandas as pd

importing numpy as np
import numpy as np

dictionary of lists
dict = {'First Score':[100, 90, np.nan, 95],
 'Second Score': [30, 45, 56, np.nan],
 'Third Score':[np.nan, 40, 80, 98]}

creating a dataframe from dictionary
df = pd.DataFrame(dict)

filling missing value using fillna()
df.fillna(0)
```

```
First Score Second Score Third Score
0 100.0 30.0 NaN
1 90.0 45.0 40.0
2 NaN 56.0 80.0
3 95.0 NaN 98.0
```

|   | First Score | Second Score | Third Score |
|---|-------------|--------------|-------------|
| 0 | 100.0       | 30.0         | 0.0         |
| 1 | 90.0        | 45.0         | 40.0        |
| 2 | 0.0         | 56.0         | 80.0        |
| 3 | 95.0        | 0.0          | 98.0        |

## Filling missing values using fillna()

### Filling null values with the previous ones

```
importing pandas as pd
import pandas as pd

importing numpy as np
import numpy as np

dictionary of lists
dict = {'First Score':[100, 90, np.nan, 95],
 'Second Score': [30, 45, 56, np.nan],
 'Third Score':[np.nan, 40, 80, 98]}

creating a dataframe from dictionary
df = pd.DataFrame(dict)

filling a missing value with
previous ones
df.fillna(method='pad')
```

|   | First Score | Second Score | Third Score |
|---|-------------|--------------|-------------|
| 0 | 100.0       | 30.0         | NaN         |
| 1 | 90.0        | 45.0         | 40.0        |
| 2 | NaN         | 56.0         | 80.0        |
| 3 | 95.0        | NaN          | 98.0        |

### Output:

|   | First Score | Second Score | Third Score |
|---|-------------|--------------|-------------|
| 0 | 100.0       | 30.0         | NaN         |
| 1 | 90.0        | 45.0         | 40.0        |
| 2 | 90.0        | 56.0         | 80.0        |
| 3 | 95.0        | 56.0         | 98.0        |

## Filling missing values using fillna()

### Filling null value with the next ones

```
importing pandas as pd
```

```
import pandas as pd
```

```
importing numpy as np
```

```
import numpy as np
```

```
dictionary of lists
```

```
dict = {'First Score':[100, 90, np.nan, 95],
 'Second Score': [30, 45, 56, np.nan],
 'Third Score':[np.nan, 40, 80, 98]}
```

```
creating a dataframe from dictionary
```

```
df = pd.DataFrame(dict)
```

```
filling null value using fillna() function
```

```
df.fillna(method='bfill')
```

|   | First Score | Second Score | Third Score |
|---|-------------|--------------|-------------|
| 0 | 100.0       | 30.0         | NaN         |
| 1 | 90.0        | 45.0         | 40.0        |
| 2 | NaN         | 56.0         | 80.0        |
| 3 | 95.0        | NaN          | 98.0        |

### Output:

|   | First Score | Second Score | Third Score |
|---|-------------|--------------|-------------|
| 0 | 100.0       | 30.0         | 40.0        |
| 1 | 90.0        | 45.0         | 40.0        |
| 2 | 95.0        | 56.0         | 80.0        |
| 3 | 95.0        | NaN          | 98.0        |

## Filling a null values using replace() method

```
importing pandas as pd
import pandas as pd
```

```
importing numpy as np
import numpy as np
```

```
dictionary of lists
dict = {'First Score':[100, 90, np.nan, 95],
 'Second Score': [30, 45, 56, np.nan],
 'Third Score':[np.nan, 40, 80, 98]}
```

```
creating a dataframe from dictionary
df = pd.DataFrame(dict)
df.replace(to_replace = np.nan, value = -99,inplace=True)
print(df)
```

### Output

|   | First Score | Second Score | Third Score |
|---|-------------|--------------|-------------|
| 0 | 100.0       | 30.0         | NaN         |
| 1 | 90.0        | 45.0         | 40.0        |
| 2 | NaN         | 56.0         | 80.0        |
| 3 | 95.0        | NaN          | 98.0        |

|   | First Score | Second Score | Third Score |
|---|-------------|--------------|-------------|
| 0 | 100.0       | 30.0         | -99.0       |
| 1 | 90.0        | 45.0         | 40.0        |
| 2 | -99.0       | 56.0         | 80.0        |
| 3 | 95.0        | -99.0        | 98.0        |



## Dropping missing values using dropna()

#Dropping rows with at least 1 null value.

```
import pandas as pd
import numpy as np
```

# dictionary of lists

```
dict = {'First Score':[100, 90, np.nan, 95],
 'Second Score': [30, np.nan, 45, 56],
 'Third Score':[52, 40, 80, 98],
 'Fourth Score':[np.nan, np.nan, np.nan, 65]}
```

# creating a dataframe from dictionary

```
df = pd.DataFrame(dict)
```

```
print(df)
```

# using dropna() function

```
print(df.dropna())
```

|   | First Score | Second Score | Third Score | Fourth Score |
|---|-------------|--------------|-------------|--------------|
| 0 | 100.0       | 30.0         | 52          | NaN          |
| 1 | 90.0        | NaN          | 40          | NaN          |
| 2 | NaN         | 45.0         | 80          | NaN          |
| 3 | 95.0        | 56.0         | 98          | 65.0         |

|   | First Score | Second Score | Third Score | Fourth Score |
|---|-------------|--------------|-------------|--------------|
| 3 | 95.0        | 56.0         | 98          | 65.0         |

## Drop rows whose all data is missing

```
importing pandas as pd
import pandas as pd
```

```
importing numpy as np
import numpy as np
```

```
dictionary of lists
```

```
dict = {'First Score':[100, np.nan, np.nan, 95],
 'Second Score': [30, np.nan, 45, 56],
 'Third Score':[52, np.nan, 80, 98],
 'Fourth Score':[np.nan, np.nan, np.nan, 65]}
```

```
df = pd.DataFrame(dict)
using dropna() function
df.dropna(how = 'all')
```

| First | Second | Third | Fourth |
|-------|--------|-------|--------|
| 100   | 30     | 52    | NaN    |
| NaN   | NaN    | NaN   | NaN    |
| NaN   | 45     | 80    | NaN    |
| 95    | 56     | 98    | 65     |

|   | First Score | Second Score | Third Score | Fourth Score |
|---|-------------|--------------|-------------|--------------|
| 0 | 100.0       | 30.0         | 52.0        | NaN          |
| 2 | NaN         | 45.0         | 80.0        | NaN          |
| 3 | 95.0        | 56.0         | 98.0        | 65.0         |

## Drop a columns which have at least 1 missing values

```
importing pandas as pd
```

```
import pandas as pd
```

```
importing numpy as np
```

```
import numpy as np
```

|   | First Score | Second Score | Third Score | Fourth Score |
|---|-------------|--------------|-------------|--------------|
| 0 | 100.0       | 30.0         | 52.0        | 60           |
| 1 | NaN         | NaN          | NaN         | 67           |
| 2 | NaN         | 45.0         | 80.0        | 68           |
| 3 | 95.0        | 56.0         | 98.0        | 65           |

```
dictionary of lists
```

```
dict = {'First Score':[100, np.nan, np.nan, 95],
 'Second Score': [30, np.nan, 45, 56],
 'Third Score':[52, np.nan, 80, 98],
 'Fourth Score':[60, 67, 68, 65]}
```

|   | Fourth Score |
|---|--------------|
| 0 | 60           |
| 1 | 67           |
| 2 | 68           |
| 3 | 65           |

```
creating a dataframe from dictionary
```

```
df = pd.DataFrame(dict)
```

```
using dropna() function
```

```
df.dropna(axis = 1)
```

# Iterating over rows and columns

Iteration is a general term for taking each item of something, one after another. Pandas DataFrame consists of rows and columns so, in order to iterate over dataframe, we have to iterate a dataframe like a dictionary.

In Pandas Dataframe we can iterate an element in two ways:

- ✓ Iterating over rows
- ✓ Iterating over columns

# Iteration over rows using itertuples()

In order to iterate over rows, we apply a function `itertuples()` this function return a tuple for each row in the DataFrame. The first element of the tuple will be the row's corresponding index value, while the remaining values are the row values.

```
import pandas as pd
```

```
dict = {'name':["aparna", "pankaj", "sudhir", "Geeku"],
 'degree': ["MBA", "BCA", "M.Tech", "MBA"],
 'score':[90, 40, 80, 98]}
```

```
df = pd.DataFrame(dict)
```

```
for i in df.itertuples():
 print(i)
```

```
Pandas(Index=0, name='aparna', degree='MBA', score=90)
Pandas(Index=1, name='pankaj', degree='BCA', score=40)
Pandas(Index=2, name='sudhir', degree='M.Tech', score=80)
Pandas(Index=3, name='Geeku', degree='MBA', score=98)
```

# Working With Text Data

- Series and Indexes are equipped with a set of string processing methods that make it easy to operate on each element of the array.
- Perhaps most importantly, these methods exclude missing/NA values automatically.
- These are accessed via the str attribute and generally, have names matching the equivalent (scalar) built-in string methods.

# Lowercasing and Uppercasing a Data

1. `str.lower( )` this function converts all uppercase characters to lowercase.
2. `str.upper( )` this function converts all lowercase characters to uppercase.

```
import pandas as pd
Define a dictionary containing employee data
data = {'Name':['Jai', 'Princi', 'Gaurav', 'Anuj'],
 'Age':[27, 24, 22, 32],
 'Address':['Delhi', 'Kanpur', 'Allahabad', 'Kannauj'],
 'Qualification':['Msc', 'MA', 'MCA', 'Phd']}
```

|   | Name   | Age | Address   | Qualification |
|---|--------|-----|-----------|---------------|
| 0 | Jai    | 27  | Delhi     | Msc           |
| 1 | Princi | 24  | Kanpur    | MA            |
| 2 | Gaurav | 22  | Allahabad | MCA           |
| 3 | Anuj   | 32  | Kannauj   | Phd           |

```
Convert the dictionary into DataFrame
df = pd.DataFrame(data)
```

```
converting and overwriting values in column
df["Name"] = df["Name"].str.lower()
print(df)
```

# Splitting and Replacing a Data

- In order to split a data, we use `str.split()`
- This function returns a list of strings after breaking the given string by the specified separator
- It can only be applied to an individual string.
- Pandas `str.split()` method can be applied to a whole series.
- `.str` has to be prefixed every time before calling this method to differentiate it from the Python's default function otherwise, it will throw an error.
- In order to replace a data, we use `str.replace()` this function works like Python `.replace()` method only, but it works on Series too.
- Before calling `.replace()` on a Pandas series, `.str` has to be prefixed in order to differentiate it from the Python's default replace method.



# Splitting and Replacing a Data

```
import pandas as pd
```

```
data = {'Name':['Jai', 'Princi', 'Gaurav', 'Anuj'],
 'Age':[27, 24, 22, 32],
 'Address':['Nagpur', 'Kanpur', 'Allahabad', 'Knnuaj'],
 'Qualification':['Msc', 'MA', 'MCA', 'Phd']}
```

```
df = pd.DataFrame(data)
```

```
df.dropna(inplace = True)
```

```
df["Address"] = df["Address"].str.split("a")
```

```
print(df)
```

|   | Name   | Age | Address   | Qualification |
|---|--------|-----|-----------|---------------|
| 0 | Jai    | 27  | Nagpur    | Msc           |
| 1 | Princi | 24  | Kanpur    | MA            |
| 2 | Gaurav | 22  | Allahabad | MCA           |
| 3 | Anuj   | 32  | Knnuaj    | Phd           |

|   | Name   | Age | Address | Qualification |
|---|--------|-----|---------|---------------|
| 0 | Jai    | 27  | N       | Msc           |
| 1 | Princi | 24  | K       | MA            |
| 2 | Gaurav | 22  | All     | MCA           |
| 3 | Anuj   | 32  | Knnu    | Phd           |

# Splitting and Replacing a Data

## Replace

```
import pandas as pd
```

```
data = {'Name':['Jai', 'Princi', 'Gaurav', 'Anuj'],
 'Age':[27, 25, 22, 25],
 'Address':['Nagpur', 'Kanpur', 'Allahabad', 'Knnuaj'],
 'Qualification':['Msc', 'MA', 'MCA', 'Phd']}
```

```
df = pd.DataFrame(data)
print(df)
```

```
df["Age"] = df["Age"].replace(25 , "Twenty five")
```

```
print(df)
```

|   | Name   | Age | Address   | Qualification |
|---|--------|-----|-----------|---------------|
| 0 | Jai    | 27  | Nagpur    | Msc           |
| 1 | Princi | 25  | Kanpur    | MA            |
| 2 | Gaurav | 22  | Allahabad | MCA           |
| 3 | Anuj   | 25  | Knnuaj    | Phd           |

|   | Name   | Age         | Address   | Qualification |
|---|--------|-------------|-----------|---------------|
| 0 | Jai    | 27          | Nagpur    | Msc           |
| 1 | Princi | Twenty five | Kanpur    | MA            |
| 2 | Gaurav | 22          | Allahabad | MCA           |
| 3 | Anuj   | Twenty five | Knnuaj    | Phd           |

# Concatenation of Data

**str.cat()** this function is used to concatenate strings to the passed caller series of string.

```
import pandas as pd
```

```
data = {'Name':['Jai', 'Princi', 'Gaurav', 'Anuj'],
 'Age':[27, 24, 22, 32],
 'Address':['Nagpur', 'Kanpur', 'Allahabad', 'Kannuaj'],
 'Qualification':['Msc', 'MA', 'MCA', 'Phd']}
```

|   | Name              | Age | Address   | Qualification |
|---|-------------------|-----|-----------|---------------|
| 0 | Jai, Nagpur       | 27  | Nagpur    | Msc           |
| 1 | Princi, Kanpur    | 24  | Kanpur    | MA            |
| 2 | Gaurav, Allahabad | 22  | Allahabad | MCA           |
| 3 | Anuj, Kannuaj     | 32  | Kannuaj   | Phd           |

```
df = pd.DataFrame(data)
```

```
new = df["Address"].copy()
```

```
df["Name"] = df["Name"].str.cat(new, sep = ", ")
```

```
print(df)
```

# Removing Whitespaces of Data

- In order to remove a whitespaces, we use `str.strip()`, `str.rstrip()`, `str.lstrip()` these function used to handle white spaces(including New line) in any text data.
- As it can be seen in the name, `str.lstrip()` is used to remove spaces from the left side of string, `str.rstrip()` to remove spaces from right side of the string and `str.strip()` removes spaces from both sides.
- Since these are pandas function with same name as Python's default functions, `.str` has to be prefixed to tell the compiler that a Pandas function is being called.

# Removing Whitespaces of Data

```
import pandas as pd

data = {'Name':['Jai', 'Princi', 'Gaurav', 'Anuj'],
 'Age':[27, 24, 22, 32],
 'Address':['Nagpur junction', 'Kanpur junction',
 'Nagpur junction', 'Kannuaj junction'],
 'Qualification':['Msc', 'MA', 'MCA', 'Phd']}

df = pd.DataFrame(data)

new = df["Address"].replace("Nagpur junction", " Nagpur junction ").copy()

print(new.str.strip()==" Nagpur junction")
print(new.str.strip()=="Nagpur junction ")
print(new.str.strip()==" Nagpur junction ")
```

## Pandas str methods:

| Function                             | Description                                                                                 |
|--------------------------------------|---------------------------------------------------------------------------------------------|
| <a href="#"><u>str.lower()</u></a>   | Method to convert a string's characters to lowercase                                        |
| <a href="#"><u>str.upper()</u></a>   | Method to convert a string's characters to uppercase                                        |
| <a href="#"><u>str.find()</u></a>    | Method is used to search a substring in each string present in a series                     |
| <a href="#"><u>str.rfind()</u></a>   | Method is used to search a substring in each string present in a series from the Right side |
| <a href="#"><u>str.findall()</u></a> | Method is also used to find substrings or separators in each string in a series             |
| <a href="#"><u>str.isalpha()</u></a> | Method is used to check if all characters in each string in series are alphabetic(a-z/A-Z)  |

## Pandas str methods:

| Function                               | Description                                                                           |
|----------------------------------------|---------------------------------------------------------------------------------------|
| <a href="#"><u>str.isdecimal()</u></a> | Method is used to check whether all characters in a string are decimal                |
| <a href="#"><u>str.title()</u></a>     | Method to capitalize the first letter of every word in a string                       |
| <a href="#"><u>str.len()</u></a>       | Method returns a count of the number of characters in a string                        |
| <a href="#"><u>str.replace()</u></a>   | Method replaces a substring within a string with another value that the user provides |
| <b>str.contains()</b>                  | Method tests if pattern or regex is contained within a string of a Series or Index    |
| <b>str.extract()</b>                   | Extract groups from the first match of regular expression pattern.                    |

# Pandas str methods:

| Function                                | Description                                                                   |
|-----------------------------------------|-------------------------------------------------------------------------------|
| <a href="#"><u>str.startswith()</u></a> | Method tests if the start of each string element matches a pattern            |
| <a href="#"><u>str.endswith()</u></a>   | Method tests if the end of each string element matches a pattern              |
| <a href="#"><u>str.isdigit()</u></a>    | Method is used to check if all characters in each string in series are digits |
| <a href="#"><u>str.lstrip()</u></a>     | Method removes whitespace from the left side (beginning) of a string          |
| <a href="#"><u>str.rstrip()</u></a>     | Method removes whitespace from the right side (end) of a string               |
| <a href="#"><u>str.strip()</u></a>      | Method to remove leading and trailing whitespace from string                  |



## Pandas str methods:

| Function                            | Description                                                                           |
|-------------------------------------|---------------------------------------------------------------------------------------|
| <a href="#"><u>str.split()</u></a>  | Method splits a string value, based on an occurrence of a user-specified value        |
| <a href="#"><u>str.join()</u></a>   | Method is used to join all elements in list present in a series with passed delimiter |
| <a href="#"><u>str.cat()</u></a>    | Method is used to concatenate strings to the passed caller series of string.          |
| <a href="#"><u>str.repeat()</u></a> | Method is used to repeat string values in the same position of passed series itself   |
| <a href="#"><u>str.get()</u></a>    | Method is used to get element at the passed position                                  |

# Pandas str methods:

| Function                                | Description                                                                                                           |
|-----------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <a href="#"><u>str.partition()</u></a>  | Method splits the string only at the first occurrence unlike str.split()                                              |
| <a href="#"><u>str.rpartition()</u></a> | Method splits string only once and that too reversely. It works in a similar way like str.partition() and str.split() |
| <a href="#"><u>str.pad()</u></a>        | Method to add padding (whitespaces or other characters) to every string element in a series                           |
| <a href="#"><u>str.swapcase()</u></a>   | Method to swap case of each string in a series                                                                        |

# References

- [www.w3schools.com](http://www.w3schools.com)
- [www.tutorialpoint.com](http://www.tutorialpoint.com)
- Python Programming – Pearson – Taneja Kumar

Thank You  
Any Query

# Data Science using Python Programming

## Unit-16: < Matplotlib Module >

# Contents to be covered

1. Matplotlib Installation
2. Using Matplotlib
3. Formatting the style of plot
4. Plotting the graph with keyword
5. Plotting the graph with category
6. Scatter Graph
7. Bar Graph
8. Pie Graph

# Python - Matplotlib

When data is shown in the form of pictures, it becomes easy for user to understand it. Representing the data in the form of pictures or graph is called data visualization. For this purpose, we can use pyplot, a submodule of the matplotlib.

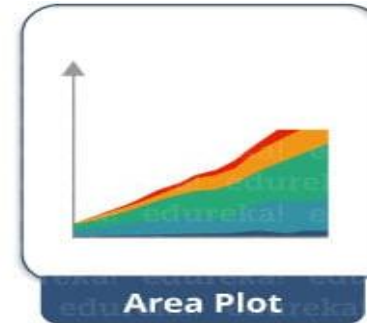
matplotlib.pyplot is a plotting library used for 2D graphics in python programming language. It can be used in python scripts, shell, web application servers and other graphical user interface toolkits.

There are several toolkits which are available that extend python matplotlib functionality

- Basemap
- Cartopy
- Excel tools
- Mplot3d
- Natgrid

# Python Matplotlib : Types of Plots

There are various plots which can be created using python matplotlib. Some of them are listed below:





# Installing Matplotlib

Now, open a *cmd* window like before. Use the next set of commands to install *Matplotlib*:

```
python -m pip install matplotlib
```

# Importing Matplotlib

The package is imported into the Python script by adding the following statement –

```
from matplotlib import pyplot as plt
```

# Matplotlib Example

```
import matplotlib.pyplot as plt
```

```
#the x and y coordinates
```

```
x = ['A','B','C','D','E']
```

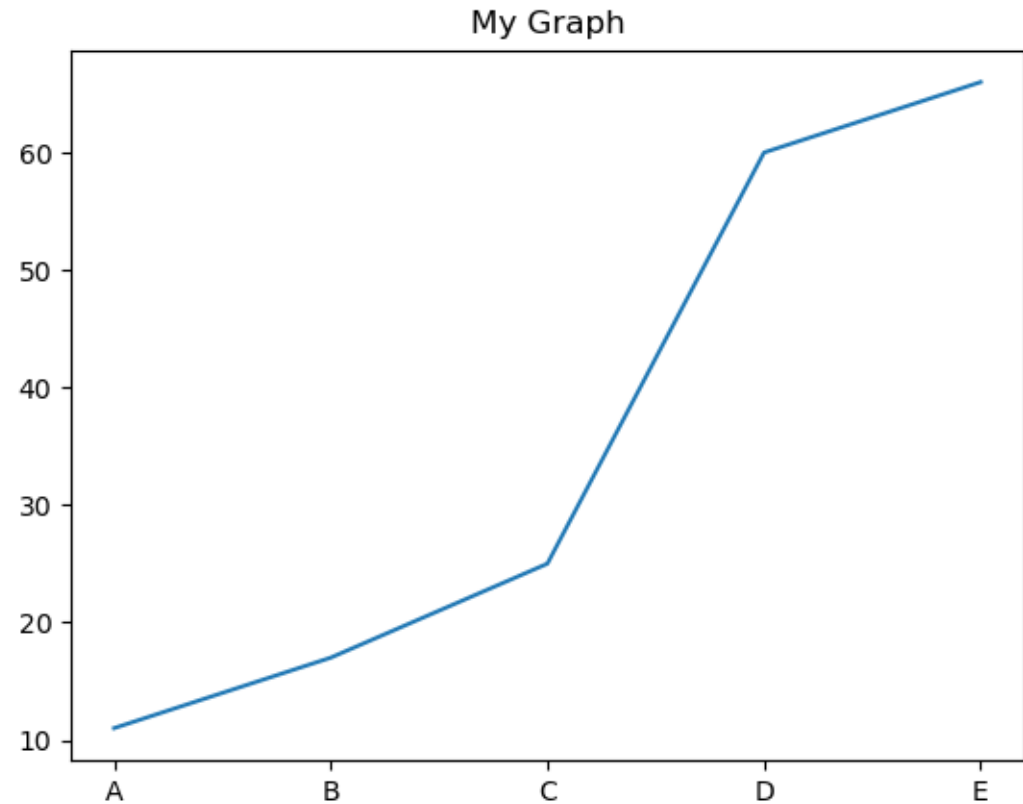
```
y = [11,17,25,60,66]
```

```
plt.title("My Graph")
```

```
Plot the points using matplotlib
```

```
plt.plot(x, y)
```

```
plt.show()
```



# Matplotlib Functions

| Function                 | Description                                                     |
|--------------------------|-----------------------------------------------------------------|
| <code>plot( )</code>     | Plot y versus x as lines and/or markers.                        |
| <code>xlabel( )</code>   | Set the label for the x-axis.                                   |
| <code>ylabel( )</code>   | Set the label for the y-axis.                                   |
| <code>show( )</code>     | Display a figure.                                               |
| <code>axis( )</code>     | Convenience method to get or set some axis properties.          |
| <code>scatter( )</code>  | A scatter plot of y vs x with varying marker size and/or color. |
| <code>subplot( )</code>  | Add a subplot to the current figure.                            |
| <code>bar( )</code>      | Make a bar plot.                                                |
| <code>suptitle( )</code> | Add a centered title to the figure.                             |
| <code>close( )</code>    | Close a figure window.                                          |

# Line Style

| character | description         |
|-----------|---------------------|
| ' - '     | solid line style    |
| ' - - '   | dashed line style   |
| ' - . '   | dash-dot line style |
| ' : '     | dotted line style   |
| ' . '     | point marker        |
| ' , '     | pixel marker        |
| ' o '     | circle marker       |

|       |                       |
|-------|-----------------------|
| ' v ' | triangle_down marker  |
| ' ^ ' | triangle_up marker    |
| ' < ' | triangle_left marker  |
| ' > ' | triangle_right marker |
| ' 1 ' | tri_down marker       |
| ' 2 ' | tri_up marker         |
| ' 3 ' | tri_left marker       |
| ' 4 ' | tri_right marker      |

|       |                 |
|-------|-----------------|
| ' s ' | square marker   |
| ' p ' | pentagon marker |
| ' * ' | star marker     |
| ' h ' | hexagon1 marker |
| ' H ' | hexagon2 marker |
| ' + ' | plus marker     |
| ' x ' | x marker        |

# Formatting the style of your plot

1. For every x, y pair of arguments, there is an optional third argument which is the format string that indicates the color and line type of the plot.
2. The letters and symbols of the format string are from MATLAB, and you concatenate a color string with a line style string.
3. The default format string is 'b-', which is a solid blue line.
4. For example, to plot with red circles, we write

```
plt.plot([1, 2, 3, 4], [1, 4, 9, 16], 'ro')
plt.axis([0, 6, 0, 20])
plt.show()
```

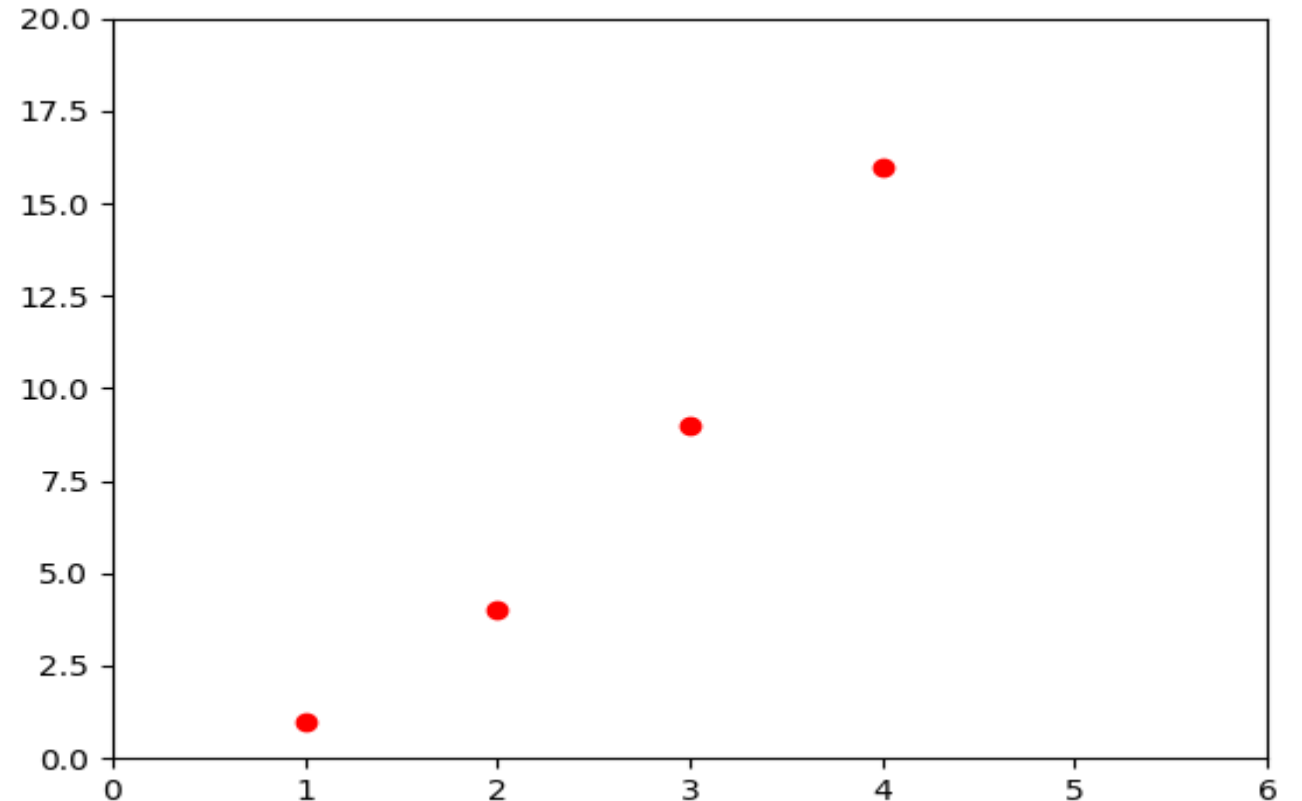
**Note:** The `axis( )` command takes a list of `[xmin, xmax, ymin, ymax]`

| character | color   |
|-----------|---------|
| 'b'       | blue    |
| 'g'       | green   |
| 'r'       | red     |
| 'c'       | cyan    |
| 'm'       | magenta |
| 'y'       | yellow  |
| 'k'       | black   |
| 'w'       | white   |

# Formatting the style of your plot

## Output

```
plt.plot([1, 2, 3, 4], [1, 4, 9, 16], 'ro')
plt.axis([0, 6, 0, 20])
plt.show()
```



# Plotting with keyword strings

There are some instances where you have data in a format that lets you access particular variables with strings. For example, with **numpy.recarray** or **pandas.DataFrame**.

Matplotlib allows you provide such an object with the data keyword argument. If provided, then you may generate plots with the strings corresponding to these variables.



# Plotting with keyword strings

## Example

```
import pandas as pd
import matplotlib.pyplot as plt

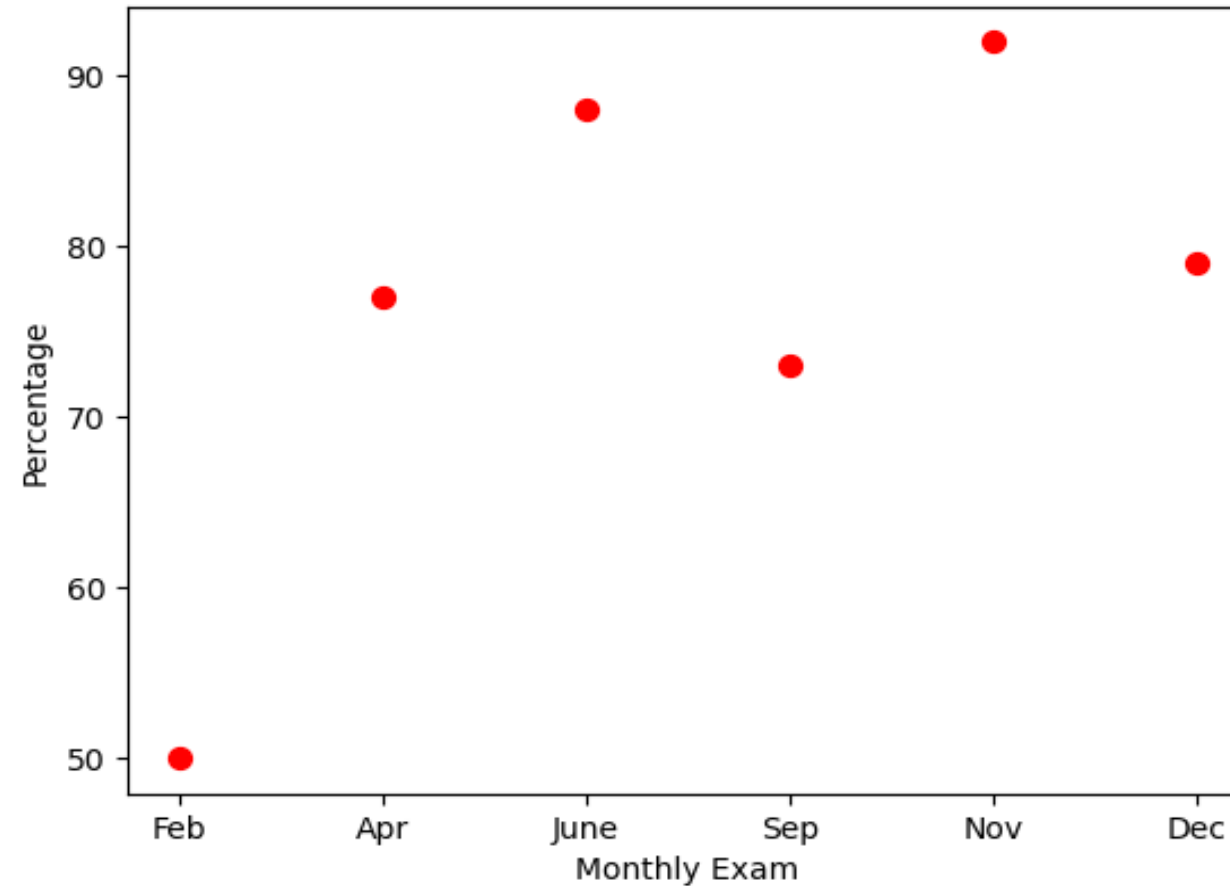
Student1={ 'Monthly': ['Feb' , 'Apr' , 'June' , 'Sep' , 'Nov' , 'Dec'] ,
 'Eng' : [45,67,78,58,87,89],
 'Maths': [55,87,98,88,97,69]
 }

df1=pd.DataFrame(Student1)
df1['Total']= df1['Eng']+df1['Maths']
df1['PCT']=df1['Total']/2

plt.scatter('Monthly', 'PCT', s=50,color='r' , data=df1)
plt.xlabel('Monthly Exam')
plt.ylabel('Percentage')
plt.title('Compare Percentage of Two Students')
plt.show()
```

# Plotting with keyword strings

Output



# Subplot()

With the `subplot()` function you can draw multiple plots in one figure

The `subplot()` function takes three arguments that describes the layout of the figure. The layout is organized in rows and columns, which are represented by the *first* and *second* argument.

The third argument represents the index of the current plot.

```
plt.subplot(1, 2, 1)
```

#the figure has 1 row, 2 columns, and this plot is the *first* plot.

```
plt.subplot(1, 2, 2)
```

#the figure has 1 row, 2 columns, and this plot is the *second* plot.

So, if we want a figure with 2 rows an 1 column (meaning that the two plots will be displayed on top of each

other instead of side-by-side), we can write the syntax like this:

# Example

- `import matplotlib.pyplot as plt`  
`import numpy as np`

`#plot 1:`

```
x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])
plt.subplot(1, 2, 1)
plt.plot(x,y)
```

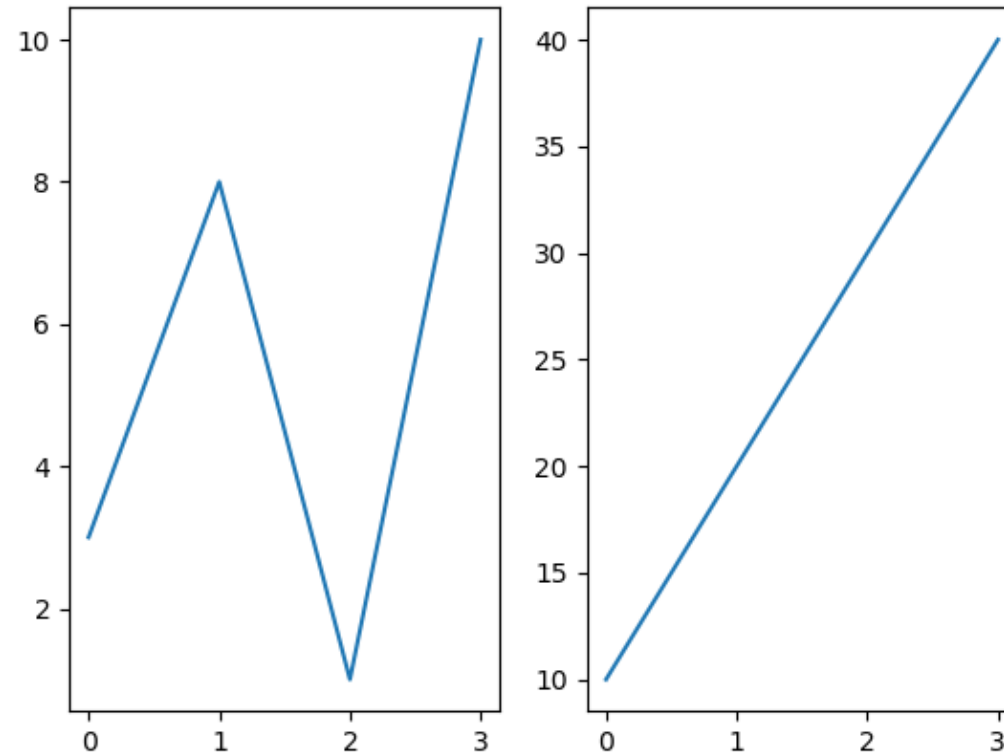
`#plot 2:`

```
x = np.array([0, 1, 2, 3])
y = np.array([10, 20, 30, 40])
```

```
plt.subplot(1, 2, 2)
plt.plot(x,y)
plt.show()
```

-

# output



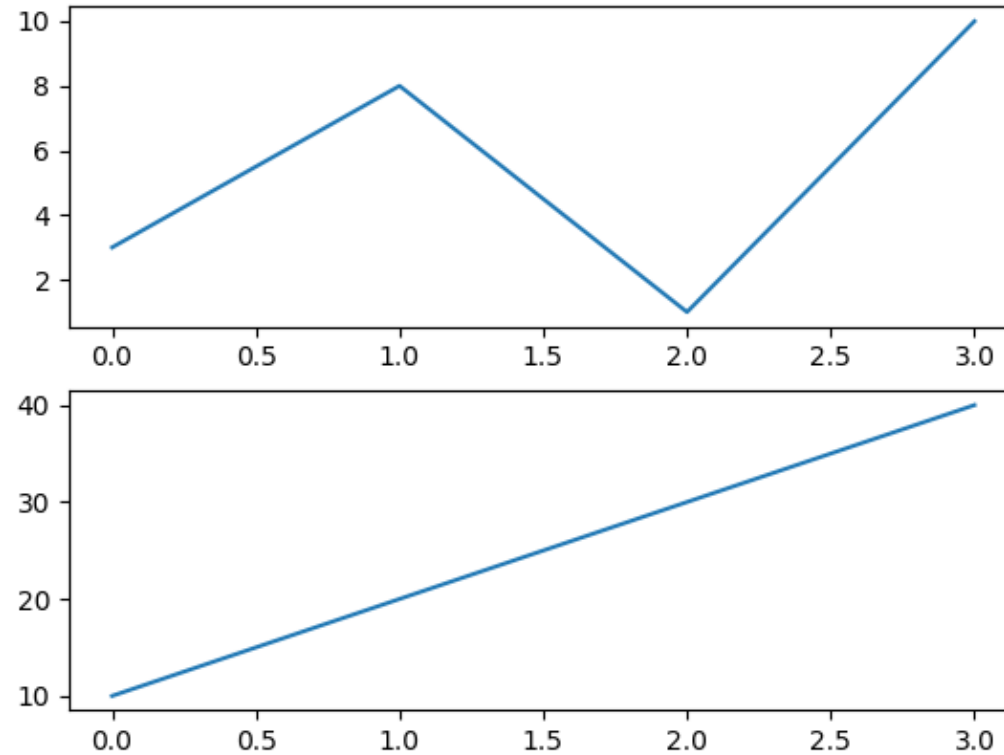
# Example

- ```
import matplotlib.pyplot as plt
import numpy as np
#plot 1:
x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])
plt.subplot(2, 1, 1)
plt.plot(x,y)
#plot 2:
x = np.array([0, 1, 2, 3])
y = np.array([10, 20, 30, 40])

plt.subplot(2, 1, 2)
plt.plot(x,y)

plt.show()
```

output



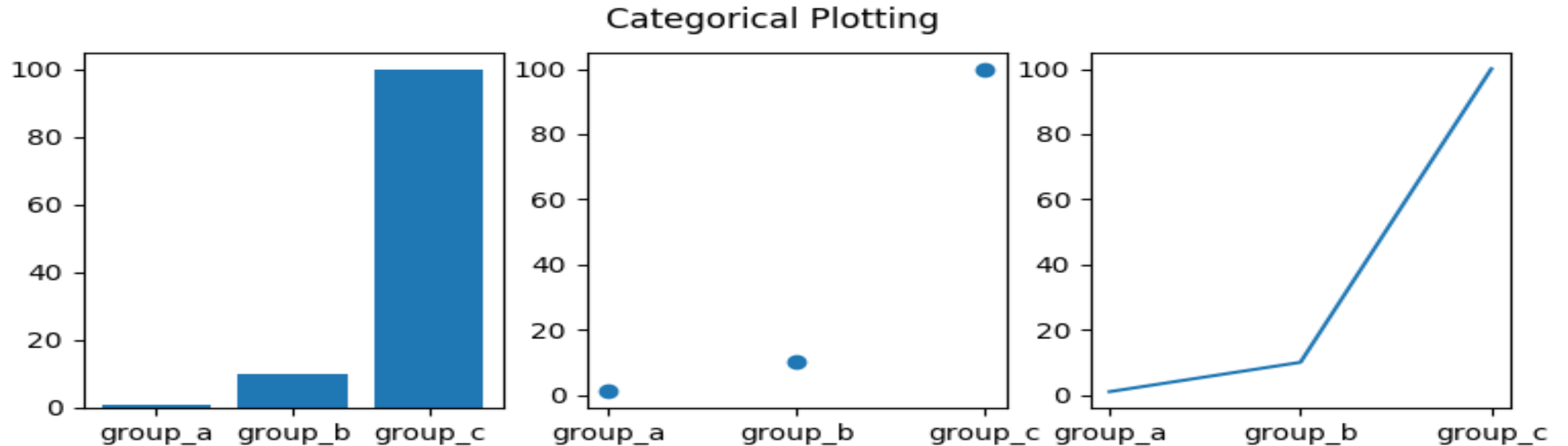
Plotting with categorical variables

It is also possible to create a plot using categorical variables. Matplotlib allows you to pass categorical variables directly to many plotting functions. For example:

```
names = ['group_a', 'group_b', 'group_c']  
values = [1, 10, 100]  
plt.figure(figsize=(9, 3)) #figsize(float, float): These parameter are the width, height in inches.  
  
plt.subplot(131)#1 row,3col,1 index  
plt.bar(names, values)  
  
plt.subplot(132) )#1 row,3col,2 index  
plt.scatter(names, values)  
  
plt.subplot(133) )#1 row,3col,3 index  
plt.plot(names, values)  
  
plt.suptitle('Categorical Plotting')  
plt.show( )
```


Plotting with categorical variables

Output



Controlling line properties

Lines have many attributes that you can set: linewidth, dash style, antialiased, etc.
There are several ways to set line properties.

1. Use **keyword args**:
`plt.plot(x, y, linewidth=2.0)`
2. Use **the setter methods** of a Line2D instance. plot returns a list of Line2D objects;
e.g., `line1= plt.plot(x1, y1).`

Controlling line properties

Use the **setp()** command.

1. The example below uses a MATLAB-style command to set multiple properties on a list of lines.
2. **setp** works transparently with a list of objects or a single object. You can either use python keyword arguments or MATLAB-style string/value pairs:

```
lines = plt.plot(x1, y1)
```

```
# use keyword args
```

```
plt.setp(lines, color='r', linewidth=2.0)
```

```
# or MATLAB style string value pairs
```

```
plt.setp(lines, 'color', 'r', 'linewidth', 2.0)
```

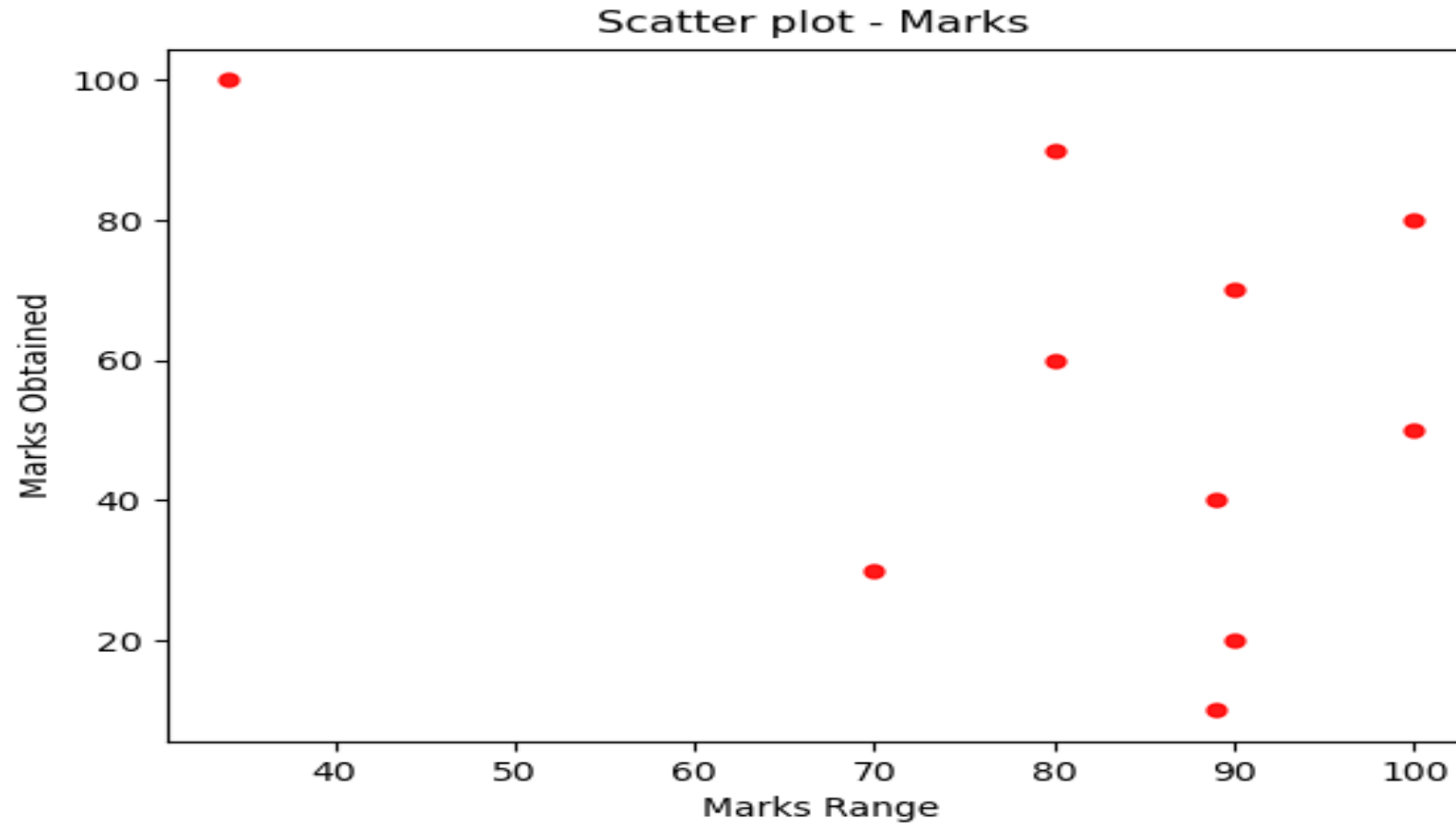
Scatter Plot

1. This type of plot shows all individual data points. Here, they aren't connected with lines.
2. Each data point has the value of the x-axis value and the value from the y-axis values.
3. This type of plot can be **used to display trends or correlations**.
4. In data science, it **shows how 2 variables compare**.
5. To make a scatter plot with Matplotlib, we can use the **plt.scatter()** function.
6. Again, the **first argument** is used for the data on the **horizontal axis**, and the **second** - for the **vertical axis**.

Scatter Plot

```
import matplotlib.pyplot as plt
# Create data
Marks = [89, 90, 70, 89, 100, 80, 90, 100, 80, 34]
Marks_Range=[10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
colors = 'r'
circle_area = 25
contrast=0.9 # has value from 0 - 1
# Plot
plt.scatter(Marks,Marks_Range, s=circle_area, c=colors, alpha=contrast)
plt.title('Scatter plot - Marks ')
plt.xlabel('Marks Range')
plt.ylabel('Marks Obtained')
plt.show( )
```

Scatter Plot



Scatter plot with groups

Data can be classified in several groups. The code below demonstrates that:

```
import numpy as np
import matplotlib.pyplot as plt

# Create data
g1 = np.array( [ [89, 90, 70, 89, 100, 80, 90, 100, 80, 34],
                 [1,2,3,4,5,6,7,8,9,10]] )
g2 = np.array([ [30, 29, 49, 48, 100, 48, 38, 45, 20, 30],
                 [1,2,3,4,5,6,7,8,9,10]])
g3 = np.array([ [ 69, 90, 70, 59, 70, 80, 40, 80, 55, 24],
                 [1,2,3,4,5,6,7,8,9,10]])

# Create plot
x, y = g1
plt.scatter(y, x, alpha=0.8, c='red', edgecolors='none', s=30, label='Red House')

x, y = g2
plt.scatter(y, x, alpha=0.8, c='green', edgecolors='none', s=30, label='Green House')

x, y = g3
plt.scatter(y, x, alpha=0.8, c='blue', edgecolors='none', s=30, label='Blue House')
```

Code continue...

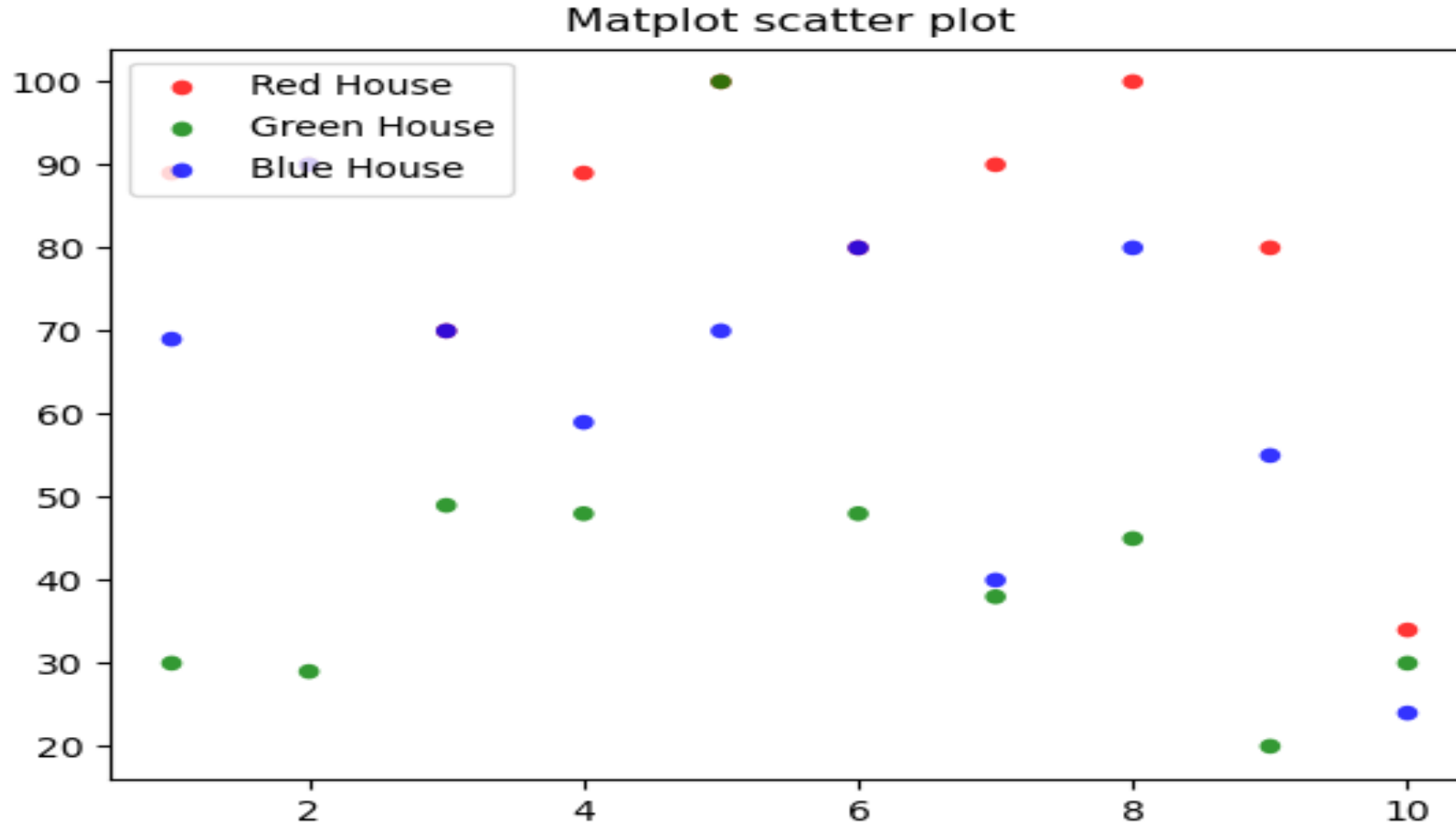
Scatter plot with groups

```
#data = (g1, g2, g3)
#colors = ("red", "green", "blue")
#groups = ("Red House", "Green House", "Blue House")

#for data, color, group in zip(data, colors, groups):
# x, y = data
# plt.scatter(y, x, alpha=0.8, c=color, edgecolors='none', s=30, label=group)

plt.title('Matplot scatter plot')
plt.legend(loc=2)
plt.show()
```


Scatter plot with groups



Bar Chart

Bar chart represents categorical data with rectangular bars. Each bar has a height corresponds to the value it represents.

To make a bar chart with Matplotlib, we'll need the **plt.bar()** function.

Bar Chart

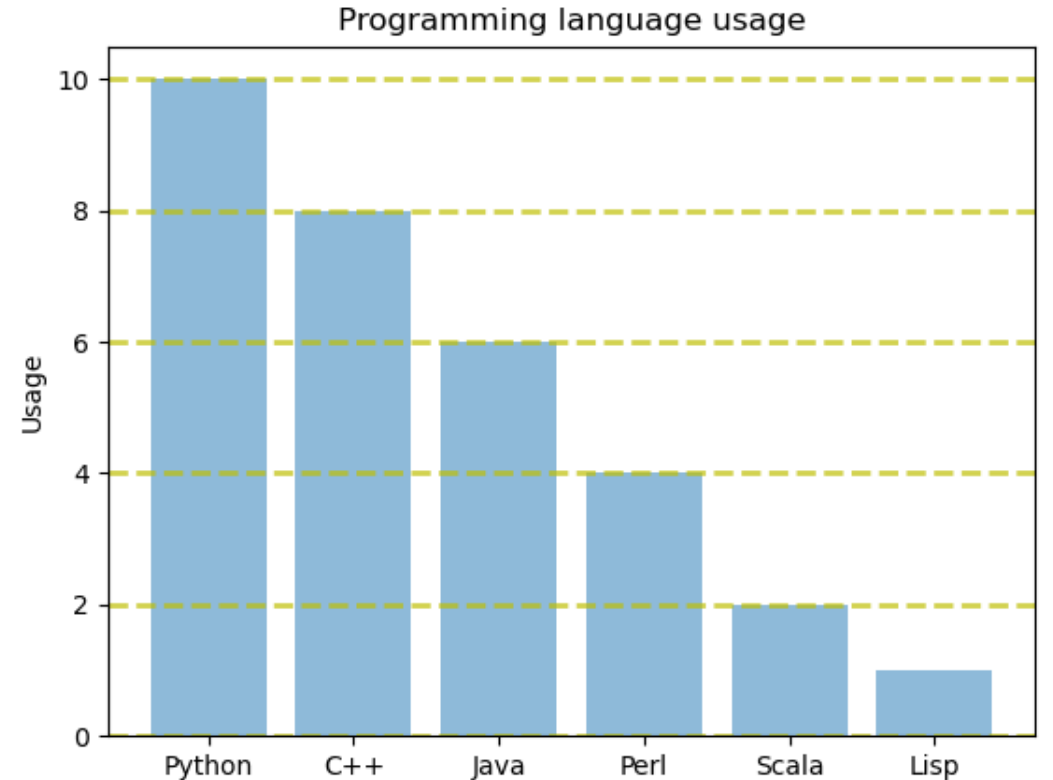
```
import numpy as np
import matplotlib.pyplot as plt

subjects= ('Python', 'C++', 'Java', 'Perl', 'Scala', 'Lisp')
y_pos = np.arange(len(subjects))
performance = [10,8,6,4,2,1]

plt.grid(color='y', linestyle='--', linewidth=2, axis='y', alpha=0.7)
plt.bar(y_pos, performance, align='center', alpha=0.5)
plt.xticks(y_pos, subjects)

plt.ylabel('Usage')
plt.title('Programming language usage')

plt.show()
```



Bar charts

You can compare two data series using this Matplotlib code:

```
import numpy as np
import matplotlib.pyplot as plt

# data to plot
n_groups = 4
Sales_Voltas = (90, 55, 40, 65)
Sales_Samsung = (85, 62, 54, 20)
Year_Sales= ('2016', '2017', '2018', '2020')

# create plot
y_pos= np.arange(n_groups)
bar_width = 0.35
opacity = 0.8
```

Bar charts

```
rects1 = plt.bar(y_pos , Sales_Voltas , bar_width, alpha=opacity, color='b', label='Voltas')
```

```
rects2 = plt.bar(y_pos + bar_width, Sales_Samsung , bar_width, alpha=opacity, color='g', label='Samsung')
```

```
plt.xlabel('Year')
```

```
plt.ylabel('Sales')
```

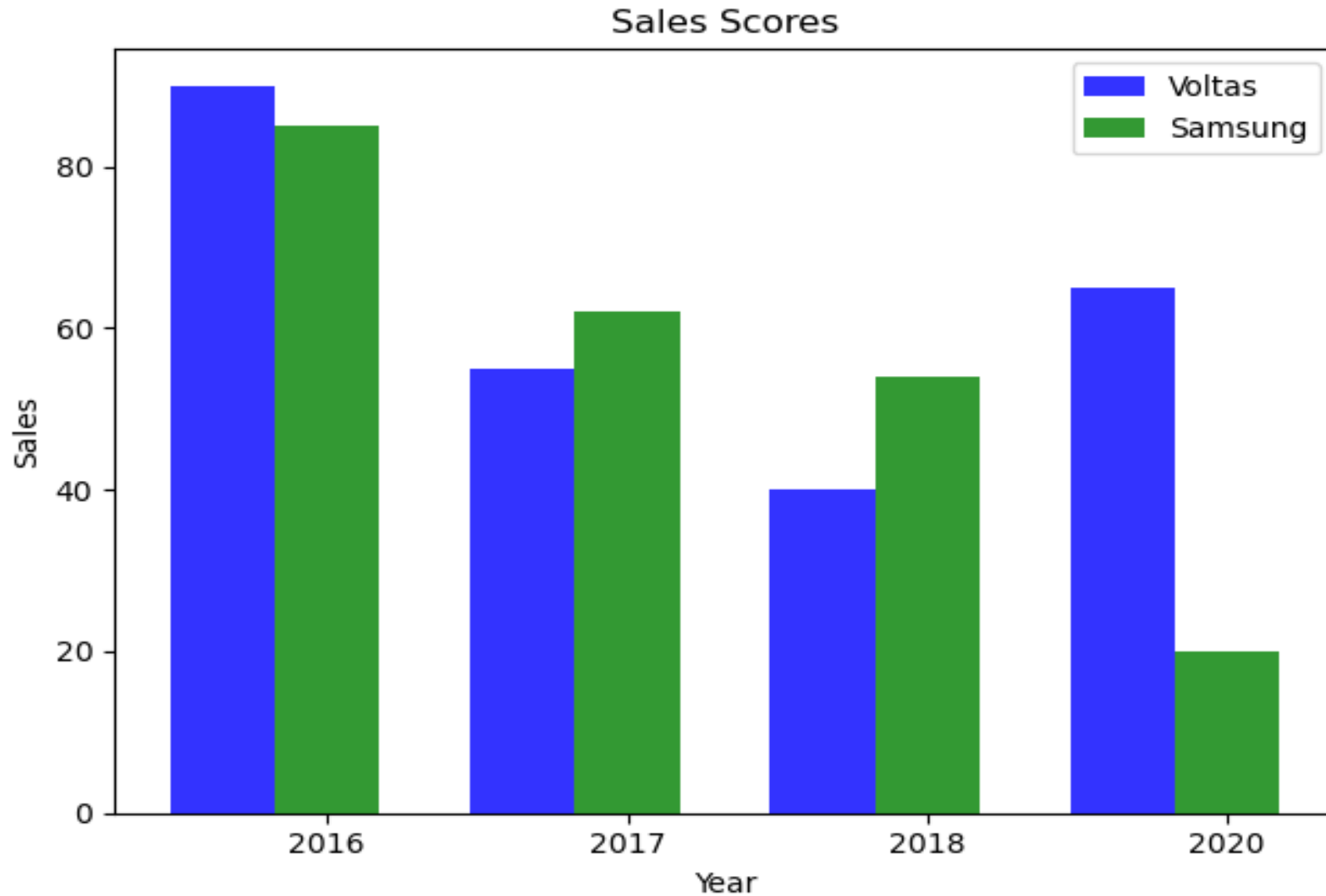
```
plt.title('Sales Scores')
```

```
plt.xticks(y_pos + bar_width, Year_Sales)
```

```
plt.legend()
```

```
plt.show()
```

Bar charts



Pie chart

Pie chart: a circular plot, divided into slices to show numerical proportion. They are widely used in the business world.

```
import matplotlib.pyplot as plt
```

```
labels = 'Python', 'C++', 'Ruby', 'Java'
```

```
sizes = [215, 130, 245, 210]
```

```
colors = ['gold', 'yellowgreen', 'lightcoral', 'lightskyblue']
```

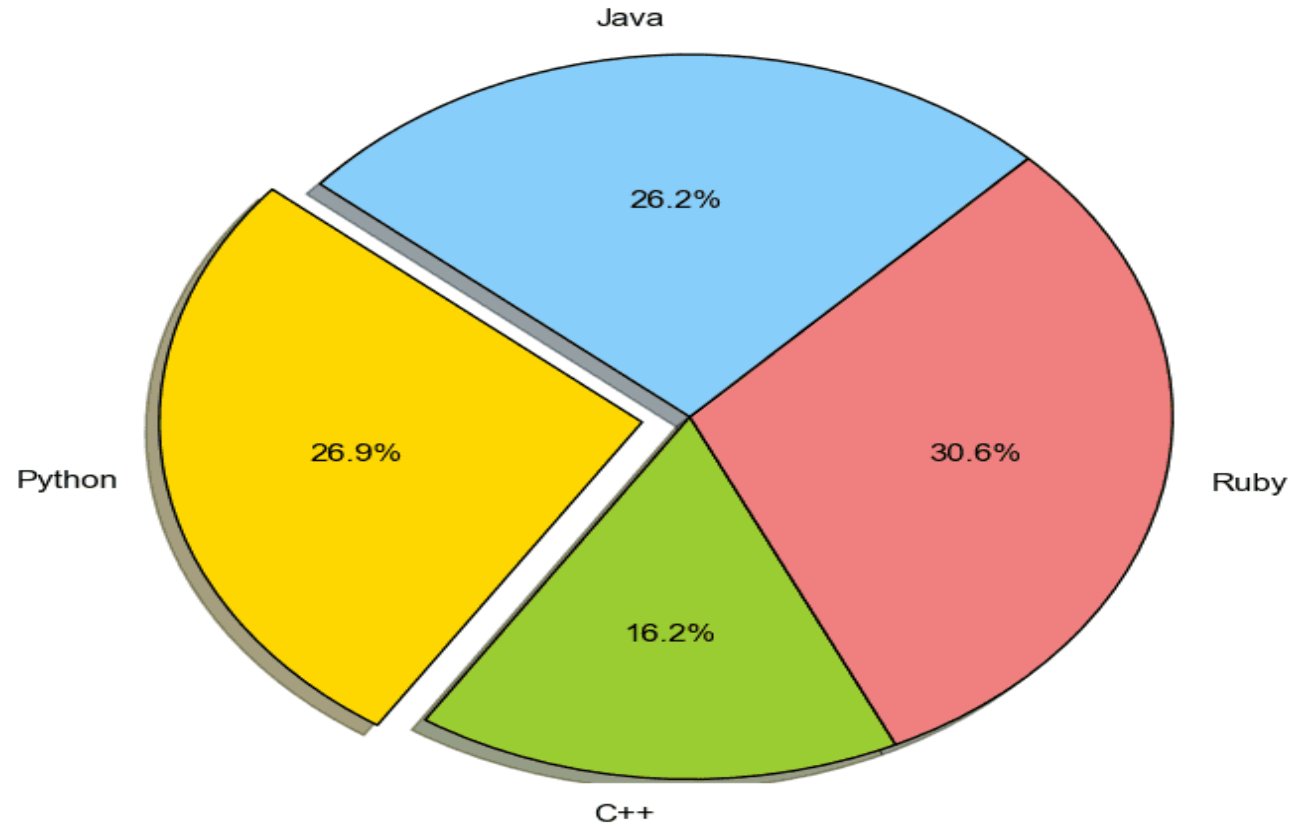
```
explode = (0.1, 0, 0, 0) # explode 1st slice
```

```
plt.pie(sizes, explode=explode, labels=labels, colors=colors,  
autopct='%1.1f%%', shadow=True, startangle=140)
```

```
plt.axis('equal')
```

```
plt.show()
```

Pie chart



Pie chart

To add a legend use the plt.legend() function:

```
import matplotlib.pyplot as plt
```

```
labels = ['Cookies', 'Jellybean', 'Milkshake', 'Cheesecake']
```

```
sizes = [38.4, 40.6, 20.7, 10.3]
```

```
colors = ['yellowgreen', 'gold', 'lightskyblue', 'lightcoral']
```

```
patches, texts = plt.pie(sizes, colors=colors, shadow=True, startangle=90)
```

```
plt.legend(patches, labels, loc="best")
```

```
plt.axis('equal')
```

```
plt.tight_layout()
```

```
plt.show()
```

Legend Location

best

upper right

upper left

lower left

lower right

right

center left

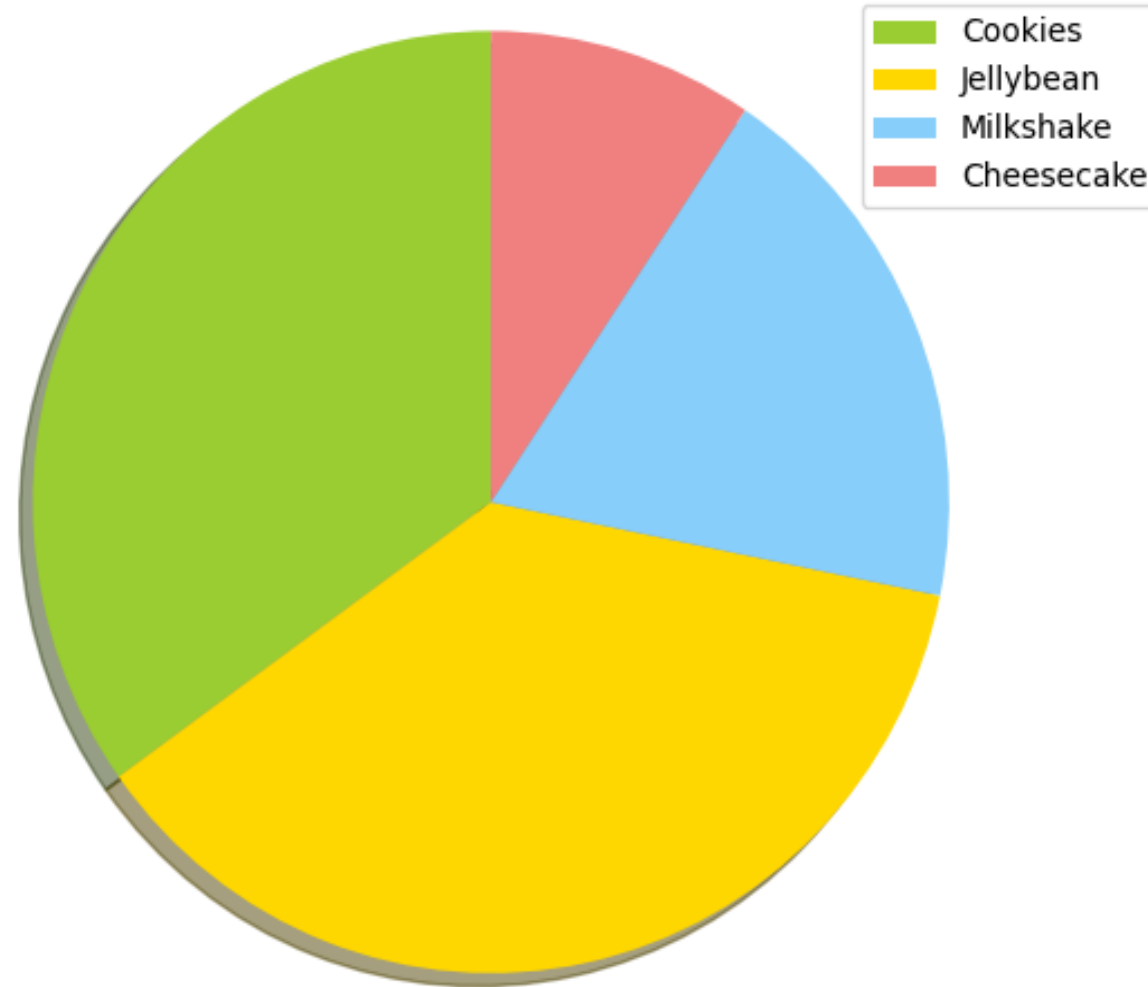
center right

lower center

upper center

center

Pie chart



Histogram

- A histogram is basically used to represent data provided in a form of some groups. It is an accurate method for the graphical representation of numerical data distribution. It is a type of bar plot where X-axis represents the bin ranges while Y-axis gives information about frequency.
- To create a histogram the first step is to create bin of the ranges, then distribute the whole range of the values into a series of intervals, and count the values which fall into each of the intervals. Bins are clearly identified as consecutive, non-overlapping intervals of variables. The `matplotlib.pyplot.hist()` function is used to compute and create histogram of `x`.

Example

```
from matplotlib import pyplot as plt
import numpy as np
```

```
# Creating dataset
```

```
a = np.array([22, 87, 5, 43, 56,
              73, 55, 54, 11,
              20, 51, 5, 79, 31,
              27])
```

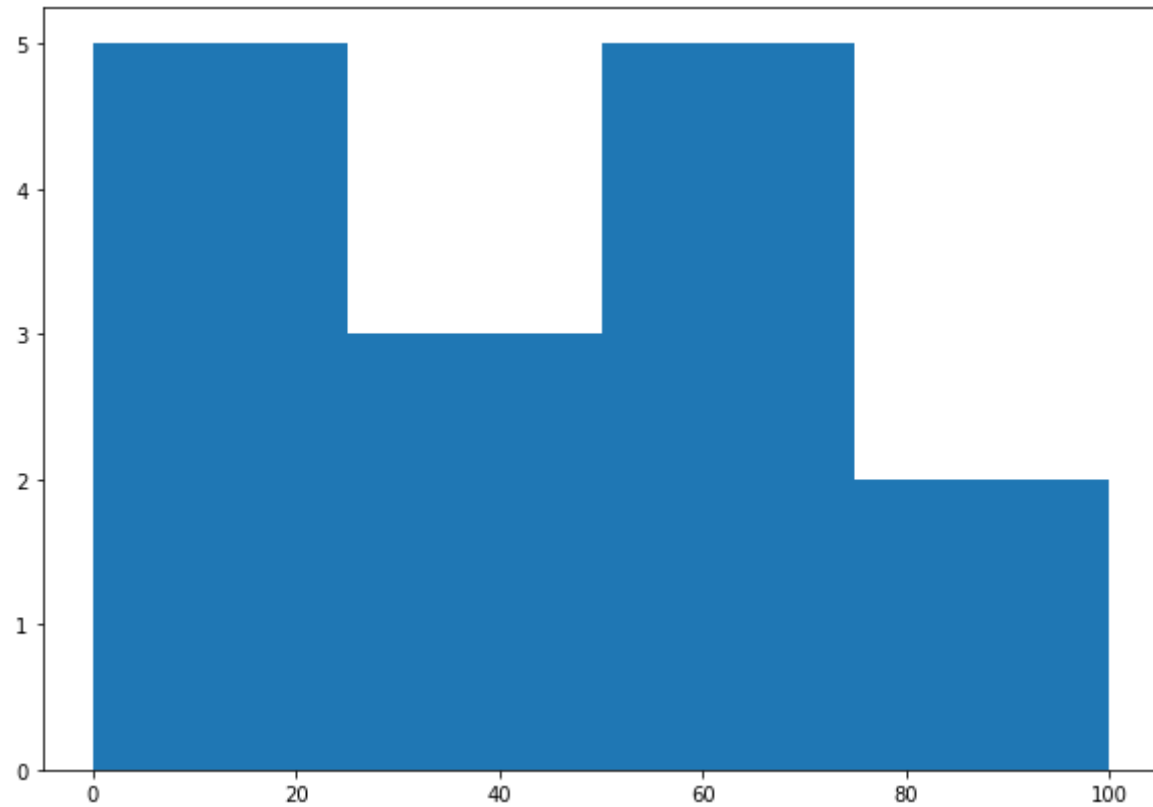
```
# Creating histogram
```

```
fig, ax = plt.subplots(figsize=(10, 7))
ax.hist(a, bins = [0, 25, 50, 75, 100])
```

```
# Show plot
```

```
plt.show()
```

Output



Python Programming and Data Science

Unit-17: < Tkinter Module >

Contents to be covered

1. Tkinter Introduction
2. Using Tkinter
3. Placing Widgets or Components in Tkinter
4. Widgets or Components in Tkinter –
(Button, CheckButton, Entry, Frame, Label , RadioButton)

GUI Programming

1. GUI stands for graphical user interface, a GUI is an interface that uses icons or other visual indicators to interact with electronic devices, rather than only text via a command line.
2. For example, all versions of Microsoft Windows is a GUI, whereas MS-DOS is a command line.

Tkinter

1. Tk was developed as a GUI extension for the Tcl scripting language by John Ousterhout.
2. The first release was in 1991. Tk proved as extremely successful in the 1990's, because it is easier to learn and to use than other toolkits.
3. Tkinter is an inbuilt Python module used to create simple GUI apps.
4. It is the most commonly used module for GUI apps in the Python.

Tkinter

1. Tkinter is a Python binding to the Tk GUI toolkit.
2. It is the standard Python interface to the Tk GUI toolkit, and is Python's de facto standard GUI.
3. Tkinter is included with standard Linux, Microsoft Windows and Mac OS X installs of Python.
4. The name Tkinter comes from Tk interface.

Opening New Window

First GUI Program

```
import tkinter
```

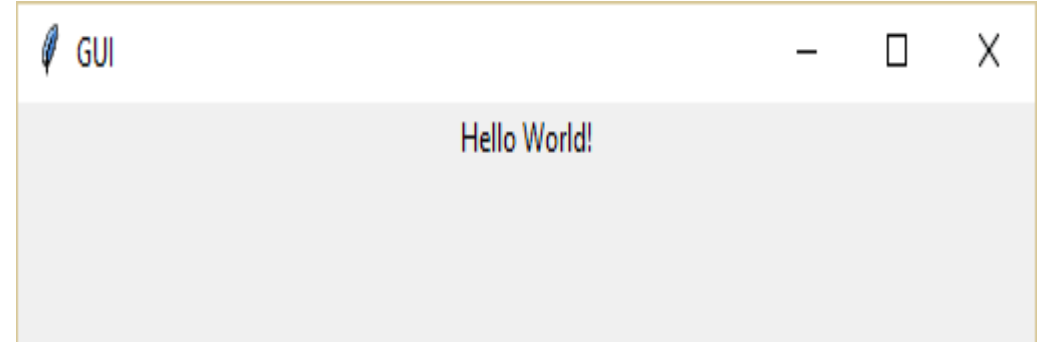
```
window = tkinter.Tk()
```

```
window.title("GUI") # to rename the title of the window
```

```
# pack is used to show the object in the window
```

```
label = tkinter.Label(window, text = "Hello World!").pack( )
```

```
window.mainloop( )
```



Opening New Window

- import the module **tkinter**.
- Initialize the window manager with the **tkinter.Tk()** method and assign it to a variable **window**. This method creates a blank window with close, maximize and minimize buttons.
- Rename the title of the window as you like with the **window.title(title_of_the_window)**.
- **Label** is used to insert some objects into the **window**. Here, we are adding a **Label** with some text.
- **pack()** attribute of the widget is used to display the **widget** in a size it requires.
- Finally, the **mainloop()** method to display the **window** until you manually close it.

Placing Widgets or Component in Window

1. Tkinter allows to access the geometric configuration of the widgets which can organize the widgets in the parent windows.
2. There are mainly three geometry manager classes.
 - **pack() method:** It organizes the widgets in blocks before placing in the parent widget.
 - **grid() method:** It organizes the widgets in grid (table-like structure) before placing in the parent widget.
 - **place() method:** It organizes the widgets by placing them on specific positions directed by the programmer.

Tkinter Widgets

Button

The Button widget is used to display buttons in your application.

Canvas

The Canvas widget is used to draw shapes, such as lines, ovals, polygons and rectangles, in your application.

Checkbutton

The Checkbutton widget is used to display a number of options as checkboxes. The user can select multiple options at a time.

Entry

The Entry widget is used to display a single-line text field for accepting values from a user.

Frame

The Frame widget is used as a container widget to organize other widgets.

Tkinter Widgets

Label

The Label widget is used to provide a single-line caption for other widgets. It can also contain images.

Listbox

The Listbox widget is used to provide a list of options to a user.

Menubutton

The Menubutton widget is used to display menus in your application.

Menu

The Menu widget is used to provide various commands to a user. These commands are contained inside Menubutton.

Message

The Message widget is used to display multiline text fields for accepting values from a user.

Tkinter Widgets

Radiobutton

The Radiobutton widget is used to display a number of options as radio buttons. The user can select only one option at a time.

Scale

The Scale widget is used to provide a slider widget.

Scrollbar

The Scrollbar widget is used to add scrolling capability to various widgets, such as list boxes.

Text

The Text widget is used to display text in multiple lines.

Toplevel

The Toplevel widget is used to provide a separate window container.

Tkinter Widgets

Spinbox

The Spinbox widget is a variant of the standard Tkinter Entry widget, which can be used to select from a fixed number of values.

PanedWindow

A PanedWindow is a container widget that may contain any number of panes, arranged horizontally or vertically.

LabelFrame

A labelframe is a simple container widget. Its primary purpose is to act as a spacer or container for complex window layouts.

tkMessageBox

This module is used to display message boxes in your applications.

Standard attributes

Some of common attributes of widgets are

- Dimensions
- Colors
- Fonts
- Anchors
- Height / Width
- Bitmaps
- Cursors

Label

```
from tkinter import *  
root = Tk()  
w = Label(root, text='This is Label')  
w.pack()  
root.mainloop()
```



Label

It refers to the display box where you can put any text or image which can be updated any time as per the code.

Syntax :

w=Label(master, option=value)

master is the parameter used to represent the parent window.

- **fg**: to set the text color.
- **bg** to set the background color.
- **command**: to call a function.
- **font**: to set the font on the button label.
- **image**: to set the image on the button.
- **width**: to set the width of the button.
- **height**” to set the height of the button.

Button

To add a button in your application, this widget is used.

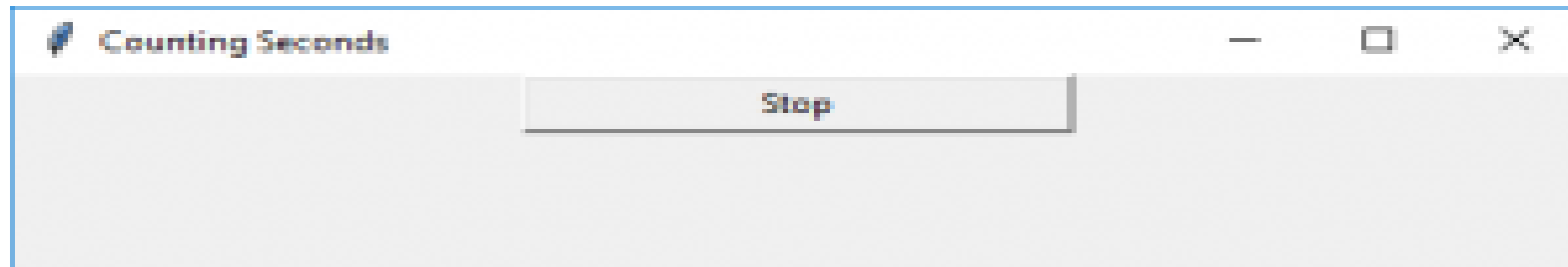
The general syntax is:

`w=Button(master , option=value)`

- **activebackground:** to set the background color when button is under the cursor.
- **activeforeground:** to set the foreground color when button is under the cursor.
- **bg:** to set the normal background color.
- **command:** to call a function.
- **font:** to set the font on the button label.
- **image:** to set the image on the button.
- **width:** to set the width of the button.
- **height:** to set the height of the button.

Button

```
import tkinter as tk  
r = tk.Tk()  
r.title('Counting Seconds')  
button = tk.Button(r, text='Stop', width=25, command=r.destroy)  
button.pack()  
r.mainloop()
```



Button

```
import tkinter as tk
def show():
    tk.Message(r,text="Hello", fg='red', font='Arial', bg='yellow', relief=tk.RAISED).pack()
    return

r = tk.Tk()
r.title('Show Message')
button = tk.Button(r, text='Click Here', font='Arial', fg='brown', width=25, command=show)
button.pack()
r.mainloop()
```



Entry

It is used to input the single line text entry from the user.. For multi-line text input, Text widget is used.

Syntax

w=Entry(master, option=value)

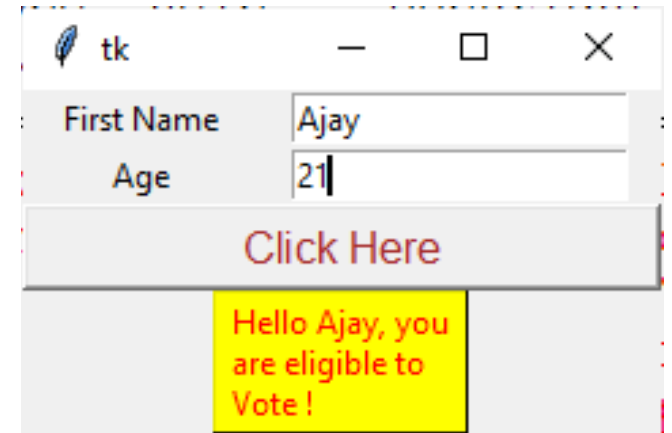
- **bd:** to set the border width in pixels.
- **bg:** to set the normal background color.
- **cursor:** to set the cursor used.
- **command:** to call a function.
- **highlightcolor:** to set the color shown in the focus highlight.
- **width:** to set the width of the button.
- **height:** to set the height of the button.

Entry

```
from tkinter import *
def show():
    age=(vAge.get())
    if age>=18: msg= "Hello "+ vName.get() + ", you are eligible to Vote !"
    else:  msg= "Hello "+ vName.get() + ", you are not eligible to Vote !"
    m1=Message(master,text=msg, fg='red', bg='yellow', width=100,relief=RAISED)
    m1.grid(row=3 , columnspan=2)
    return

master = Tk()
Label(master, text='First Name').grid(row=0)
Label(master, text='Age').grid(row=1)
vName=StringVar()
vAge=IntVar()
e1=Entry(master, textvariable=vName).grid(row=0, column=1)
e2=Entry(master, textvariable=vAge).grid(row=1, column=1)
b1 = Button(master, text='Click Here', font='Arial', fg='brown', width=25, command=show)
b1.grid(row=2,columnspan=2)
```

mainloop()



CheckBox

To select any number of options by displaying a number of options to a user as toggle buttons.

Syntax :

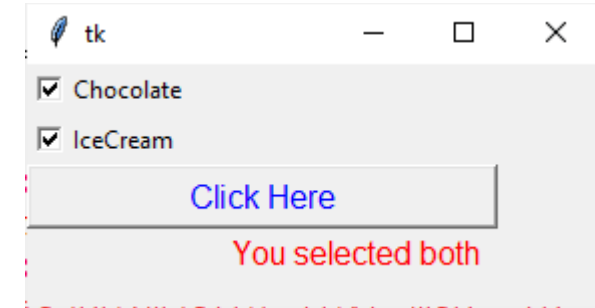
w = CheckBox(master, option=value)

- **Title:** To set the title of the widget.
- **activebackground:** to set the background color when widget is under the cursor.
- **activeforeground:** to set the foreground color when widget is under the cursor.
- **bg:** to set the normal background color.
- **command:** to call a function.
- **font:** to set the font on the button label.
- **image:** to set the image on the widget.

CheckBox

```
from tkinter import *
def show():
    v1=int(var1.get())
    v2=int(var2.get())
    if v1==1 and v2==1 : msg.set( "You selected both")
    if v1==1 and v2==0: msg.set( "You selected Chocolate")
    if v1==0 and v2==1: msg.set( "You selected IceCream")
    if v1==0 and v2==0: msg.set( "You selected None")
    return

master = Tk()
var1 = IntVar()
var2 = IntVar()
msg=StringVar()
Checkbutton(master, text='Chocolate', variable=var1).grid(row=1, sticky=W)
Checkbutton(master, text='IceCream', variable=var2).grid(row=2, sticky=W)
b1 = Button(master, text='Click Here', font='Arial', fg='blue', width=25, command=show)
b1.grid(row=3,columnspan=2)
Label(master, fg='red', font='Arial', textvariable=msg).grid(row=4 , column=1)
mainloop()
```



RadioButton

It is used to offer multi-choice option to the user. It offers several options to the user and the user has to choose one option.

The general syntax is:

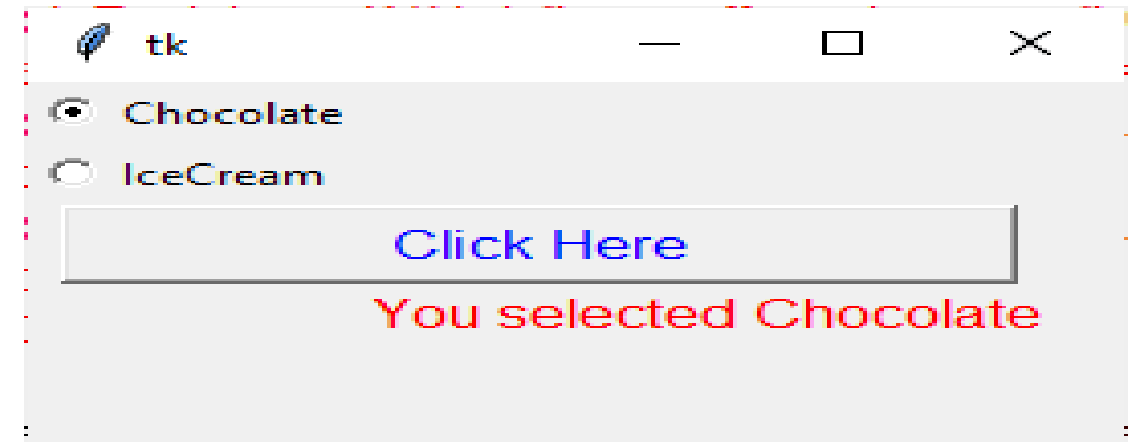
w = RadioButton(master, option=value)

- **activebackground**: to set the background color when widget is under the cursor.
- **activeforeground**: to set the foreground color when widget is under the cursor.
- **bg**: to set the normal background color.
- **command**: to call a function.
- **font**: to set the font on the button label.
- **image**: to set the image on the widget.
- **width**: to set the width of the label in characters.
- **height**: to set the height of the label in characters.

RadioButton

```
from tkinter import *  
def show():  
    v1=int(var1.get())  
    if v1==1 : msg.set( "You selected Chocolate")  
    if v1==2: msg.set( "You selected IceCream")  
    return
```

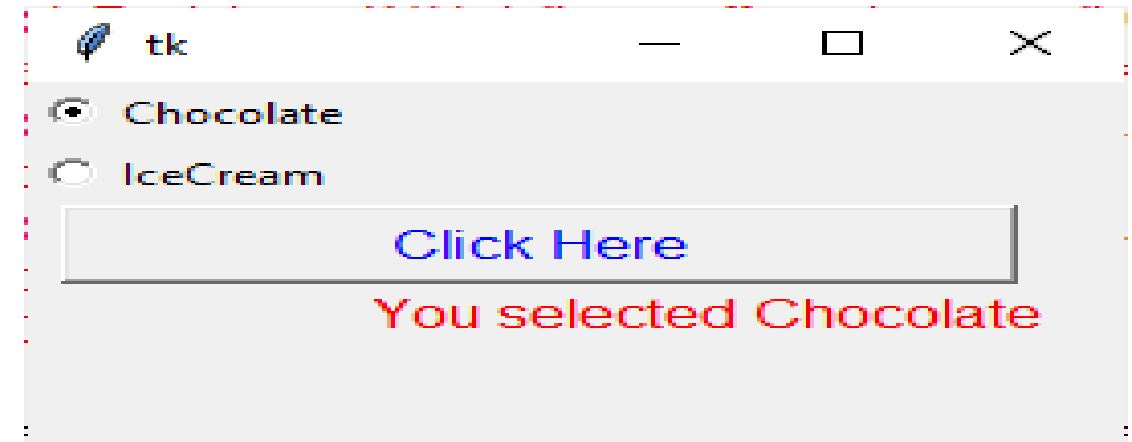
```
master = Tk()  
var1 = IntVar()  
msg=StringVar()  
var1.set(2)      # default selection  
Radiobutton(master, text='Chocolate', variable=var1 , value=1).grid(row=1, sticky=W)  
Radiobutton(master, text='IceCream', variable=var1, value=2).grid(row=2, sticky=W)  
  
b1 = Button(master, text='Click Here', font='Arial', fg='blue', width=25, command=show)  
b1.grid(row=3,columnspan=2)  
Label(master, fg='red', font='Arial', textvariable=msg).grid(row=4 , column=1)  
mainloop()
```



RadioButton

```
from tkinter import *  
def show():  
    v1=int(var1.get())  
    if v1==1 : msg.set( "You selected Chocolate")  
    if v1==2: msg.set( "You selected IceCream")  
    return
```

```
master = Tk()  
var1 = IntVar()  
msg=StringVar()  
var1.set(2)      # default selection  
Radiobutton(master, text='Chocolate', variable=var1 , value=1, indicatoron=0).grid(row=1, sticky=W)  
Radiobutton(master, text='IceCream', variable=var1, value=2 , indicatoron=0).grid(row=2, sticky=W)  
  
b1 = Button(master, text='Click Here', font='Arial', fg='blue', width=25, command=show)  
b1.grid(row=3,columnspan=2)  
Label(master, fg='red', font='Arial', textvariable=msg).grid(row=4 , column=1)  
mainloop()
```



Frame

It acts as a container to hold the widgets. It is used for grouping and organizing the widgets.

Syntax :

w = Frame(master, option=value)

- master is the parameter used to represent the parent window.
- **highlightcolor**: To set the color of the focus highlight when widget has to be focused.
- **bd**: to set the border width in pixels.
- **bg**: to set the normal background color.
- **cursor**: to set the cursor used.
- **width**: to set the width of the widget.
- **height**: to set the height of the widget.

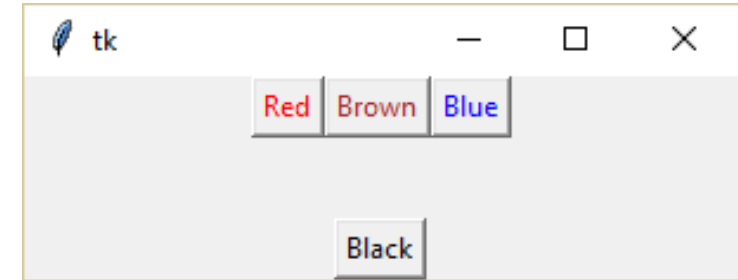
Frame

```
from tkinter import *  
root = Tk()  
topframe = Frame(root)  
topframe.pack()
```

```
bottomframe = Frame(root)  
bottomframe.pack( side = BOTTOM )
```

```
redbutton = Button(topframe, text = 'Red', fg = 'red')  
redbutton.pack( side = LEFT)  
greenbutton = Button(topframe, text = 'Brown', fg = 'brown')  
greenbutton.pack( side = LEFT )  
bluebutton = Button(topframe, text = 'Blue', fg = 'blue')  
bluebutton.pack( side = LEFT )
```

```
blackbutton = Button(bottomframe, text = 'Black', fg = 'black')  
blackbutton.pack( side = BOTTOM)  
root.mainloop()
```



Data Science using Python Programming

Unit-18: < Tkinter Module >

Contents to be covered

1. Widgets in Tkinter –
(Images on Label, ListBox, Menu , Text)
2. Tkinter Events
3. Events Binding
4. Complete Program using Tkinter
5. Graphs Plotting using Tkinter

Image on Label

#Using the tkinter PhotoImage

from tkinter import *

```
root = Tk()
```

```
root.geometry("700x500")
```

```
img= PhotoImage(file="d:\\aaa.png")
```

```
Label(root, image=img).grid(row=1)
```

```
root.mainloop()
```

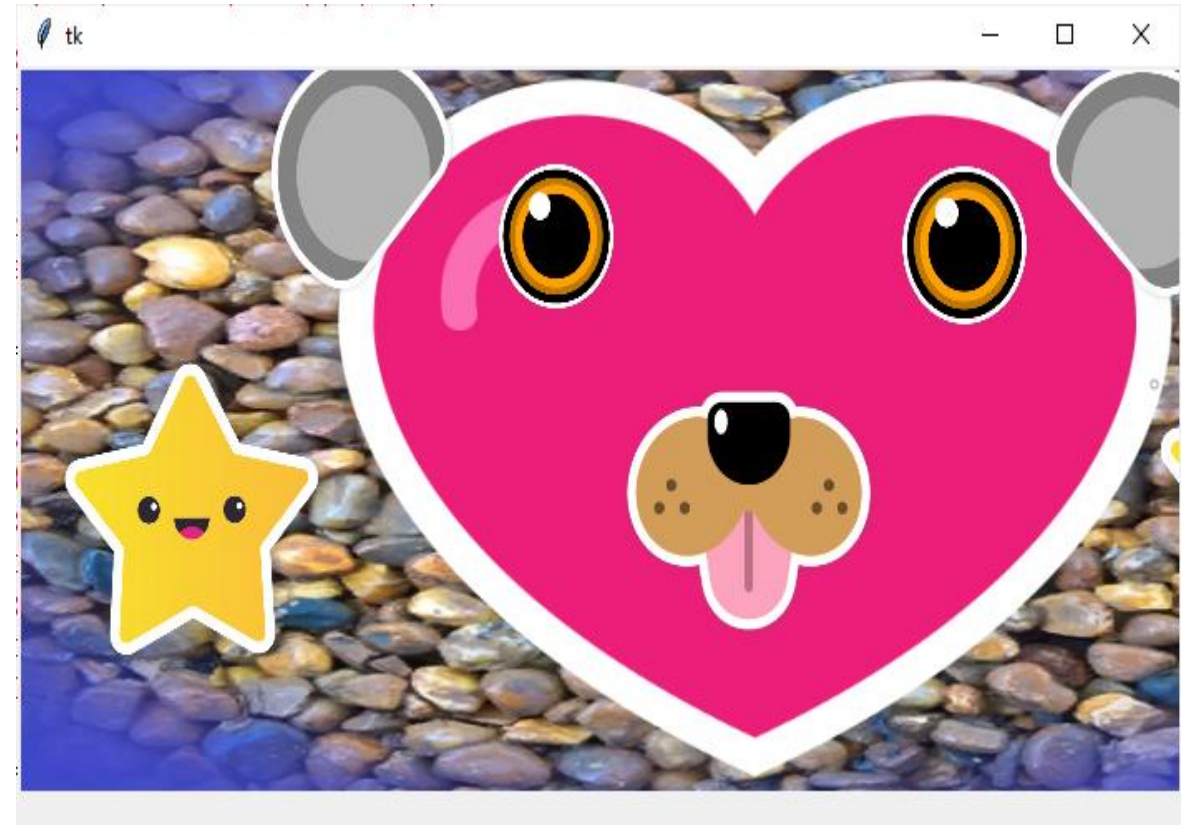


Image on Label

```
from tkinter import *  
#python -m pip install pillow  
from PIL import Image , ImageTk  
  
root = Tk()  
root.geometry("700x500")  
logo=Image.open( "d:\\aaa.png").resize( (100,100), Image.BOX )  
img= ImageTk.PhotoImage( logo)  
Label(root, height=300, width=300, image=img).grid(row=1)  
root.mainloop()
```



Listbox

It offers a list to the user from which the user can accept any number of options.
The general syntax is:

w = Listbox(master, option=value)

master is the parameter used to represent the parent window.

- **highlightcolor:** To set the color of the focus highlight when widget has to be focused.
- **bg:** to set the normal background color.
- **bd:** to set the border width in pixels.
- **font:** to set the font on the button label.
- **image:** to set the image on the widget.
- **width:** to set the width of the widget.
- **height:** to set the height of the widget.

Listbox

It offers a list to the user from which the user can accept any number of options.
The general syntax is:

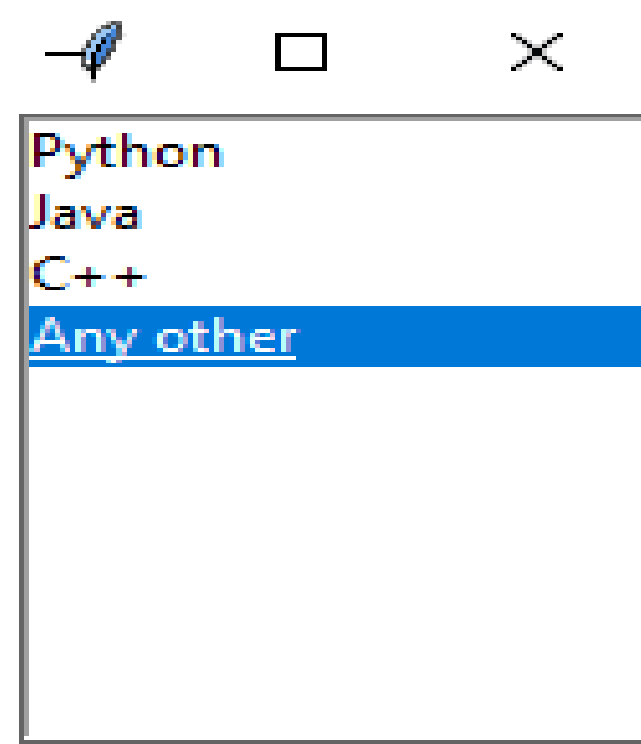
w = Listbox(master, option=value)

master is the parameter used to represent the parent window.

- **highlightcolor:** To set the color of the focus highlight when widget has to be focused.
- **bg:** to set the normal background color.
- **bd:** to set the border width in pixels.
- **font:** to set the font on the button label.
- **image:** to set the image on the widget.
- **width:** to set the width of the widget.
- **height:** to set the height of the widget.

Listbox

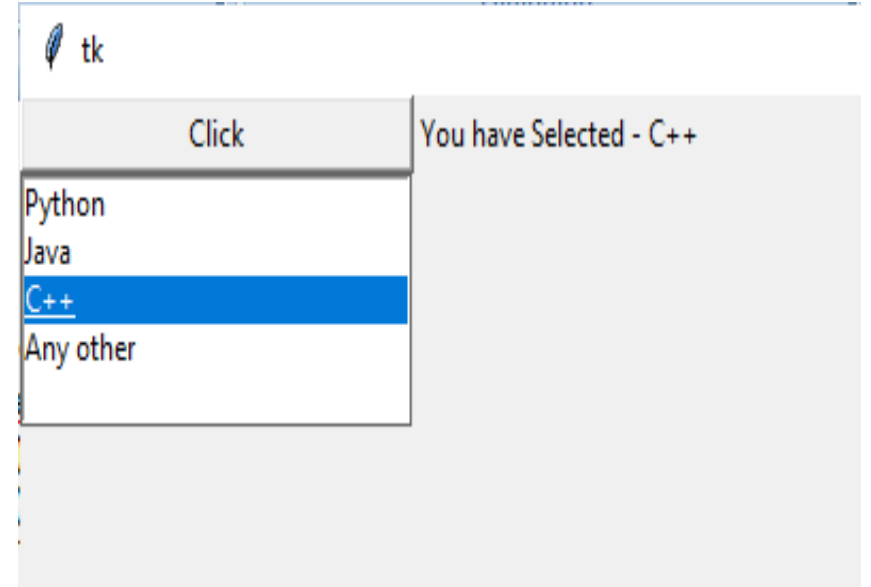
```
from tkinter import *  
top = Tk()  
Lb = Listbox(top)  
Lb.insert(1, 'Python')  
Lb.insert(2, 'Java')  
Lb.insert(3, 'C++')  
Lb.insert(4, 'Any other')  
Lb.pack()  
top.mainloop()
```



Listbox - Fetching Selected Value

Show the selected value in Label

```
from tkinter import *  
def show( ):  
    v=Lb.get(Lb.curselection())  
    Lb1.set("You have Selected - " + v)  
    return  
w = Tk()  
#.geometry("window width x height + position right + down")  
w.geometry("700x400+300+300")  
Lb = Listbox(w, width=30, height=5)  
Lb.insert(1, 'Python')  
Lb.insert(2, 'Java')  
Lb.insert(3, 'C++')  
Lb.insert(4, 'Any other')  
Lb.grid( row=1 )  
Lb1=StringVar()  
b1=Button(w, text="Click",width=25, command=show).grid(row=0)  
lb1=Label(w, textvariable=Lb1).grid( row=0, column=1)  
w.mainloop()
```



PanedWindow

It is a part of top-down menu which stays on the window all the time. Every menubutton has its own functionality.

Syntax :

w = PanedWindow(master, option=value)

master is the parameter used to represent the parent window.

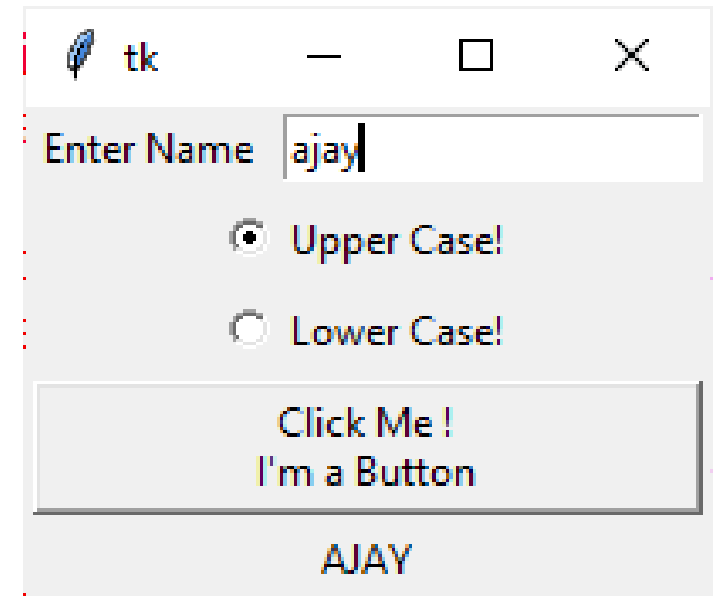
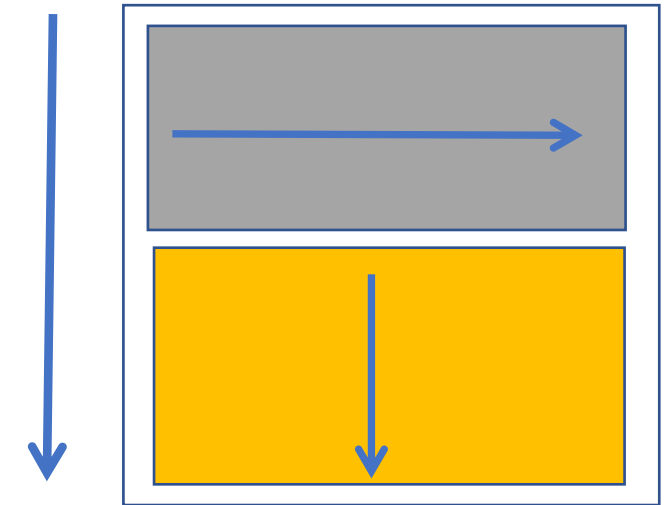
- **orient**: Set the orientation for child widgets. Default is HORIZONTAL.
- **bg**: to set the normal background color.
- **bd**: to set the size of border around the indicator.
- **cursor**: To appear the cursor when the mouse over the menubutton.
- **image**: to set the image on the widget.
- **width**: to set the width of the widget.
- **height**: to set the height of the widget.
- **highlightcolor**: To set the color of the focus highlight when widget has to be focused.

PanedWindow

```
from tkinter import *
def show():
    v=var2.get()
    s=var1.get()
    if v==1: var3.set(s.upper())
    if v==2: var3.set(s.lower())

# main tkinter window
root = Tk()
var1=StringVar()
var2=IntVar()
var3=StringVar()

# panedwindow object
pwM = PanedWindow(orient='vertical')
pwHTop = PanedWindow(orient='horizontal')
pwHBot = PanedWindow(orient='vertical')
pwM.add(pwHTop)
pwM.add(pwHBot)
```



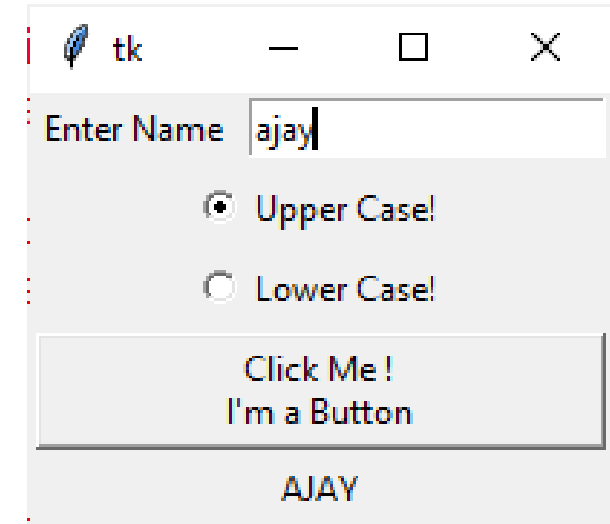
PanedWindow

```
L1=Label(pwHTop, text='Enter Name ' )  
L1.pack(side = LEFT)  
pwHTop.add(L1)  
E1=Entry(pwHTop, textvariable=var1)  
E1.pack(side = LEFT)  
pwHTop.add(E1)
```

```
R1 = Radiobutton(pwHBot, text ="Fact!", value=1, variable=var2)  
R1.pack(side = TOP)  
R2 = Radiobutton(pwHBot, text ="Sum !", value=2, variable=var2)  
R2.pack(side = TOP)  
pwHBot.add(R1)  
pwHBot.add(R2)
```

```
top=Button(pwHBot, text ="Click Me !\nI'm a Button", command=show)  
pwHBot.add(top)  
L2=Label(pwHBot, text='Enter Name ', textvariable=var3)  
L2.pack(side = LEFT)  
pwHBot.add(L2)
```

```
pwM.pack(fill = BOTH, expand = True)  
mainloop()
```



Menu

It is used to create all kinds of menus used by the application.

The general syntax is:

`w = Menu(master, option=value)`

master is the parameter used to represent the parent window.

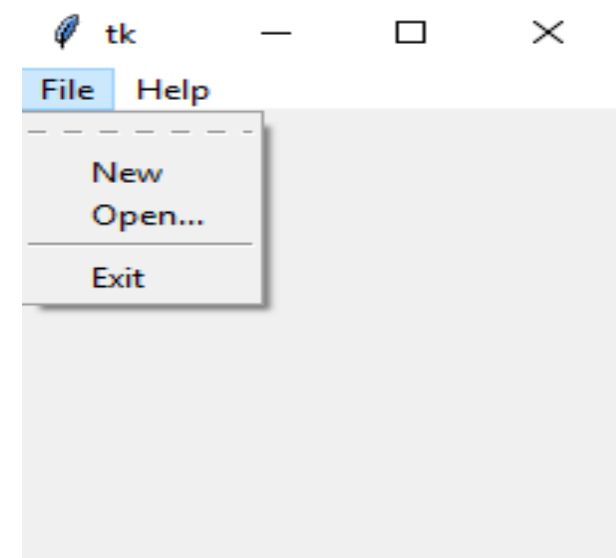
- **title:** To set the title of the widget.
- **activebackground:** to set the background color when widget is under the cursor.
- **activeforeground:** to set the foreground color when widget is under the cursor.
- **bg:** to set the normal background color.
- **command:** to call a function.
- **font:** to set the font on the button label.
- **image:** to set the image on the widget.

Menu

```
from tkinter import *
root = Tk()
M= Menu(root)
root.config(menu=M)

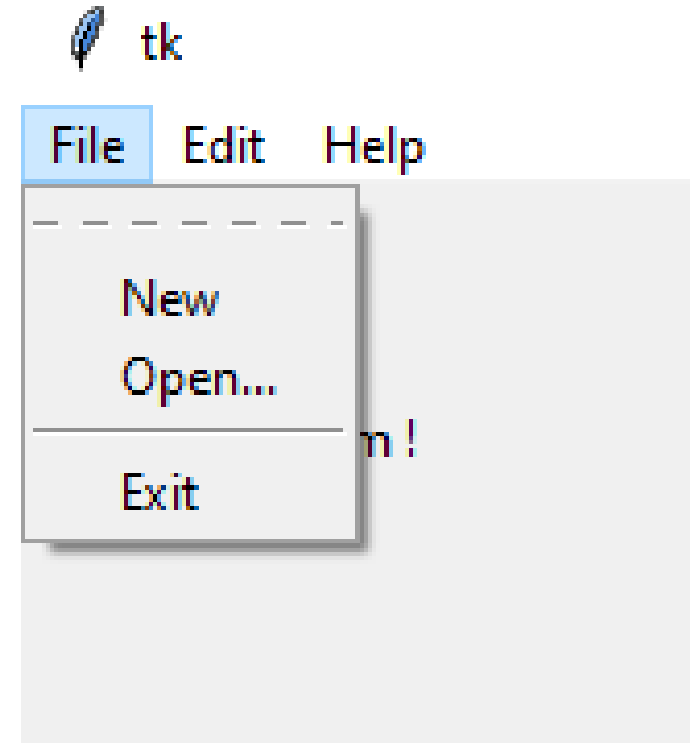
filemenu = Menu(M)
M.add_cascade(label='File', menu=filemenu)
filemenu.add_command(label='New')
filemenu.add_command(label='Open...')
filemenu.add_separator()
filemenu.add_command(label='Exit', command=root.quit)

helpmenu = Menu(M)
M.add_cascade(label='Help', menu=helpmenu)
helpmenu.add_command(label='About')
mainloop()
```



Menu

```
from tkinter import *  
  
def show( menu ):  
    if menu=='E_Text_B':  L1.config( fg='blue')  
    if menu=='E_Text_R':  L1.config( fg='red')  
    if menu=='E_Back_R':  L1.config( bg='red')  
    if menu=='E_Back_Y':  L1.config( bg='yellow')  
    return  
  
root = Tk()  
root.geometry("700x400")  
menu = Menu(root)  
root.config(menu=menu)  
  
filemenu = Menu(menu)  
menu.add_cascade(label='File', menu=filemenu)  
filemenu.add_command(label='New')  
filemenu.add_command(label='Open...')  
filemenu.add_separator()  
filemenu.add_command(label='Exit', command=root.quit)
```



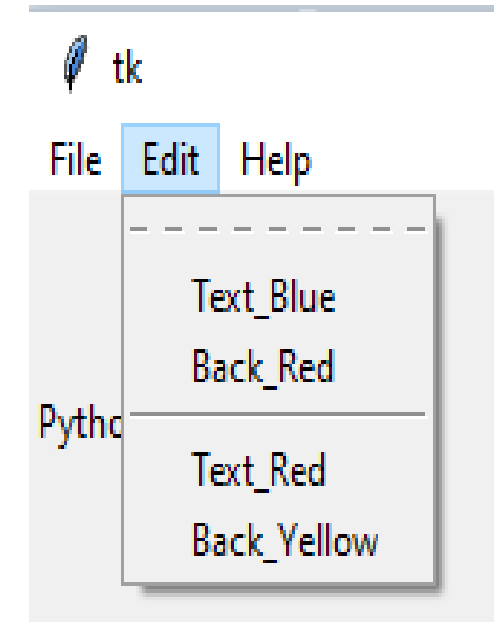
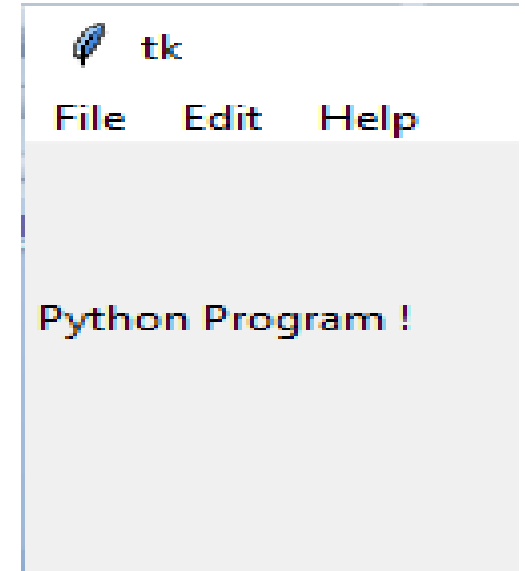
Menu

```
editmenu = Menu(menu)
menu.add_cascade(label='Edit', menu=editmenu)
editmenu.add_command(label='Text_Blue', command=lambda: show('E_Text_B'))
editmenu.add_command(label='Back_Red', command=lambda: show('E_Back_R'))
editmenu.add_separator()
editmenu.add_command(label='Text_Red', command=lambda: show('E_Text_R'))
editmenu.add_command(label='Back_Yellow', command=lambda: show('E_Back_Y'))
```

```
helpmenu = Menu(menu)
menu.add_cascade(label='Help', menu=helpmenu)
helpmenu.add_command(label='About')
```

```
Frame(root,bd=5, height=50).grid(row=1)
L1=Label(root, text="Python Program !")
L1.grid(row=4)
root.mainloop()
```

```
mainloop()
```



Text

To edit a multi-line text and format the way it has to be displayed.

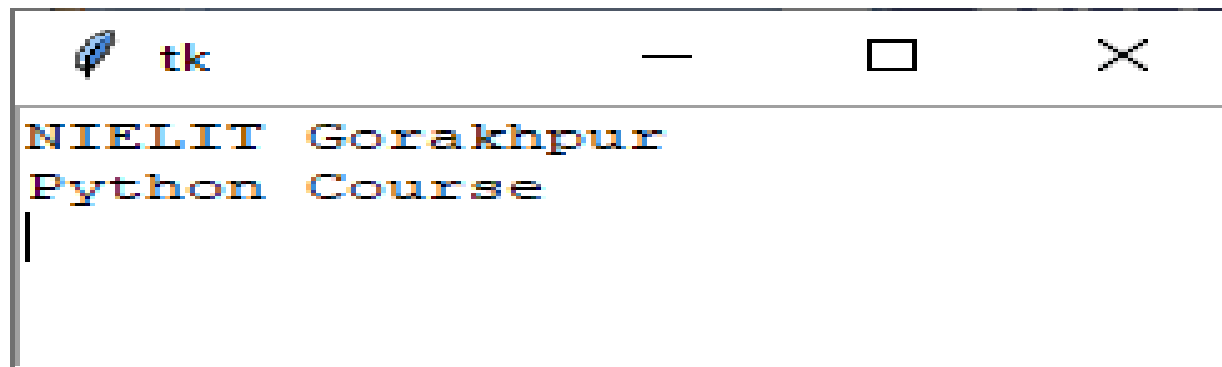
The general syntax is:

`w =Text(master, option=value)`

- **highlightcolor**: To set the color of the focus highlight when widget has to be focused.
- **insertbackground**: To set the background of the widget.
- **bg**: to set the normal background color.
- **font**: to set the font on the button label.
- **image**: to set the image on the widget.
- **width**: to set the width of the widget.
- **height**: to set the height of the widget.

Text

```
from tkinter import *  
root = Tk()  
T = Text(root, height=2, width=30)  
T.pack()  
T.insert(END, 'NIELIT Gorakhpur\nPython Course\n')  
mainloop()
```



Events

Mouse Events

<Enter>—The mouse pointer entered the widget.

<Leave>—The mouse pointer left the widget.

<Button-1>,<ButtonPress-1>, or<1>—A mouse button is pressed over the widget.<B1-Motion>—The mouse is moved, with mouse button 1 being held down.

<ButtonRelease-1>—Button 1 was released.

<Double-Button-1>—Button 1 was double-clicked.

Events

Keyboard Events

<Key>—The user has pressed any key.

<Control-Up>— The user pressed the Control key, while pressing the Up arrow.

<Return>— The user pressed the Enter key.

<Escape>— The user pressed the Esc key.

The same concept can also be applied for all the other special keys found in the keyboard, including:

F1, F2, F3, F4, F5, F6, F7, F8, F9, F10, F11, F12, Num_Lock, Scroll_Lock, Caps_Lock, Print, Insert, Delete, Pause, Prior (Page Up), Next (Page Down), BackSpace, Tab, Cancel (Break), Control _L (any Control key), Alt _L (any Alt key), Shift _L (any Shift key), End, Home, Up, Down, Left, and Right

Events and Binds

A Tkinter application runs most of its time inside an event loop, which is entered via the `mainloop` method. It waiting for events to happen. Events can be key presses or mouse operations by the user.

Tkinter provides a mechanism to let the programmer deal with events. For each widget, it's possible to bind Python functions and methods to an event.

`widget.bind(event, handler)`

If the defined event occurs in the widget, the "handler" function is called with an event object. describing the event.

Events and Binds

```
from tkinter import *  
def hello(event):  
    print("Single Click, Button-1")  
def quit(event):  
    print("Double Click, so let's stop")  
    import sys; sys.exit()
```

```
widget = Button(None, text='Mouse Clicks')  
widget.pack()  
widget.bind('<Button-1>', hello)  
widget.bind('<Double-1>', quit)  
widget.mainloop()
```

Data Science using Python Programming

Unit-19: < Data Science Concept>

Contents to be covered

1. Data Science Introduction
2. Data Science Application
3. Data Science –Python Libraries
4. Data Science Step Involved
5. Data Preprocessing

Data Science

- Data Science deals with the extraction of knowledge from large set of data, that may be structured or unstructured.
- Large volume of data source are
 - Scientific Experiments
 - Internet of Things
 - E-commerce
 - Financial Transactions,
 - Bank/credit transaction
 - Social Network

Big Data

- Big Data are large and complex data sets which are difficult for traditional methods to store , access and analyze.
- However, lot of potential values are hidden in this data. This makes it valuable for organizations.
- Big Data technologies and approaches are used to drive values out of data in a efficient ways.
- Types of Data
 - Structured – Fields / Tables / columns eg- Spreadsheet, RDBMS
 - Semi-Structured – Tags to separate items eg – XML, HTML
 - Unstructured – No field/attributes. eg. email , articles

Data Science Fields

- **Artificial Intelligence -**

Artificial intelligence, or AI, is technology **that enables computers and machines to simulate human intelligence and problem-solving** capabilities. such as perceiving, reasoning, learning, interacting

- **Machine Learning** – Using algorithm to train and predict on data.

- **Deep Learning** – It is a type of **machine learning** that has networks capable of **learning** unsupervised data that is unstructured or unlabeled. Also known as **deep neural learning** or **deep neural network**.

Data Science - Python Libraries Support

- **NumPy**- It is a library for adding support for large, multi dimensional arrays and matrices, along with a large collection of mathematical functions to operate on these arrays.
- **Scikits** – It is machine learning library that has various classification, regression and clustering algorithms including support vector machines, random forest, gradient boosting and k-means and is designed to interoperate with the python numerical and scientific libraries Numpy and SciPy.
- **Pandas**– It is a software library for data manipulation and analysis. It offers data structures and operations for manipulating numerical tables and time series.

Data Science - Python Libraries Support

- **Matplotlib**- It is a library for plotting various type of charts for data visualisation.
- **TensorFlow** – It is a library for machine learning applications such as neural networks.
- **Keras**– It is a neural network library. It works on top of TensorFlow and focus on enabling fast experimentation.

Data Science - Steps

1. **Data Gathering** - Put the necessary systems in place to gather data. Data collected may be fragmented and scattered.
2. **Data Preparation** - Clean and format the messy data sets, to make it usable for analysis. This includes managing missing values, error in data collection, data formatting, normalization.
3. **Exploration** - Understand the structure of data, by using the clustering algorithms and visualizing methods - scatter plot, bar graphs.
4. **Model Building** - Explore the variety of models (Random Forest, SVM, Neural Network, K-nearest Neighbors etc) on the data sets and identify and develop the model which fit for the problem.
5. **Model Validation** – Analyze the prediction accuracy of the model based on evaluation matrices.
6. **Model Deployment** – Deploy the model. Tweak and improve it based on feedback.

Data Preparation - Steps

- Selecting data objects and attributes for the analysis.
- Creating/changing the attributes.
- Normally following steps are taken before it is ready to be used for machine learning models
 - Importing the libraries
 - Importing the Dataset
 - Handling of Missing Data
 - Handling of Categorical Data
 - Splitting the dataset into training and testing datasets
 - Feature Scaling

Data Preparation

Dataset - Filename – property_data.csv

	PID	ST_NUM	ST_NAME	OWN_OCCUPIED	NUM_BEDROOMS	NUM_BATH	SQ_FT
0	100001000.0	104.0	PUTNAM	Y	3	1	1000
1	100002000.0	197.0	LEXINGTON	N	3	1.5	--
2	100003000.0	NaN	LEXINGTON	N	NaN	1	850
3	100004000.0	201.0	BERKELEY	12	1	NaN	700
4	NaN	203.0	BERKELEY	Y	3	2	1600
5	100006000.0	207.0	BERKELEY	Y	NaN	1	800
6	100007000.0	NaN	WASHINGTON	NaN	2	HURLEY	950
7	100008000.0	213.0	TREMONT	Y	--	1	NaN
8	100009000.0	215.0	TREMONT	Y	na	2	1800

Data Preparation - Steps

Find out the following questions:

- What are the features?
- What are the expected types (int, float, string, boolean)?
- Standard Missing data (values that Pandas can detect)?
- Non Standard Missing Data (can't easily detect with Pandas)?

Data Preparation – Loading Dataset

```
# Importing libraries
import pandas as pd
import numpy as np
```

```
# Read csv file into a pandas dataframe
df = pd.read_csv("d:\data\property_data.csv")
```

```
# Show the first 5 rows of the dataframe
print(df.head( ))
```

	PID	ST_NUM	ST_NAME	OWN_OCCUPIED	NUM_BEDROOMS	NUM_BATH	SQ_FT
0	100001000.0	104.0	PUTNAM	Y	3	1	1000
1	100002000.0	197.0	LEXINGTON	N	3	1.5	--
2	100003000.0	NaN	LEXINGTON	N	NaN	1	850
3	100004000.0	201.0	BERKELEY	12	1	NaN	700
4	NaN	203.0	BERKELEY	Y	3	2	1600

Data Preparation - Dataset Features

Structure of the Dataframe

```
print(df.info ( ))
```

Column of the Dataframe

```
print(df.columns)
```

Row and Column of the Dataframe

```
print(df.columns)
```

what are my features?

ST_NUM : Street number

ST_NAME : Street name

OWN_OCCUPIED : Is the residence owner occupied

NUM_BEDROOMS : Number of bedrooms

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype
---  -
0   PID             8 non-null     float64
1   ST_NUM          7 non-null     float64
2   ST_NAME         9 non-null     object
3   OWN_OCCUPIED    8 non-null     object
4   NUM_BEDROOMS    7 non-null     object
5   NUM_BATH        8 non-null     object
6   SQ_FT           8 non-null     object
dtypes: float64(2), object(5)
memory usage: 388.0+ bytes
None
```

Data Preparation - Expected Data Types

what are the expected types?

ST_NUM : float or int... some sort of numeric type

ST_NAME : string

OWN_OCCUPIED : string... Y ("Yes") or N ("No")

NUM_BEDROOMS : float or int, a numeric type

	PID	ST_NUM	ST_NAME	OWN_OCCUPIED	NUM_BEDROOMS	NUM_BATH	SQ_FT
0	100001000.0	104.0	PUTNAM	Y	3	1	1000
1	100002000.0	197.0	LEXINGTON	N	3	1.5	--
2	100003000.0	NaN	LEXINGTON	N	NaN	1	850
3	100004000.0	201.0	BERKELEY	12	1	NaN	700
4	NaN	203.0	BERKELEY	Y	3	2	1600

Data Preparation - Standard Missing Data

Standard Missing data (values that Pandas can detect)?

Finding Missing data in ST_NUM Column

```
print(df['ST_NUM'])
```

```
print(df['ST_NUM'].isnull( ))
```

Standard Missing Data

none

NaN

```
0      104.0
1      197.0
2         NaN
3      201.0
4      203.0
5      207.0
6         NaN
7      213.0
8      215.0
Name: ST_NUM,
```

```
0      False
1      False
2       True
3      False
4      False
5      False
6       True
7      False
8      False
Name: ST_NUM,
```

Data Preparation - Non Standard Missing Data

Non Standard Missing data (values that Pandas can detect)?

Finding Missing data in ST_NUM Column

```
print(df['NUM_BEDROOMS'])
```

```
print(df['NUM_BEDROOMS'].isnull( ))
```

```
0      3
1      3
2     NaN
3      1
4      3
5     NaN
6      2
7     --
8     na
Name: NUM_BEDROOMS,
```

Non standard Missing Data

n/a

NA

—

na

```
0     False
1     False
2      True
3     False
4     False
5      True
6     False
7     False
8     False
Name: NUM_BEDROOMS,
```

Data Preparation - Non Standard Missing Data

Non Standard Missing data (Pandas can detect now)

Importing libraries

```
import pandas as pd  
import numpy as np
```

Making a list of missing value types

```
missing_values = [ "n/a", "na", "--" ]  
df = pd.read_csv("d:\data\property_data.csv", na_values = missing_values)
```

Finding Missing data in NUM_BEDROOMS Column

```
print(df['NUM_BEDROOMS'])  
print(df['NUM_BEDROOMS'].isnull( ))
```

```
0      3.0  
1      3.0  
2      NaN  
3      1.0  
4      3.0  
5      NaN  
6      2.0  
7      NaN  
8      NaN  
Name: NUM_BEDROOMS,
```

```
0      False  
1      False  
2       True  
3      False  
4      False  
5       True  
6      False  
7       True  
8       True  
Name: NUM_BEDROOMS,
```

Data Preparation - Unexpected Missing Values

Unexpected Missing Values

1. If Column expected to be a string, but having a numeric type data, is technically a missing value.
2. **Owner Occupied** should clearly be a string (Y or N), so this numeric type should be a missing value.

Finding Missing data in OWN_OCCUPIED Column

```
print(df['OWN_OCCUPIED'])
```

```
print("\n")
```

```
print(df['OWN_OCCUPIED'].isnull())
```

```
0      Y
1      N
2      N
3     12
4      Y
5      Y
6    NaN
7      Y
8      Y
Name: OWN_OCCUPIED,
```

```
0    False
1    False
2    False
3    False
4    False
5    False
6     True
7    False
8    False
Name: OWN_OCCUPIED,
```

Data Preparation - Unexpected Missing Values

Unexpected Missing Values

Convert the unexpected value to NaN

```
cnt=0
for row in df['OWN_OCCUPIED']:
    try:
        int(row)
        df.loc[cnt, 'OWN_OCCUPIED']=np.nan
    except ValueError:
        pass
    cnt+=1
```

```
0      Y
1      N
2      N
3     NaN
4      Y
5      Y
6     NaN
7      Y
8      Y
Name: OWN_OCCUPIED,
```

```
0     False
1     False
2     False
3      True
4     False
5     False
6      True
7     False
8     False
Name: OWN_OCCUPIED,
```


Data Preparation - Summarizing Missing Value

Summarizing Missing Values

Total missing values for each Column

```
print( df.isnull( ).sum( ))
```

```
PID          1
ST_NUM       2
ST_NAME      0
OWN_OCCUPIED 2
NUM_BEDROOMS 4
NUM_BATH      1
SQ_FT        2
dtype: int64
```

Any missing values?

```
print(df.isnull( ).values.any( ))
```

```
-----
True
-----
```

#Total number of missing values

```
print(df.isnull( ).sum( ).sum( ))
```

```
12
```

Data Preparation - Replace Missing Value

Imputation – To replace the missing value with mean/median/mode of all the value for that column.

Replacing Missing Values

Replace missing values with a number

```
df['ST_NUM'].fillna(125, inplace=True)
```

#Common to replace missing values is using a median

```
median = df['NUM_BEDROOMS'].median( )
```

```
df['NUM_BEDROOMS'].fillna(median, inplace=True)
```

Total missing values for each feature

```
print( df.isnull( ).sum( ))
```

```
PID          1
ST_NUM       2
ST_NAME      0
OWN_OCCUPIED 2
NUM_BEDROOMS 4
NUM_BATH     1
SQ_FT        2
dtype: int64
```

```
PID          1
ST_NUM       0
ST_NAME      0
OWN_OCCUPIED 2
NUM_BEDROOMS 0
NUM_BATH     1
SQ_FT        2
dtype: int64
```

Data Preparation - Replacing Values

Replacing Missing Values

Replace using median

```
median = df['SQ_FT'].median()
```

```
df['SQ_FT'].fillna(median, inplace=True)
```

Replace using median

```
df['NUM_BATH'].loc[df['NUM_BATH']=='HURLEY']=1
```

```
median = df['NUM_BATH'].median()
```

```
df['NUM_BATH'].fillna(median, inplace=True)
```

```
df['OWN_OCCUPIED'].fillna('Y', inplace=True)
```

```
df['PID'].fillna(100005000, inplace=True)
```

Total missing values for each feature

```
print( df.isnull( ).sum( ))
```

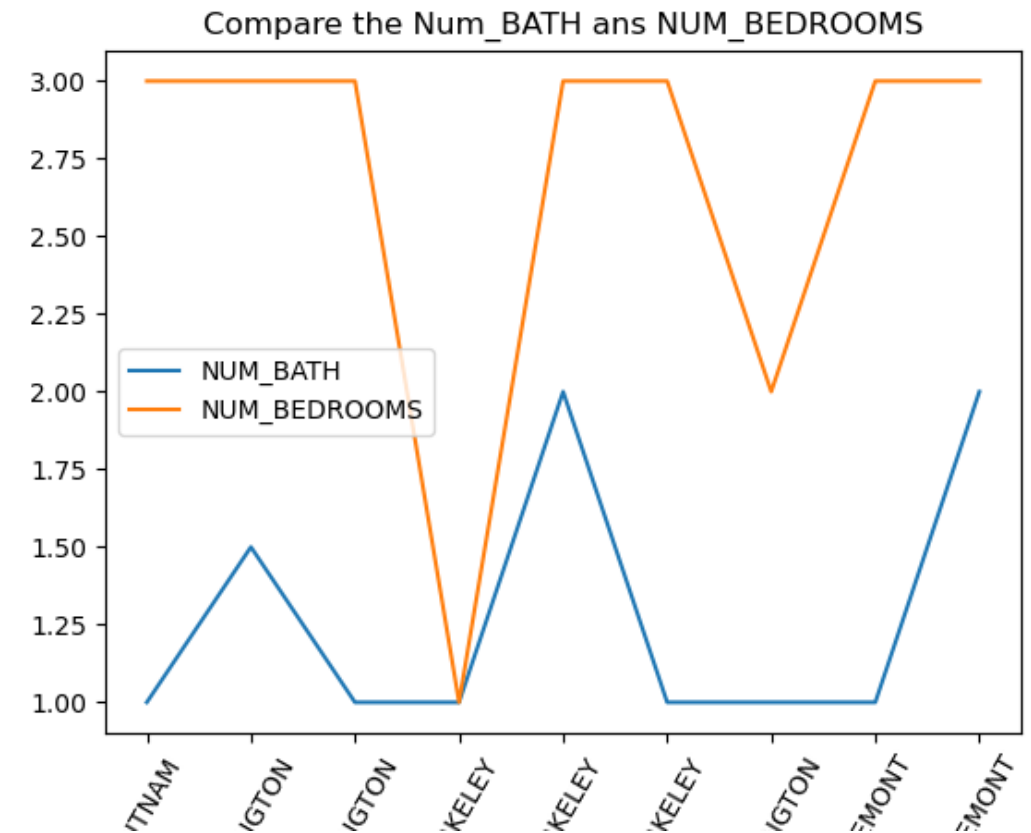
```
PID          0
ST_NUM       0
ST_NAME      0
OWN_OCCUPIED 0
NUM_BEDROOMS 0
NUM_BATH     0
SQ_FT        0
dtype: int64
```

Data Preparation - Visualisation in Pandas

Plotting Graph in Pandas

```
df['NUM_BATH']=df['NUM_BATH'].astype('float',copy=False)  
print(df.info( ))
```

```
ax=plt.gca( )  
plt.title("Compare the Num_BATH ans NUM_BEDROOMS")  
df.plot( x='ST_NAME' , y='NUM_BATH', kind='line',ax=ax)  
df.plot( x='ST_NAME' , y='NUM_BEDROOMS', kind='line', ax=ax)  
plt.xticks(rotation=90)  
plt.show( )
```



References

- www.w3schools.com
- www.tutorialpoint.com
- Python Programming – Pearson – Taneja Kumar

Thank You
Any Query

National Institute of Electronics & Information Technology

Gorakhpur Center

Ministry of Electronics & Information Technology (MeitY), Government of India

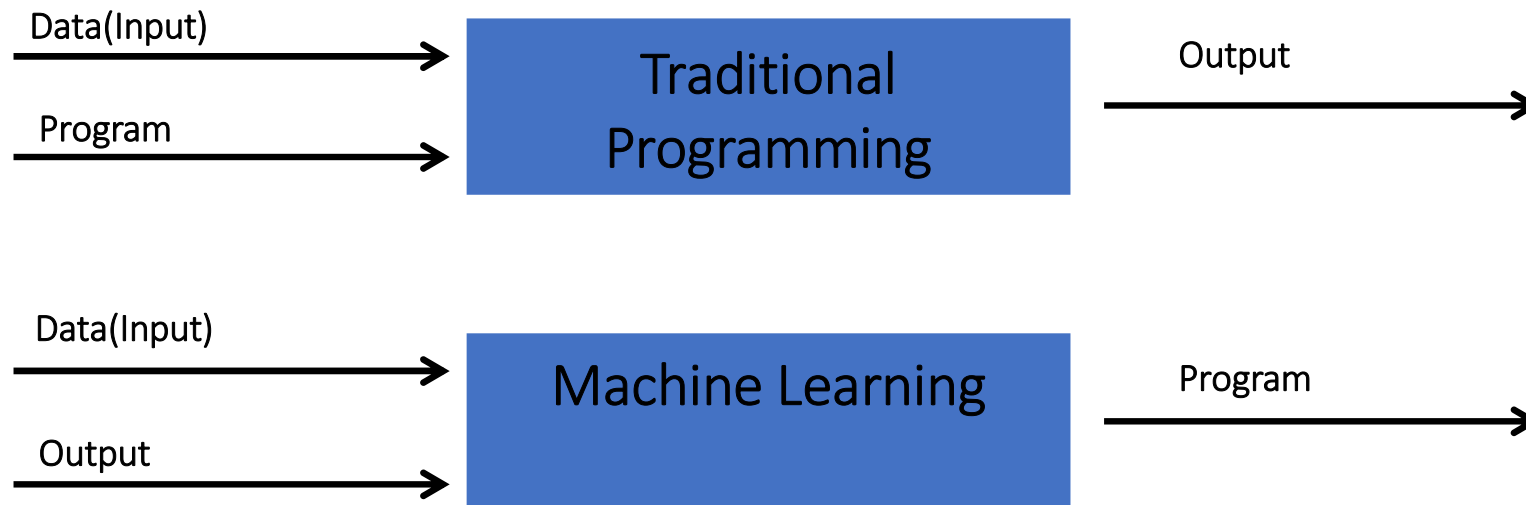
**Data Science using
Python Programming
Unit-20: < Machine Learning>**

Contents to be covered

1. Machine Learning
2. Machine Learning Classification
3. Machine Learning Working
4. Machine Learning Application

Machine Learning

- **Traditional Programming** : We feed in DATA (Input) + PROGRAM (logic), run it on machine and get output.
- **Machine Learning** : We feed in DATA(Input) + Output, run it on machine during training and the machine creates its own program(logic), which can be evaluated while testing.



Machine Learning - Classification

- Machine learning implementations are classified into three major categories, depending on the nature of the learning “signal” or “response” available to a learning system which are as follows:-
 1. Supervised learning
 2. Unsupervised learning
 3. Reinforcement learning

Machine Learning - Classification

1. Supervised learning –

- When an algorithm learns from example data and associated target responses that can consist of numeric values or string labels, such as classes or tags, in order to later predict the correct response when posed with new examples.
- The teacher provides good examples for the student to memorize, and the student then derives general rules from these specific examples.

2. Unsupervised learning

- Whereas when an algorithm learns from plain examples without any associated response, leaving to the algorithm to determine the data patterns on its own.
- It resembles the methods humans use to figure out that certain objects or events are from the same class, by observing the degree of similarity between objects.

3. Reinforcement learning –

- When you present the algorithm with examples that lack labels, as in unsupervised learning. However, you can provide an example with positive or negative feedback according to the solution. In the human world, it is just like learning by trial and error.
- Errors help you learn because they have a penalty added (cost, loss of time, regret, pain, and so on), teaching you that a certain course of action is less likely to succeed than others.

Supervised vs Unsupervised learning

Example 1:

- You get a bunch of photos **with information about what is on them**.
- Then you train a model to recognize new photos.

Example 2:

- You have a bunch of molecules and information about which are drugs.
- Then you train a model to answer whether a new molecule is also a drug.

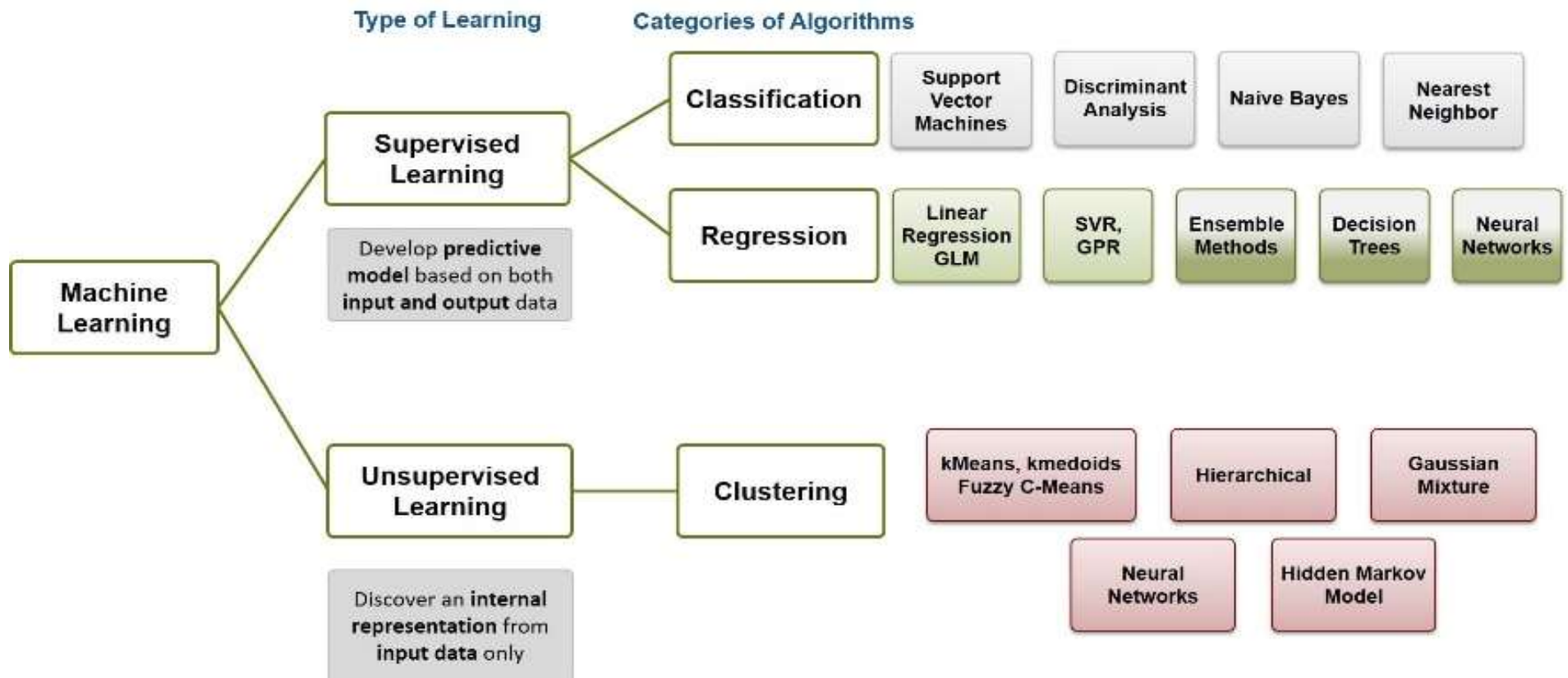
Example 1:

- You have a bunch of photos of 6 people
- **No information about who is on which one.**
- You want to **divide** this dataset into 6 piles, each with the photos of one individual.

Example 2:

- You have molecules, part of them are drugs and part are not.
- But you do not know which are which.
- You want the algorithm to discover the drugs.

Category of Algorithms



Regression/classification Example

User ID	Gender	Age	Salary	Purchased	Temperature	Pressure	Relative Humidity	Wind Direction	Wind Speed
15624510	Male	19	19000	0	10.69261758	986.882019	54.19337313	195.7150879	3.278597116
15810944	Male	35	20000	1	13.59184184	987.8729248	48.0648859	189.2951202	2.909167767
15668575	Female	26	43000	0	17.70494885	988.1119385	39.11965597	192.9273834	2.973036289
15603246	Female	27	57000	0	20.95430404	987.8500366	30.66273218	202.0752869	2.965289593
15804002	Male	19	76000	1	22.9278274	987.2833862	26.06723423	210.6589203	2.798230886
15728773	Male	27	58000	1	24.04233986	986.2907104	23.46918024	221.1188507	2.627005816
15598044	Female	27	84000	0	24.41475295	985.2338867	22.25082295	233.7911987	2.448749781
15694829	Female	32	150000	1	23.93361956	984.8914795	22.35178837	244.3504333	2.454271793
15600575	Male	25	33000	1	22.68800023	984.8461304	23.7538641	253.0864716	2.418341875
15727311	Female	35	65000	0	20.56425726	984.8380737	27.07867944	264.5071106	2.318677425
15570769	Female	26	80000	1	17.76400389	985.4262085	33.54900114	280.7827454	2.343950987
15606274	Female	26	52000	0	11.25680746	988.9386597	53.74139903	68.15406036	1.650191426
15746139	Male	20	86000	1	14.37810685	989.6819458	40.70884681	72.62069702	1.553469896
15704987	Male	32	18000	0	18.45114201	990.2960205	30.85038484	71.70604706	1.005017161
15628972	Male	18	82000	0	22.54895853	989.9562988	22.81738811	44.66042709	0.264133632
15697686	Male	29	80000	0	24.23155922	988.796875	19.74790765	318.3214111	0.329656571
15733883	Male	47	25000	1					

Figure A: CLASSIFICATION

Figure B: REGRESSION

Supervised learning

- **Supervised learning** is when the model is getting trained on a labelled dataset. A **labelled** dataset is one that has both input and output parameters. In this type of learning both training and validation.
- **Figure A:** It is a dataset of a shopping store that is useful in predicting whether a customer will purchase a particular product under consideration or not based on his/ her gender, age, and salary.
Input: Gender, Age, Salary
Output: Purchased i.e. 0 or 1; 1 means yes the customer will purchase and 0 means that the customer won't purchase it.
- **Figure B:** It is a Meteorological dataset that serves the purpose of predicting wind speed based on different parameters.
Input: Dew Point, Temperature, Pressure, Relative Humidity, Wind Direction
Output: Wind Speed

Supervised learning can be further classified into two categories:

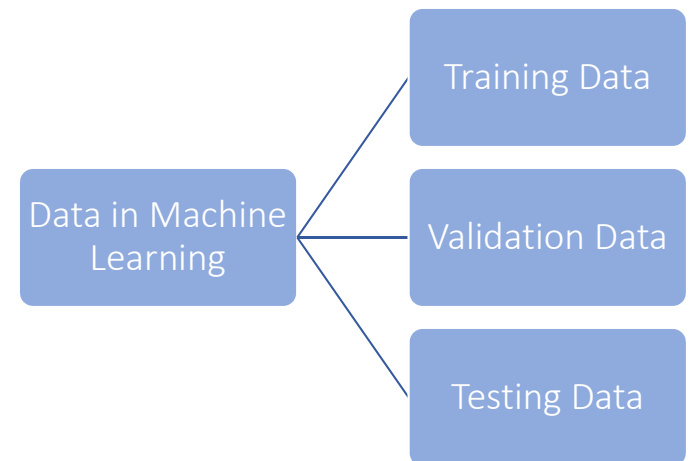
- **Regression:** In regression, the target variable is a continuous value. The goal of regression is to predict the value of the target variable based on the input variables. **Linear regression**, polynomial regression, and decision trees are some of the examples of regression algorithms.
- **Classification:** In classification, the target variable is a categorical value. The goal of classification is to predict the class or category of the target variable based on the input variables. Some examples of classification algorithms include **logistic regression**, decision trees, support vector machines, and neural networks.

Machine Learning - Working

- Divide the input data into
 - training,
 - cross-validation and
 - test sets.
- The ratio between the respective sets must be 6:2:2
- Building models with suitable algorithms and techniques on the training set.
- Testing our conceptualized model with data which was not fed to the model at the time of training and evaluating its performance using metrics such as F1 score, precision and recall.

Machine Learning – Splitting Data

- **Training Data:** The part of data we use to train our model. This is the data which your model actually sees(both input and output) and learn from.
- **Validation Data:** The part of data is used to do a frequent evaluation of model, fit on training dataset along with improving involved parameters (initially set parameters before the model begins learning). This data plays it's part when the model is actually training.
- **Testing Data:** Once model is completely trained, testing data provides the unbiased evaluation. When we feed in the inputs of Testing data, our model will predict some values(without seeing actual output). After prediction, we evaluate our model by comparing it with actual output present in the testing data. This is how we evaluate and see how much our model has learned from the experiences feed in as training data, set at the time of training.



Machine Learning - Examples

1. **Siri**(part of Apple Inc.'s iOS, watchOS, macOS, and tvOS operating systems),
2. **Google Now** (a feature of Google Search offering predictive cards with information and daily updates in the Google app for Android and iOS.),
3. **Cortana** (Cortana is a virtual assistant created by Microsoft for Windows 10) are intelligent digital personal assistants on the platforms like iOS, Android and Windows respectively.
4. They help to find relevant information when requested using voice. For instance, for answering queries like 'What's the temperature today?' or 'What is the way to the nearest supermarket' etc. and the assistant will react by searching information, transferring that information from the phone or sending commands to various other applications.

Regression model

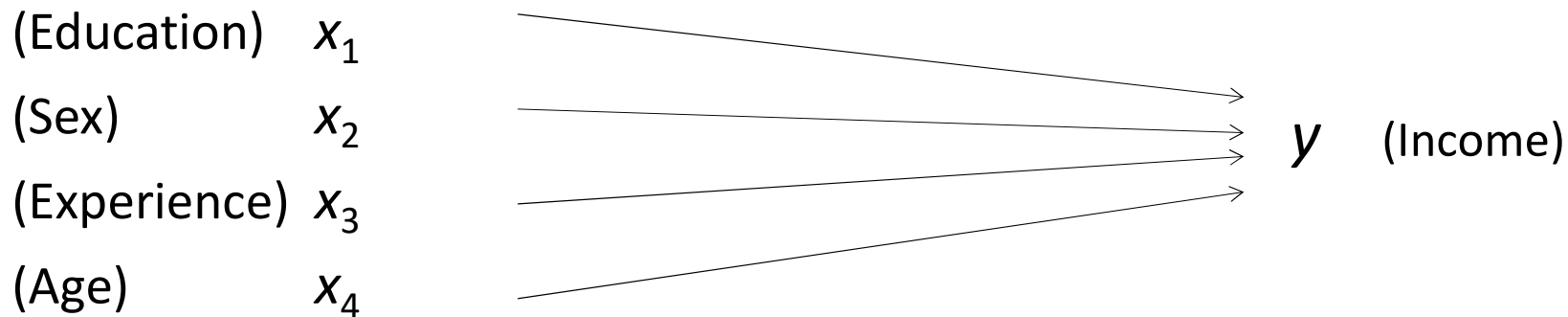
- Relation between variables where changes in some variables may “explain” or possibly “cause” changes in other variables.
- Explanatory variables are termed the **independent** variables and the variables to be explained are termed the **dependent** variables.
- Dependent variable is employment income – independent variables might be hours of work, education, occupation, sex, age, region, years of experience, unionization status, etc.
- Price of a product and quantity produced or sold:
 - Quantity sold affected by price. Dependent variable is quantity of product sold – independent variable is price.
 - Price affected by quantity offered for sale. Dependent variable is price – independent variable is quantity sold.

Bivariate and multivariate models

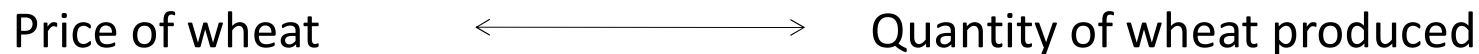
Bivariate or simple regression model



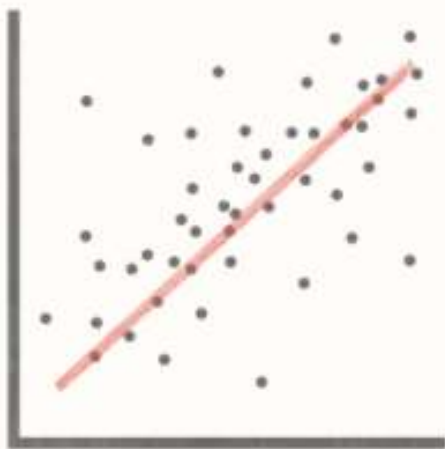
Multivariate or multiple regression model



Model with simultaneous relationship



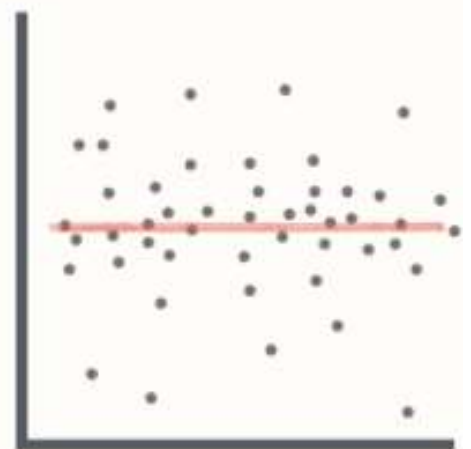
Regression model



Positive Correlation



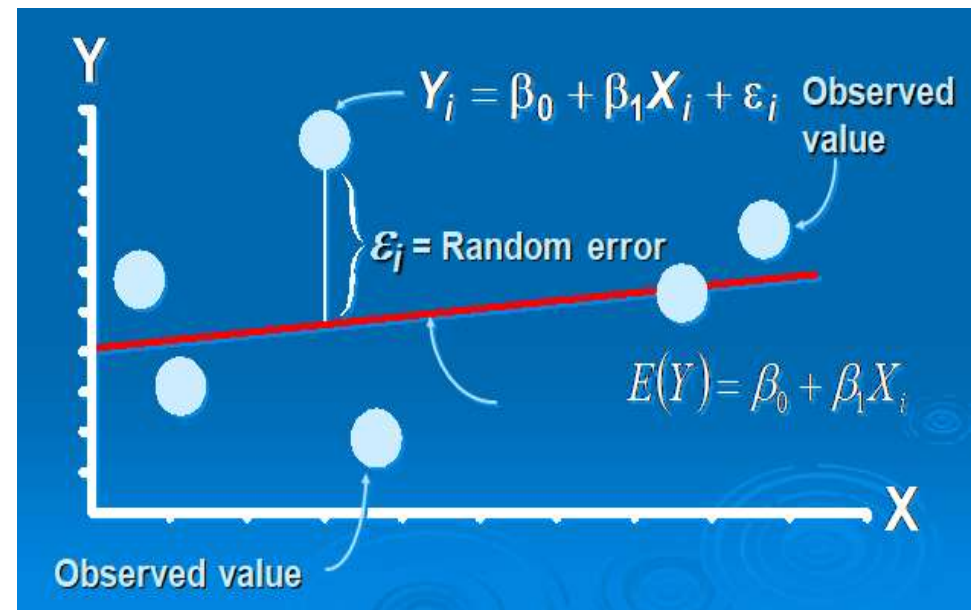
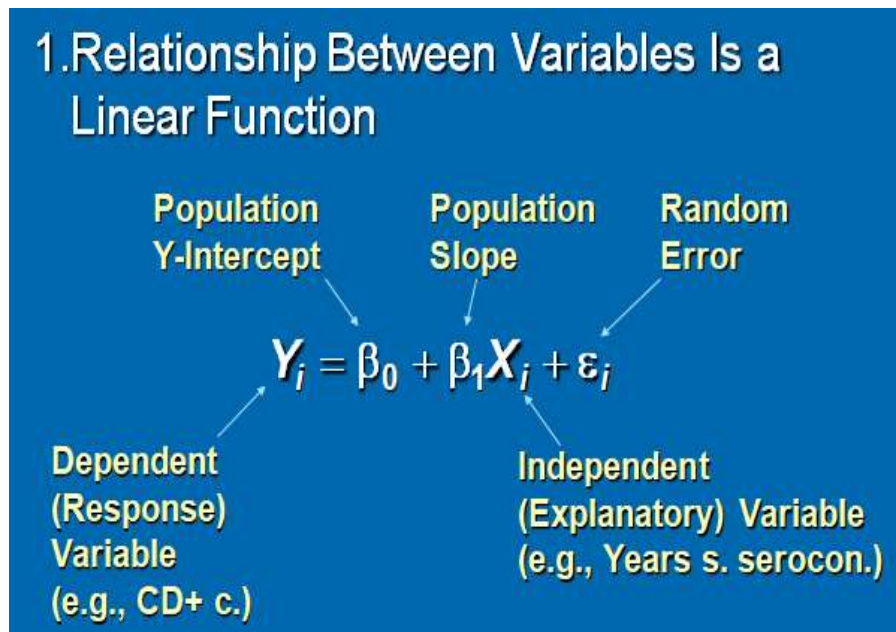
Negative Correlation



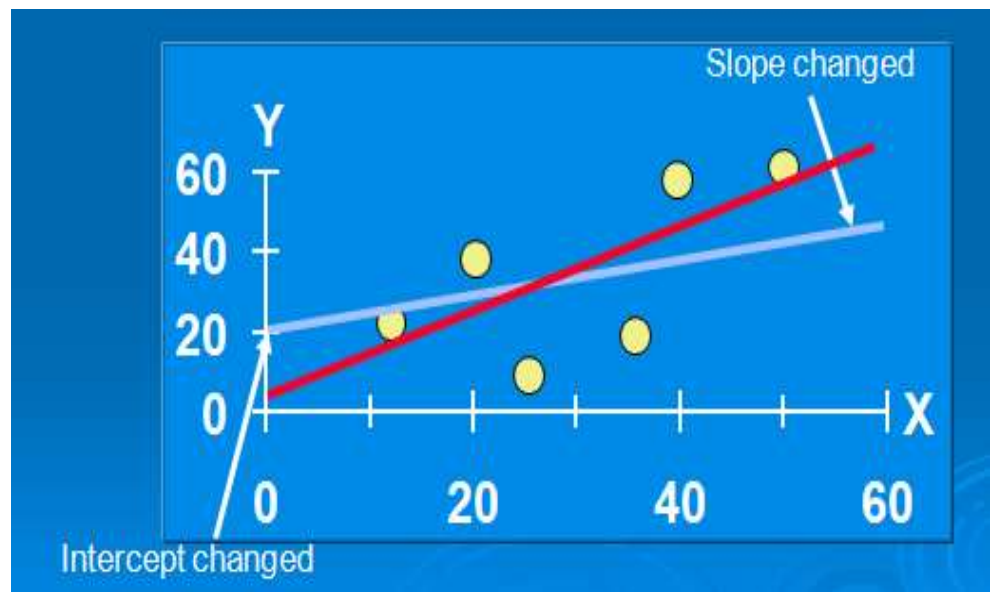
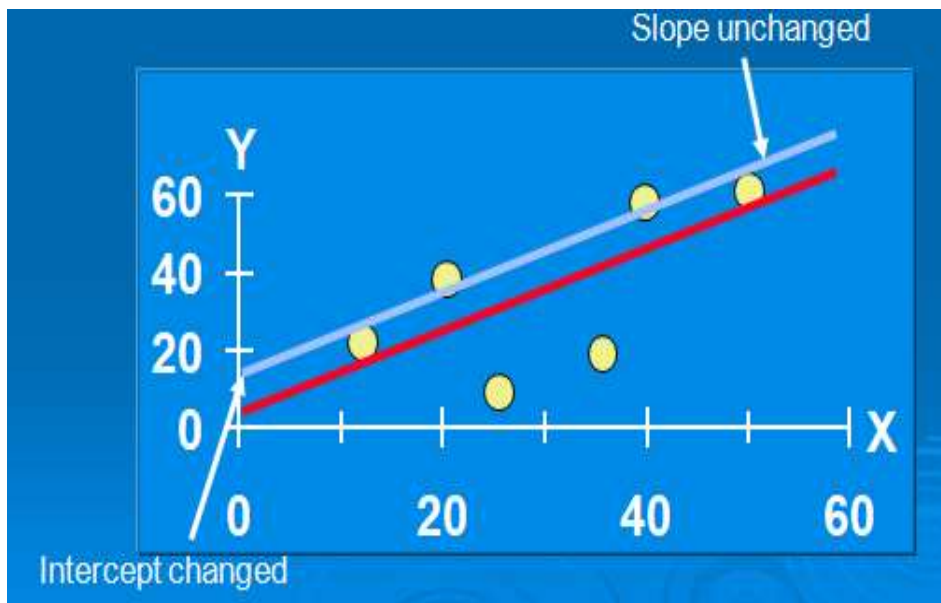
No Correlation

Regression model

1. Relationship Between Variables Is a Linear Function



Regression model



Deprecated sklearn package

```
#deprecated sklearn package, use scikit-learn instead  
#pip uninstall sklearn will actually not uninstall  
#scikit-learn, you can still do import sklearn afterwards  
#so we will install: pip install -U scikit-learn
```

Install the 64bit version of Python 3, for instance
from <https://www.python.org>.

Then run:

```
pip install -U scikit-learn
```

In order to check your installation you can use
`python -m pip show scikit-learn` # to see which version and where
scikit-learn is installed

Machine Learning - Implementation

#Loading the CSV File into DataFrame

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

df=pd.read_csv("d:\\pyprg\\htwgtMale.csv")
print(df.head( ))
print(df.info( ))
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 38 entries, 0 to 37
Data columns (total 2 columns):
#   Column  Non-Null Count  Dtype
---  -
0   Height   38 non-null    int64
1   Weight   38 non-null    int64
dtypes: int64(2)
memory usage: 672.0 bytes
None
```

	Height	Weight
0	105	12
1	110	10
2	119	21
3	120	28
4	132	33

Machine Learning - Implementation

Extracting the required Columns from the DataFrame

#Returns a Numpy representation of the DataFrame.

```
X=df["Height"].values
```

#Only the values in the DataFrame will be returned, the axes labels will be removed.",

```
Y=df["Weight"].values
```

```
print('type(X):', type(X))
```

```
print('X.shape:', X.shape, 'Y.shape:', Y.shape)
```

#Reshaping the array from (1 X N) to (N X 1)

```
X=X.reshape(-1,1)
```

```
print('Changed X.shape:', X.shape)
```

```
type(X): <class 'numpy.ndarray'>
```

```
X.shape: (38,) y.shape: (38,)
```

```
Changed X.shape: (38, 1)
```

Machine Learning - Implementation

Extracting the required Columns from the DataFrame

#Returns a Numpy representation of the DataFrame.

```
X=df["Height"].values
```

#Only the values in the DataFrame will be returned, the axes labels will be removed.",

```
Y=df["Weight"].values
```

```
print('type(X):', type(X))
```

```
print('X.shape:', X.shape, 'Y.shape:', Y.shape)
```

#Reshaping the array from (1 X N) to (N X 1)

```
X=X.reshape(-1,1)
```

```
Y=Y.reshape(-1,1)
```

```
print('Changed X.shape:', X.shape)
```

```
type(X): <class 'numpy.ndarray'>
```

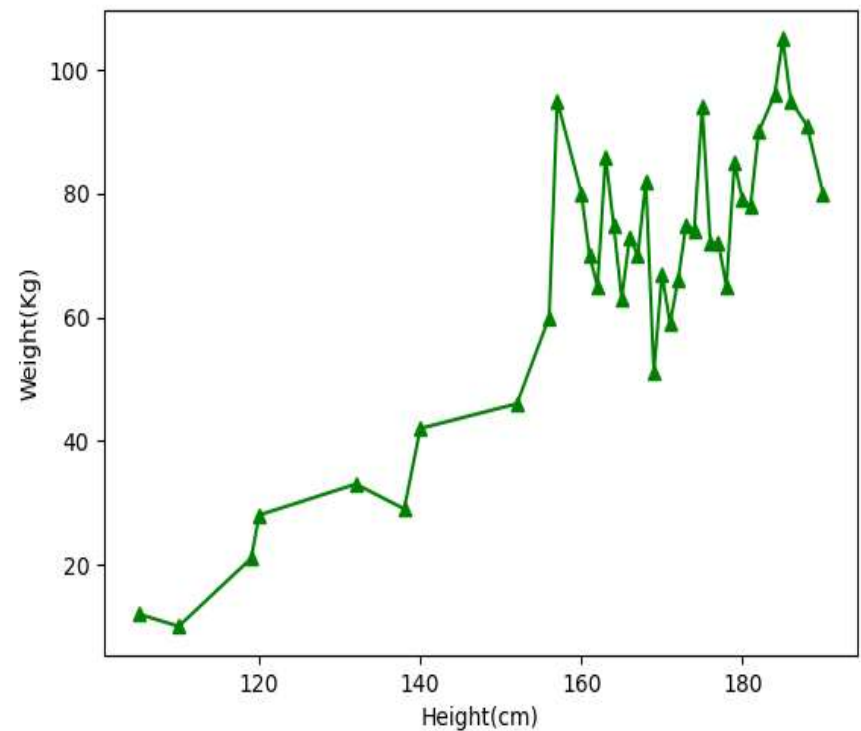
```
X.shape: (38,) y.shape: (38,)
```

```
Changed X.shape: (38, 1)
```

Machine Learning - Implementation

Plotting the Trend in the Data

```
#Plot the Trend of Original Data  
plt.plot(X, Y, c='g', marker="^", label='Original Data')  
plt.xlabel('Height(cm)')  
plt.ylabel('Weight(Kg)')  
plt.show( )
```



Machine Learning - Implementation

Splitting the Data for Training & Testing

```
#from sklearn.model_selection import train_test_split
```

```
#Split the Data into Training & Validation Data
```

```
X_train, X_test, Y_train, Y_test = train_test_split (X, Y, random_state=0, test_size=.3)
```

```
print('type(X_train):', type(X_train))
```

```
print('Shapes of X_train, X_test, Y_train, Y_test are:')
```

```
print(X_train.shape, X_test.shape, Y_train.shape, Y_test.shape )
```

Data Original Shape

X.shape: (38,) y.shape: (38,)

Shapes of X_train, X_test, Y_train, Y_test are:

(26, 1) (12, 1) (26,) (12,)

Machine Learning - Implementation

Training the Model - LinearRegression

```
#from sklearn.linear_model import LinearRegression
```

```
lr=LinearRegression()
```

```
#Training of Data
```

```
lr.fit(X_train, Y_train )
```

```
#Prediction
```

```
Y_pred=lr.predict(X_test)
```

```
print('Shape of y_pred is:')
```

```
print(Y_pred.shape)
```

```
print("Test Val<--Vs-->Predicted Val")
```

```
for i in range(len(Y_test)):
```

```
    print(Y_test[i],"<--Vs-->",Y_pred[i])
```

```
Test Val<--Vs-->Predicted Val
[66] <--Vs--> [74.83920048]
[85] <--Vs--> [81.63826886]
[73] <--Vs--> [69.01142759]
[63] <--Vs--> [68.0401321]
[70] <--Vs--> [64.15495017]
[67] <--Vs--> [72.89660952]
[72] <--Vs--> [78.72438241]
[80] <--Vs--> [63.18365469]
[94] <--Vs--> [77.75308693]
[21] <--Vs--> [23.36053991]
[78] <--Vs--> [83.58085982]
[96] <--Vs--> [86.49474627]
```

Machine Learning - Implementation

Plotting the Model - LinearRegression

#We can plot X vs y using the equation of the form $y=m*x+c$

#lr.coef_ gives m and lr.intercept_ gives

$m=lr.coef_$

$c=lr.intercept_$

`print('Slope =',m,'y-intercept=',c)`

$y_{Calc}= m*X+c$

`plt.plot(X,yCalc)`

`plt.scatter(X_test, Y_pred, c='r', marker="s", label='Prediction')`

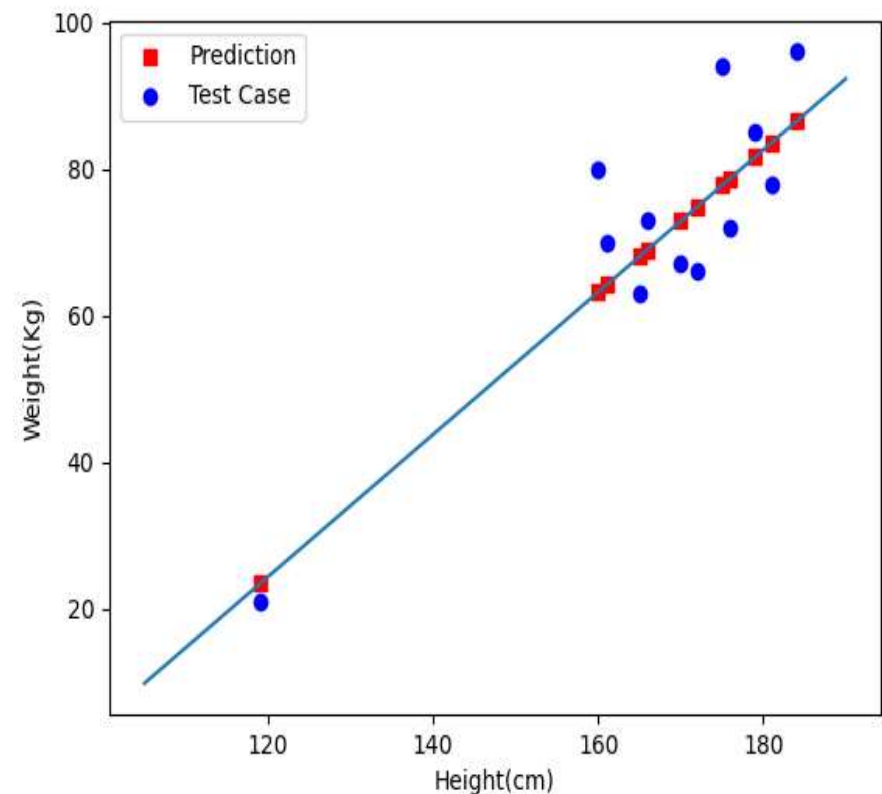
`plt.scatter(X_test, Y_test, c='b', marker="o", label='Test Case')`

`plt.legend(loc='upper left')`

`plt.xlabel('Height(cm)')`

`plt.ylabel('Weight(Kg)')`

`plt.show( )`



Machine Learning - Implementation

Accuracy of the Model - LinearRegression

#R-squared is a statistical measure of how close the data are to the fitted regression line.

```
print("R-Square Value on Training Data: ", lr.score(X_train, Y_train))
```

```
print("R-Square Valueon Testing Data : ", lr.score(X_test, Y_test))
```

R-Square Value on Training Data: 0.7858953710613918

R-Square Valueon Testing Data : 0.7746542312358535

References

- www.w3schools.com
- www.tutorialpoint.com
- Python Programming – Pearson – Taneja Kumar

Thank You
Any Query

R J SHUKLA



Artificial Intelligence using python programming

Day – 29

Database-Sqlite

SQLite

SQLite is embedded relational database management system. It is self-contained, serverless, zero configuration and transactional SQL database engine. SQLite is free to use for any purpose commercial or private. In other words, "SQLite is an open source, zero-configuration, self-contained, stand alone, transaction relational database engine designed to be embedded into an application".

SQLite is different from other SQL databases because unlike most other SQL databases, SQLite does not have a separate server process. It reads and writes directly to ordinary disk files. A complete SQL database with multiple tables, indices, triggers, and views, is contained in a single disk file.

SQLite History

SQLite was designed originally on August 2000. It is named SQLite because it is very light weight (less than 500Kb size) unlike other database management systems like SQL Server or Oracle. In year 2000 SQLite was designed by D. Richard Hipp for the purpose of no administration required for operating a program.

SQLite Installation

SQLite is known for its zero configuration which means no complex setup or administration is required. Let's see how to install SQLite on your system.

Go to SQLite official website download page <http://www.sqlite.org/download.html> And download precompiled binaries from Windows section

SQLite Features/ Why to use SQLite

Following is a list of features which makes SQLite popular among other lightweight databases:

SQLite is totally free: SQLite is open-source. So, no license is required to work with it.

SQLite is serverless: SQLite doesn't require a different server process or system to operate.

SQLite is very flexible: It facilitates you to work on multiple databases on the same session on the same time.

Configuration Not Required: SQLite doesn't require configuration. No setup or administration required.

SQLite is a cross-platform DBMS: You don't need a large range of different platforms like Windows, Mac OS, Linux, and Unix. It can also be used on a lot of embedded operating systems like Symbian, and Windows CE.

Storing data is easy: SQLite provides an efficient way to store data.

Variable length of columns: The length of the columns is variable and is not fixed. It facilitates you to allocate only the space a field needs. For example, if you have a varchar(200) column, and you put a 10 characters' length value on it, then SQLite will allocate only 10 characters' space for that value not the whole 200 space.

Provide large number of API's: SQLite provides API for a large range of programming languages. **For example:** .Net languages (Visual Basic, C#), PHP, Java, Objective C, Python and a lot of other programming language.

SQLite is written in **ANSI-C** and provides simple and easy-to-use API.

SQLite is available on UNIX (Linux, Mac OS-X, Android, iOS) and Windows (Win32, WinCE, WinRT).



SQLite Commands

SQLite commands are similar to SQL commands. There are three types of SQLite commands:

- **DDL:** Data Definition Language
- **DML:** Data Manipulation Language
- **DQL:** Data Query Language

SQLite Commands

Data Definition Language

There are three commands in this group:

CREATE: This command is used to create a table, a view of a table or other object in the database.

ALTER: It is used to modify an existing database object like a table.

DROP: The DROP command is used to delete an entire table, a view of a table or other object in the database.

Data Manipulation language

There are three commands in data manipulation language group:

INSERT: This command is used to create a record.

UPDATE: It is used to modify the records.

DELETE: It is used to delete records.

Data Query Language

SELECT: This command is used to retrieve certain records from one or more table.

SQLite dot Command

SQLite dot commands are not terminated by a semicolon (;).

EX: .help command:



SQLite Data Types

SQLite data types are used to specify type of data of any object. Each column, variable and expression has related data type in SQLite. These data types are used while creating table. SQLite uses a more general dynamic type system. In SQLite, the datatype of a value is associated with the value itself, not with its container.

Types of SQLite data types

SQLite Storage Classes

The stored values in a SQLite database has one of the following storage classes

Storage Class	Description
NULL	It specifies that the value is a null value.
INTEGER	It specifies the value is a signed integer, stored in 1, 2, 3, 4, 6, or 8 bytes depending on the magnitude of the value.
REAL	It specifies the value is a floating point value, stored as an 8-byte IEEE floating point number.
text	It specifies the value is a text string, stored using the database encoding (utf-8, utf-16be or utf-16le)
BLOB	It specifies the value is a blob of data, stored exactly as it was input.



SQLite Create Table

In SQLite, CREATE TABLE statement is used to create a new table. While creating the table, we name that table and define its column and data types of each column.

Syntax:

```
CREATE TABLE database_name.table_name(
    column1 datatype PRIMARY KEY(one or more columns),
    column2 datatype,
    column3 datatype,
    .....
    columnN datatype,
);
```

```
CREATE TABLE STUDENT(
    ID INT PRIMARY KEY NOT NULL,
    NAME TEXT NOT NULL,
    AGE INT NOT NULL,
    ADDRESS CHAR(50),
    FEES REAL
);
```



SQLite Drop Table

In SQLite, DROP TABLE statement is used to remove a table definition and all associated data, indexes, triggers, constraints and permission specifications associated with that table.

Syntax:

DROP TABLE database_name.table_name;

DROP TABLE STUDENT;

SQLite Insert Query

In SQLite, INSERT INTO statement is used to add new rows of data into a table. After creating the table, this command is used to insert data into the table.

There are two types of basic syntaxes for INSERT INTO statement:

Syntax1:

INSERT INTO TABLE_NAME [(column1, column2, column3,...columnN)]

VALUES (value1, value2, value3,...valueN);

INSERT INTO STUDENT (ID,**NAME**,AGE,ADDRESS,FEES)

VALUES (1, 'Ajeet', 27, 'Delhi', 20000.00);

Second Method:

You can also insert the data into the table by second method.

INSERT INTO STUDENT **VALUES** (6, 'Kanchan', 21, 'Meerut', 10000.00);



You can see the output by using the SELECT statement:

SELECT * FROM STUDENT **SQLite UPDATE Query**

In SQLite, UPDATE query is used to modify the existing records in a table. It is used with WHERE clause to select the specific row otherwise all the rows would be updated.

Syntax:

UPDATE table_name

SET column1 = value1, column2 = value2..., columnN = valueN

WHERE [condition];

UPDATE STUDENT **SET** ADDRESS = 'Noida' **WHERE** ID = 1;

SQLite DELETE Query

In SQLite, DELETE query is used to delete the existing records from a table. You can use it with WHERE clause or without WHERE clause. WHERE clause is used to delete the specific records (selected rows), otherwise all the records would be deleted.

DELETE FROM table_name

WHERE [conditions.....];;

DELETE FROM STUDENT **WHERE** ID = 4;



Connection of python with SQLite

To connect SQLite with Python, you do not need to install the connection module separately because its being shipped by default along with Python version 2.5.x onwards.

SQLite with Python

Create a python file "connect.py", having the following code:

```
#!/usr/bin/python
```

```
import sqlite3
```

```
conn = sqlite3.connect('jvatpoint.db')
```

```
print "Opened database successfully"
```

Now connection is created with the jvatpoint database. Now you can create a table.

Create a table

Create a table "Employees" within the database "javatpoint".

Create a python file "createtable.py", having the following code:

```
#!/usr/bin/python
```

```
import sqlite3
```

```
conn = sqlite3.connect('javatpoint.db')
```

```
print "Opened database successfully";
```

```
conn.execute("""CREATE TABLE Employees
```

```
(ID INT PRIMARY KEY NOT NULL,
```

```
NAME TEXT NOT NULL,
```

```
AGE INT NOT NULL,
```

```
ADDRESS CHAR(50),
```

```
SALARY REAL);""")
```

```
print "Table created successfully";
```

```
conn.close()
```



Insert Records

Insert some records in "Employees" table.

Create a python file "connection.py", having the following code:

```
#!/usr/bin/python
```

```
import sqlite3
```

```
conn = sqlite3.connect('javatpoint.db')
```

```
print "Opened database successfully";
```

```
conn.execute("INSERT INTO Employees (ID,NAME,AGE,ADDRESS,SALARY) \
VALUES (1, 'Ajeet', 27, 'Delhi', 20000.00 );");
```

```
conn.execute("INSERT INTO Employees (ID,NAME,AGE,ADDRESS,SALARY) \
VALUES (2, 'Allen', 22, 'London', 25000.00 );");
```

```
conn.execute("INSERT INTO Employees (ID,NAME,AGE,ADDRESS,SALARY) \
VALUES (3, 'Mark', 29, 'CA', 200000.00 );");
```

```
conn.commit()
```

```
print "Records inserted successfully";
```

```
conn.close()
```



Select Records

Now you can fetch and display your records from the table "Employees" by using SELECT statement.

```
import sqlite3
```

```
conn = sqlite3.connect('javatpoint.db')
```

```
data = conn.execute("select * from Employees");
```

```
for row in data:
```

```
    print "ID = ", row[0]
```

```
    print "NAME = ", row[1]
```

```
    print "ADDRESS = ", row[2]
```

```
    print "SALARY = ", row[3], "\n"
```

```
conn.close();
```



References

- www.w3schools.com
- www.tutorialpoint.com
- Python Programming – Pearson – Taneja Kumar

Thank You
Any Query